Documentație Telefonie – II

Student,
Mădălin GAVRILAȘ
Grupa 2232/1

Profesor coordonator,
Conf.dr.ing. Zsolt POLGAR
Sl.dr.ing. Zsuzsanna SUTA

Cuprins

1. Script Unu – Pulse Coded Modulation (PCM) 4

Descrierea teoretică și înțelegerea următoarelor aspecte: PCM uniform, PCM neuniform, legile de compresie A și μ. 4

Implementarea în Matlab: PCM uniform și neuniform folosind ambele legi de codare A și μ. .. 6

Analiza Implementării Filtrării Semnalului Audio 6

Analiza Implementării Cuantizării Uniforme PCM 9

Analiza Implementării Cuantizării Neuniforme PCM 11

Implementarea unei interfețe grafice 13

Descriere Generală a Funcționării Interfeței GUI pentru Utilizatori 18

Elementele Interfeței Grafice și Funcționalitățile 19

Prezentare interfata grafica: 20

2. Script Doi – Modulația Delta 27

Descrierea teoretică și înțelegerea următoarelor aspecte: modulația/demodulația Delta fixă, modulația/demodulația Delta adaptivă Song și Jayant. 27

Modulația / Demodulația Delta Fixă 28

Modulația / Demodulația Delta Adaptivă Song și Jayant 28

Algoritmul Song 28

Algoritmul Jayant 29

Implementările în Matlab: modulația/demodulația Delta fixă, modulația/demodulația Delta adaptivă Song și Jayant. 30

Analiza Modulatiei / Demodulatiei Delta Fixa 30

Analiza Modulației / Demodulației Delta Adaptivă Song și Jayant 35

Implementarea unei interfețe grafice 46

Exemple pentru modulatia Delta Fixa 50

Exemple pentru Modulatia Delta Adaptivă Song 52

Exemple pentru Modulatia Delta Adaptivă Jayant 53

Studiul SNR-ului pentru modulația Delta fixă pentru un semnal periodic și un semnal audio pe baza valorii pasului de cuantizare. 55

3. Script Trei - PCM neuniform folosind tabelele de codare 57

Explicarea interfeței PCM neuniform 57

Implementarea unei interfețe grafice 57

Componente și Funcționalități Cheie 57

Descrierea și interpretarea rezultatelor cu semnal sinusoidal pentru minim 2 exemple. 58

Exemplul 1: 58

Exemplul 2: 59

Exemplul 3: 60

Anexă – Script 3 61

1. Script Unu – Pulse Coded Modulation (PCM)

Descrierea teoretică și înțelegerea următoarelor aspecte: PCM uniform, PCM neuniform, legile de compresie A și μ.

Pulse Coded Modulation (PCM), standardizată sub ITU-T G.711, reprezintă o tehnică fundamentală de codare a semnalului vocal, esențială în arhitectura rețelelor telefonice digitale fixe. Această metodă a facilitat tranziția de la sistemele analogice la cele digitale, oferind avantaje semnificative în ceea ce privește calitatea semnalului, imunitatea la zgomot și flexibilitatea în multiplexare și comutație. Prin conversia semnalelor analogice în format digital, PCM permite o transmisie eficientă și robustă a vocii pe distanțe extinse și prin medii variate, contribuind la îmbunătățirea fiabilității infrastructurilor de telecomunicații moderne.

Procesul de conversie analog-digitală în PCM implică o serie de etape secvențiale:

- **Filtrarea Semnalului Analogic:** Limitează banda de frecvență a semnalului vocal la intervalul telefonic standard (300 Hz - 3400 Hz), prevenind fenomenul de aliere prin eliminarea componentelor de frecvență superioare jumătății frecvenței de eșantionare (frecvența Nyquist), de obicei în jurul valorii de 4 kHz.

- **Eșantionarea:** Semnalul analogic este eșantionat la o frecvență de 8 kHz, respectând teorema Nyquist-Shannon pentru a permite recuperarea integrală a informației originale.

- **Cuantizarea:** Valorile continue ale eșantioanelor sunt transformate în valori discrete, mapând fiecare eșantion la cel mai apropiat nivel dintr-un set finit, introducând o eroare numită zgomot de cuantizare.

- **Codarea Binară:** Fiecare nivel cuantizat primește un cod binar unic. În sistemele telefonice standard, fiecare eșantion este codat cu 8 biți, permițând 256 de niveluri discrete și rezultând un debit binar standard de 64 kbps per canal vocal.

PCM uniform

Cuantizarea uniformă în PCM utilizează un pas constant de cuantizare pe întreaga gamă dinamică a semnalului, împărțind-o în intervale de lățime egală. Deși simplă, această metodă generează un zgomot de cuantizare distribuit uniform. Pentru a menține zgomotul la un nivel acceptabil, în special la amplitudini mici unde urechea umană este foarte sensibilă, este necesar un pas de cuantizare foarte mic. Aceasta implică un număr mare de biți per eșantion și un debit binar ridicat, făcând-o ineficientă perceptual pentru semnalele vocale, unde zgomotul la niveluri scăzute este mult mai deranjant decât la niveluri înalte.

PCM neuniform

Cuantizarea neuniformă în PCM, definită de ITU-T G.711, este esențială pentru optimizarea calității vocii, ținând cont de sensibilitatea logaritmică a urechii umane. Aceasta alocă mai multe niveluri de cuantizare (pas mai mic) semnalelor de amplitudine mică, unde urechea este mai sensibilă, și mai puține niveluri (pas mai mare) semnalelor de amplitudine mare, unde toleranța la zgomot este mai ridicată.

Această optimizare se realizează prin **compandare**, un proces de compresie la emițător și expandare la receptor. La emițător, o funcție neliniară amplifică selectiv semnalele de nivel scăzut, comprimând gama dinamică, înainte de cuantizarea uniformă. La receptor, o funcție inversă de expandare restabilește dinamica originală a semnalului, distribuind zgomotul de cuantizare conform sensibilității urechii. Implementarea digitală a compandării, prin conversia inițială pe un număr mare de biți (ex. 13 sau 14 biți) urmată de operații numerice pentru reducerea la 8 biți, a înlocuit implementările analogice, oferind precizie și robustețe superioare.

Legea de Compresie μ

Legea de compresie μ, standardizată cu parametrul μ=255, este predominantă în sistemele telefonice digitale din SUA și Japonia. Aceasta utilizează o funcție logaritmică pentru a comprima dinamic semnalul, alocând o rezoluție mai mare semnalelor de amplitudine mică. Pentru implementare, caracteristica continuă este aproximată prin 16 segmente liniare (8 pozitive, 8 negative), fiecare subdivizat în 16 subsegmente. Pe axa de ieșire, segmentele au lățime egală (cuantizare uniformă pe 8 biți), în timp ce pe axa de intrare, pasul de cuantizare crește exponențial cu indexul segmentului, realizând cuantizarea neuniformă. Această metodă oferă o calitate percepută comparabilă cu o cuantizare uniformă pe 14 biți, dar la un debit binar redus**.**

Codificarea pe 8 biți include un bit de semn, 3 biți pentru segment și 4 biți pentru subsegment. Pentru a optimiza sincronizarea și a preveni secvențele lungi de zero, cei 8 biți sunt inversați înainte de transmisie, iar codul de linie B8ZS (Bipolar with 8 Zeros Substitution) este utilizat. Un aspect specific este tratarea nivelului zero, unde codul 11111111b (FFh) este înlocuit cu 01111111b (7Fh) pentru a evita confuzia cu semnalele de alarmă.

Legea de Compresie A

Legea de compresie A este un standard esențial în codarea vocală digitală, adoptat pe scară largă în Europa, definită matematic printr-o funcție pe porțiuni cu parametrul A = 87.6, asigurând continuitatea funcției de compresie. Caracteristica sa este aproximată prin 7 segmente liniare (pozitive și negative), delimitate de valori de forma $1/2^n$ (unde n variază de la 6 la 0), cu primul și al doilea segment situate pe aceeași dreaptă, oferind o rezoluție fină la niveluri foarte scăzute. Primul segment este compus din 32 de subsegmente, iar restul segmentelor din 16 subsegmente, putând fi împărțit conceptual în două segmente identice de câte 16 subsegmente pentru a menține o structură de codare similară cu legea μ. Această lege echivalează cu o cuantizare uniformă pe 13 biți pentru o calitate percepută similară la un debit de date optimizat.

Procesul de codare PCM cu compresie conform legii A implică o cuantizare uniformă inițială pe 13 biți. Conversia la codul PCM pe 8 biți se realizează printr-un tabel de căutare specific. Spre deosebire de legea μ, biții codului pe 8 biți sunt inversați doar pe pozițiile pare înainte de transmisie, iar bitul de semn pentru eșantioanele pozitive este 1, iar pentru cele negative este 0. Codul de linie asociat legii A este HDB3 (High Density Bipolar 3 Zeros), o variantă modificată a codului AMI, concepută pentru a preveni secvențele lungi de zerouri și a asigura o recuperare fiabilă a ceasului. Implementarea digitală este simplificată, nefiind necesară adăugarea unei valori zecimale, deoarece capetele segmentelor sunt puteri ale lui 2.

Implementarea în Matlab: PCM uniform și neuniform folosind ambele legi de codare A și μ.

Analiza Implementării Filtrării Semnalului Audio

Codul MATLAB prezentat realizează filtrarea unui semnal audio, analiza spectrală a semnalului original și filtrat, precum și adăugarea de zgomot controlat pentru a evalua performanța filtrului în condiții de zgomot. Implementarea se concentrează pe aplicarea unui filtru trece-bandă, vizualizarea semnalului în domeniul timp și frecvență, și evaluarea raportului semnal-zgomot (SNR).

Încărcarea și Analiza Inițială a Semnalului Audio

```
close all; % inchide toate ferestrele grafice existente
clear all; % sterge toate variabilele din workspace
clc; % curata fereastra de comenzi

[voice,fe]=audioread('Issaaaaa.wav'); % citeste fisierul audio de test si obtine semnalul si frecventa de esantionare

t=(0:length(voice)-1)/fe; % creeaza vectorul de timp corespunzator lungimii semnalului

figure
fig0=gcf; % salveaza figura curenta in fig0
plot(t,voice) % afiseaza semnalul audio initial in timp
hold on

sound(voice,fe) % reda semnalul original in difuzor

[P,f]=analiz_sp(voice,fe); % calculeaza spectrul semnalului original
figure
fig1=gcf;
plot(f,P) % afiseaza spectrul semnalului
hold on

figure
fig2=gcf;
plot(f,mag2db(P)) % afiseaza spectrul in decibeli
hold on
```

Această secțiune începe prin citirea fișierului audio specificat ('Issaaaaa.wav') folosind funcția audioread, ceea ce permite obținerea semnalului audio propriu-zis, stocat în variabila voice, și a frecvenței de eșantionare a acestuia, stocată în variabila fe. Ulterior, se creează un vector de timp (t) care corespunde lungimii semnalului audio, esențial pentru reprezentarea corectă a semnalului în domeniul timp.

Semnalul audio original este vizualizat grafic folosind funcția plot, care afișează amplitudinea semnalului în funcție de timp, iar funcția sound este utilizată pentru a reda semnalul audio prin difuzoare, permițând ascultarea directă a semnalului inițial.

În final, se calculează spectrul semnalului original utilizând funcția analiz_sp, (a cărei implementare este presupusă a fi definită în altă parte), iar spectrul rezultat este afișat grafic, atât în termeni de magnitudine, cât și în decibeli, pentru a oferi o perspectivă completă asupra conținutului său frecvențial.

Filtrarea Semnalului Audio

```
voice_f=bandpass(voice,[300,3400],fe,'Steepness',0.8); % filtreaza semnalul intre 300-3400Hz (banda vocala)
figure(fig0)
plot(t,voice_f) % suprapune semnalul filtrat peste cel original

[Pf,f]=analiz_sp(voice_f,fe); % spectru pentru semnalul filtrat
figure(fig1)
plot(f,Pf) % suprapune spectrul filtrat
figure(fig2)
plot(f,mag2db(Pf)) % suprapune spectrul in dB filtrat
```

În această parte a codului, se realizează filtrarea semnalului audio pentru a izola componentele de interes. Funcția bandpass este aplicată semnalului (voice), cu parametrii specificați pentru a reține doar frecvențele din banda vocală, cuprinsă între 300 Hz și 3400 Hz. Parametrul 'Steepness' controlează panta de tranziție a filtrului. Semnalul filtrat (voice_f) este apoi suprapus peste semnalul original pe același grafic temporal, oferind o comparație vizuală directă a efectului filtrării. Ulterior, se calculează spectrul semnalului filtrat (Pf) folosind din nou funcția analiz_sp, iar acest spectru este afișat atât în termeni de magnitudine, cât și în decibeli, permițând o analiză comparativă a conținutului frecvențial înainte și după filtrare.

Adăugarea de Zgomot la Semnal

```
% SNR controlat: se adauga zgomot alb astfel incat sa existe un SNR de 20dB
SNR=20; % definim raportul semnal-zgomot dorit (20 dB)
P_sig=rms(voice)^2; % calculam puterea semnalului original folosind valoarea RMS (Root Mean Square)
P_sig_n_db=10*log10(P_sig)-SNR; % calculam nivelul de putere al zgomotului necesar pentru a atinge SNR-ul dorit (in dB)
voice_noise=wgn(1,length(voice),P_sig_n_db) + voice'; % generam zgomot alb Gaussian cu puterea calculata si il adaugam la semnalul original

[P_n,f]=analiz_sp(voice_noise,fe); % calculam spectrul semnalului zgomotos
figure
fig0n=gcf; % salvam figura curenta
plot(t,voice_noise) % afisam semnalul zgomotos in domeniul timp
hold on

figure
fig1n=gcf;
plot(f,P_n) % afisam spectrul semnalului zgomotos
hold on

figure
fig2n=gcf;
plot(f,mag2db(P_n)) % afisam spectrul semnalului zgomotos in decibeli
hold on
```

Această primă parte a secțiunii, se simulează condiții de transmisie cu zgomot prin adăugarea de zgomot alb Gaussian la semnalul audio original. Se definește un raport semnal-zgomot (SNR) țintă de 20 dB. Apoi, se calculează puterea semnalului original (P_sig) folosind funcția rms, iar nivelul de putere al zgomotului necesar pentru a obține SNR-ul dorit este determinat. Zgomotul alb Gaussian este generat cu funcția wgn și adăugat la semnalul original, rezultând semnalul zgomotos (voice_noise).

În continuare, acest semnal zgomotos este analizat în domeniul timp și frecvență, utilizând funcția analiz_sp pentru a calcula spectrul (P_n) și funcțiile plot și mag2db pentru vizualizarea rezultatelor.

Filtrarea Semnalului Zgomotos și Redarea Audio

```
voice_noise_f=bandpass(voice_noise,[300,3400],fe,'Steepness',0.8); % filtreaza semnalul cu zgomot

[P_n_f,f]=analiz_sp(voice_noise_f,fe); % calculeaza spectrul semnalului filtrat cu zgomot
figure(fig0n)
plot(t,voice_noise_f) % afiseaza semnalul filtrat cu zgomot in domeniul timp

figure(fig1n)
plot(f,P_n_f) %a fiseaza spectrul semnalului filtrat cu zgomot

figure(fig2n)
plot(f,mag2db(P_n_f)) % afiseaza spectrul semnalului filtrat cu zgomot in decibeli

sound(voice_noise,fe) % reda semnalul cu zgomot
pause((length(voice_noise)+10)/fe)
sound(voice_noise_f,fe) % reda semnalul filtrat cu zgomot
pause((length(voice_noise)+10)/fe)
```

În această a doua parte a secțiunii, se aplică filtrul trece-bandă semnalului zgomotos (voice_noise) pentru a reduce efectul zgomotului. Semnalul filtrat cu zgomot (voice_noise_f) este apoi analizat în domeniul timp și frecvență, similar cu etapele anterioare.

În final, pentru a permite o evaluare auditivă a impactului zgomotului și a eficacității filtrului, atât semnalul zgomotos inițial, cât și cel filtrat cu zgomot sunt redate folosind funcția sound, cu o scurtă pauză între redări, pentru a facilita compararea.

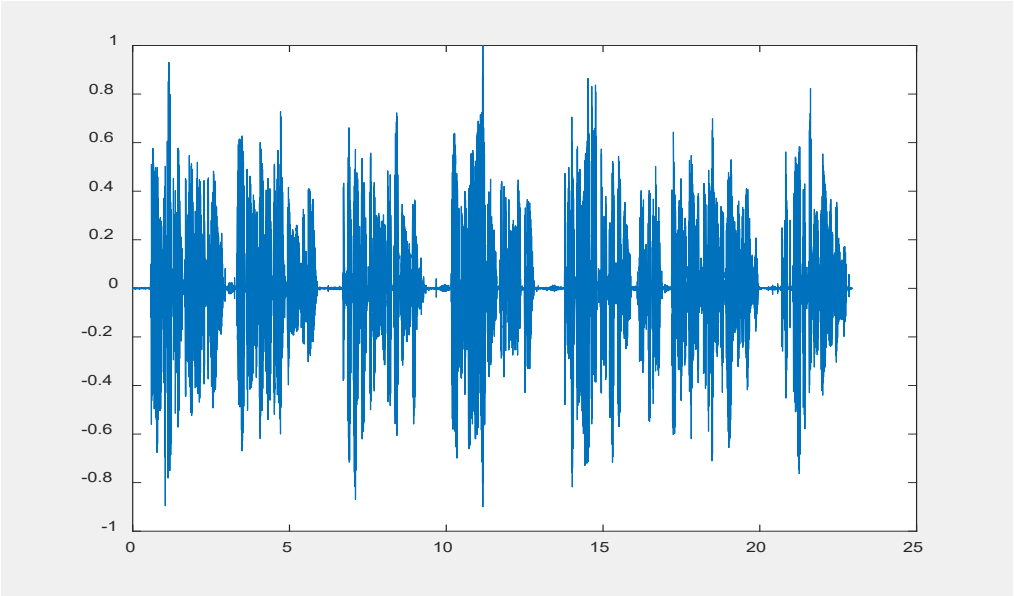

Figura 1.1. Reprezentarea Semnalului Audio în Domeniul Timp

Analiza Implementării Cuantizării Uniforme PCM

Codul MATLAB prezentat implementează procesul de cuantizare uniformă PCM (Pulse Code Modulation) pentru un semnal audio. Scopul principal este de a discretiza amplitudinea semnalului analogic, pregătindu-l pentru transmiterea sau stocarea digitală. Codul dezvoltă un semnal audio, îl cuantizează și apoi compară semnalul original cu cel cuantizat, calculând și raportul semnal-zgomot (SNR).

Inițializare și Încărcare Semnal

```
clear all;
close all;
clc;

[signal,fs]=audioread('BabyElephantWalk60.wav'); % citeste fisier audio
signal = signal(25*fs:25*fs+fs*15,1); % extrage o portiune de 15 secunde din semnal
signal = signal'; % transpune semnalul in vector linie
```

Funcția audioread este folosită pentru a citi fișierul audio 'BabyElephantWalk60.wav', stocând semnalul în variabila signal și frecvența de eșantionare în variabila fs. O porțiune specifică de 15 secunde din semnal este extrasă, începând de la secunda 25, pentru a focaliza analiza pe un segment relevant. În final, semnalul este transpus într-un vector linie cu ajutorul operatorului de transpunere ('), pregătindu-l pentru procesările ulterioare.

Parametrii de Cuantizare

```
nbit = 8; % numar de biti pentru cuantizare

Vmax = max(signal); % valoarea maxima din semnal
Vmin = min(signal); % valoarea minima din semnal

V = max(abs(Vmax), abs(Vmin)); % valoarea absoluta maxima pentru cuantizare

n_interval = 2^nbit; % numar de intervale de cuantizare

delta = 2*V / n_interval; % marimea pasului de cuantizare

partition = -V + delta : delta : +V - delta; % limitele intervalelor de cuantizare
codebook = -V + delta/2 : delta : +V - delta/2; % valorile reprezentative pentru cuantizare
```

În această parte a codului, sunt definiți parametrii esențiali pentru procesul de cuantizare uniformă. Variabila nbit stabilește numărul de biți utilizați pentru cuantizare (8 în acest caz), ceea ce determină rezoluția cuantizării.

Valorile maxime și minime ale semnalului, V_{max} și V_{min}, sunt calculate pentru a stabili intervalul de cuantizare. Valoarea absolută maximă a semnalului, V, este determinată pentru a asigura acoperirea întregului interval de valori ale semnalului. Numărul total de intervale de cuantizare, n_interval, este calculat ca 2 la puterea nbit. Dimensiunea pasului de cuantizare, delta, este calculată împărțind intervalul total de valori la numărul de intervale.

Vectorul partition definește limitele intervalelor de cuantizare, iar vectorul codebook conține valorile reprezentative pentru fiecare interval, care vor fi atribuite eșantioanelor cuantizate.

Cuantizarea Semnalului

```
[index, quants, distor] = quantiz(signal, partition, codebook); % aplica cuantizarea

figure,
stairs(signal), % afiseaza semnalul original
hold on,
stairs(quants), % afiseaza semnalul cuantizat
disp('PCM Uniform - Cuantizare finalizata')
legend('original', 'quantized');
```

Codul implementează efectiv cuantizarea semnalului și vizualizează rezultatele. Funcția quantiz realizează cuantizarea, returnând indicii intervalelor de cuantizare (index), valorile cuantizate ale semnalului (quants) și distorsiunea introdusă de cuantizare (distor).

Graficele semnalului original și cuantizat sunt create folosind funcția stairs, care este adecvată pentru a reprezenta semnalele discrete. Comanda hold on permite suprapunerea celor două grafice pe aceeași figură pentru comparație directă. Un mesaj este afișat în fereastra de comandă pentru a confirma finalizarea procesului de cuantizare, iar o legendă este adăugată la grafic pentru a clarifica ce reprezintă fiecare linie.

Calculul Raportului Semnal-Zgomot (SNR)

```
P_signal = rms(signal)^2; % puterea semnalului
P_err = rms(signal-quants)^2; % puterea erorii de cuantizare
SNR = 10*log10(P_signal/P_err); % raport semnal-zgomot
```

În continuare se calculează raportul semnal-zgomot (SNR), o măsură importantă a calității semnalului cuantizat. Puterea semnalului original (P_signal) este calculată folosind valoarea RMS, iar puterea erorii de cuantizare (P_err) este calculată similar, folosind diferența dintre semnalul original și cel cuantizat.

Raportul semnal-zgomot (SNR) este apoi calculat în decibeli (dB) utilizând formula 10*log10(P_signal/P_err), oferind o indicare a cât de puternic este semnalul util în comparație cu zgomotul introdus prin cuantizare.

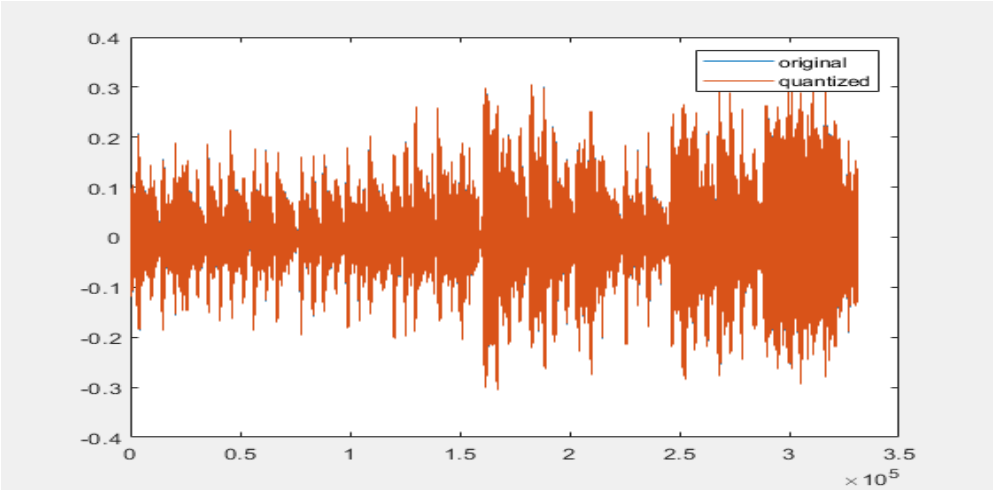

Figura 1.2. Reprezentarea Grafică a Semnalului Original și a Semnalului Cuantizat

Analiza Implementării Cuantizării Neuniforme PCM

Codul MATLAB prezentat implementează procesul de cuantizare neuniformă PCM, folosind compresia și expansiunea legea A, pentru un semnal audio. Scopul principal este de a îmbunătăți eficiența cuantizării prin alocarea mai multor niveluri de cuantizare zonelor de amplitudine a semnalului care apar mai frecvent, cum ar fi valorile mici. Codul încarcă un semnal audio, îl comprimă, îl cuantizează, îl expandează și apoi compară semnalele, calculând și rapoartele semnal-zgomot (SNR).

Inițializare și Încărcare Semnal

```
clear all;
close all;
clc;

[signal,fs]=audioread('BabyElephantWalk60.wav'); % citeste fisier audio
signal = signal(25*fs:25*fs+fs*15,1); % extrage o portiune de 15 secunde din semnal
signal = signal'; % transpune semnalul in vector linie
```

Codul se ocupă de încărcarea semnalului audio. Funcția audioread este folosită pentru a citi fișierul audio 'BabyElephantWalk60.wav', stocând semnalul în variabila signal și frecvența de eșantionare în variabila fs. O porțiune specifică de 15 secunde din semnal este extrasă, începând de la secunda 25, pentru a focaliza analiza pe un segment relevant. În final, semnalul este transpus într-un vector linie cu ajutorul operatorului de transpunere ('), pregătindu-l pentru procesările ulterioare.

Parametrii de Cuantizare

```
nbit = 8;        % numar de biti pentru cuantizare

Vmax = max(signal);  % valoarea maxima din semnal
Vmin = min(signal);  % valoarea minima din semnal
V = max(abs(Vmax), abs(Vmin));  % valoarea absoluta maxima pentru cuantizare

n_interval = 2^nbit;  % numar de intervale de cuantizare

delta = 2*V / n_interval;  % marimea pasului de cuantizare
partition = -V + delta : delta : +V - delta;  % partitii cuantizare
codebook = -V + delta/2 : delta : +V - delta/2;  % coduri pentru cuantizare
```

Mai sus sunt definiți parametrii esențiali pentru procesul de cuantizare. Variabila nbit stabilește numărul de biți utilizați pentru cuantizare (8 în acest caz), ceea ce determină rezoluția cuantizării.

Valorile maxime și minime ale semnalului, Vmax și Vmin, sunt calculate pentru a stabili intervalul de cuantizare. Valoarea absolută maximă a semnalului, V, este determinată pentru a asigura acoperirea întregului interval de valori ale semnalului. Numărul total de intervale de cuantizare, n_interval, este calculat ca 2 la puterea nbit. Dimensiunea pasului de cuantizare, delta, este calculată împărțind intervalul total de valori la numărul de intervale.

Vectorul partition definește limitele intervalelor de cuantizare, iar vectorul codebook conține valorile reprezentative pentru fiecare interval, care vor fi atribuite eșantioanelor cuantizate.

Compresie Cuantizare și Expansiune

```
signal_comp = compand(signal, 87.6, V, 'A/compressor'); % compresie folosind legea A
[index, quants, distor] = quantiz(signal_comp, partition, codebook); % cuantizare pe semnal comprimat
signal_exp = compand(quants, 87.6, V, 'A/expander'); % expansiune a semnalului cuantizat

err_uni = signal_comp - quants; % eroare intre comprimat si cuantizat
err_nonuni = signal - signal_exp; % eroare totala dupa expandare
```

Vom implementa compresia semnalului audio folosind legea A, cuantizarea semnalului comprimat și expansiunea semnalului cuantizat. Funcția compand este utilizată pentru a aplica compresia și expansiunea, cu parametrul 87.6 reprezentând valoarea aproximativă a parametrului A (de obicei 87.6 pentru sistemele europene).

Semnalul comprimat este apoi cuantizat folosind funcția quantiz, similar cu cuantizarea uniformă, iar semnalul cuantizat este expandat pentru a reveni la domeniul original. Erorile introduse de compresie și cuantizare (err_uni) și eroarea totală după expansiune (err_nonuni) sunt calculate pentru a evalua performanța sistemului.

Vizualizarea Semnalelor și Calculul SNR

```
figure,
stairs(signal), % semnal original
hold on,
stairs(signal_comp), % semnal comprimat
stairs(quants), % semnal cuantizat
stairs(signal_exp), % semnal expandat
stairs(err_uni), % eroare compandare
stairs(err_nonuni), % eroare totala
legend('original', 'compressed', 'quantized', 'expanded', 'err uni', 'err nonuni');

P_signal = rms(signal)^2; % putere semnal original
P_err_uni = rms(err_uni)^2; % putere eroare compresie-cuanti
P_err_nonuni = rms(err_nonuni)^2; % eroare totala (comp+quant+exp)

SNR_uni = 10*log10(P_signal/P_err_uni); % SNR intermediar (dupa compresie)
SNR_nonuni = 10*log10(P_signal/P_err_nonuni); % SNR final (dupa expansiune)
```

Această secțiune vizualizează grafic semnalele originale, comprimate, cuantizate și expandate, împreună cu erorile calculate. Funcția stairs este utilizată pentru a reprezenta semnalele discrete. Rapoartele semnal-zgomot (SNR) sunt calculate pentru a evalua calitatea semnalului după compresie (SNR_uni) și după expansiune (SNR_nonuni). Puterea semnalului și a erorilor este calculată folosind valoarea RMS.

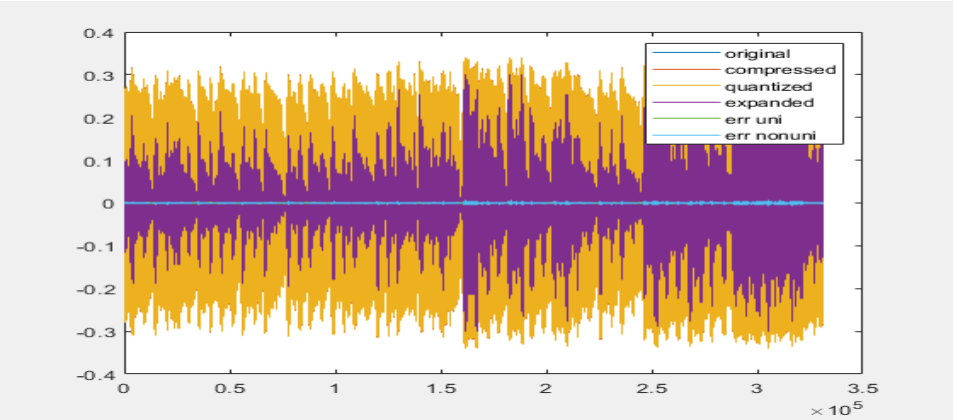

Figura 1.3. Reprezentarea Grafică a Semnalelor în Diversele Etape ale Cuantizării Neuniforme PCM cu Compresie și Expansiune Legea A

Implementarea unei interfețe grafice

```
function script1_pcm_gui()

    % ---- SETĂRI VIZUALE ----
    backgroundColor = [0.5 0.4 0.6]; % fundal mov
    textColor = [0 0 0];             % text negru
    buttonColor = [0 0.7 0];         % butoane verzi
    fontSize = 12;                   % dimensiune font redusa
    lineWidth = 1;

    % ---- FIGURA PRINCIPALĂ ----
        % creaza figura principala a GUI-ului
    f = figure('Name', 'PCM GUI', 'Units', 'normalized', 'Position', [0.05, 0.05, 0.9, 0.85], 'Color', backgroundColor);
```

Această secțiune inițializează funcția principală a interfeței grafice (script1_pcm_gui). Definește, de asemenea, setările vizuale de bază, cum ar fi culorile de fundal, text și butoane, dimensiunea fontului și grosimea liniilor pentru elementele grafice. Apoi, creează figura principală a GUI-ului, setându-i numele, unitățile de măsură, poziția și culoarea de fundal.

- **Un câmp din care se poate selecta tipul de PCM, uniform sau neuniform.**

Această parte a codului creează un panou de control în partea de sus a interfeței grafice. În interiorul acestui panou, se adaugă o etichetă text ("Tip PCM") și un meniu dropdown (popupmenu) numit pcmType. Acest meniu permite utilizatorului să aleagă între procesarea PCM uniformă și neuniformă

```
% --- CONTROALE UI DE SUS ---
    control_panel = uipanel('Parent', f, 'Title', 'Control', 'Units', 'normalized', 'Position', [0.02, 0.88, 0.96, 0.1], ...
        'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize);

    % Tip PCM
     % Eticheta pentru selectarea tipului de PCM
    uicontrol('Parent', control_panel, 'Style', 'text', 'Units', 'normalized', 'Position', [0.01, 0.55, 0.1, 0.3], 'String', 'Tip PCM:', ...
        'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, 'HorizontalAlignment', 'left');

    % Meniu dropdown pentru selectarea tipului de PCM (Uniform/Neuniform)
    pcmType = uicontrol('Parent', control_panel, 'Style', 'popupmenu', 'Units', 'normalized', 'Position', [0.11, 0.55, 0.15, 0.4], ...
        'String', {'Uniform', 'Neuniform'}, 'BackgroundColor', 'white', 'ForegroundColor', textColor, 'FontSize', fontSize);
```

- **Un câmp din care se poate selecta semnalul de intrare (undă sinusoidală sau dreptunghiulară sau triunghiulară sau dinte de fierăstrău sau fișier audio).**

Similar cu selecția tipului de PCM, această secțiune adaugă o etichetă text ("Semnal intrare") și un alt meniu dropdown (popupmenu) numit signalType. Acest meniu oferă utilizatorului opțiuni pentru a selecta tipul semnalului de intrare dorit: sinusoidal, dreptunghiular, triunghiular, dinte de fierăstrău sau un fișier audio.

```
% Semnal intrare
        % Etichetă pentru selectarea semnalului de intrare
    uicontrol('Parent', control_panel, 'Style', 'text', 'Units', 'normalized', 'Position', [0.28, 0.55, 0.13, 0.3], 'String', 'Semnal intrare:', ...
        'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, 'HorizontalAlignment', 'left');

    % Meniu dropdown pentru selectarea tipului de semnal de intrare
    signalType = uicontrol('Parent', control_panel, 'Style', 'popupmenu', 'Units', 'normalized', ...
        'Position', [0.41, 0.55, 0.15, 0.4], 'String', {'Sinusoidal', 'Dreptunghiular', 'Triunghiular', 'Dinte fierastrau', 'Audio'}, ...
        'BackgroundColor', 'white', 'ForegroundColor', textColor, 'FontSize', fontSize);
```

- **Un câmp din care se poate stabili numărul de biți pentru cuantizare.**

Aici se creează o etichetă text ("Nr. biți") și un câmp editabil (edit) numit bitsInput. Acest câmp permite utilizatorului să specifice numărul de biți care vor fi utilizați în procesul de cuantizare a semnalului. Valoarea implicită afișată în acest câmp este '8'.

```
% Număr biți
    % Etichetă pentru introducerea numărului de biți
    uicontrol('Parent', control_panel, 'Style', 'text', 'Units', 'normalized', 'Position', [0.58, 0.55, 0.09, 0.3], 'String', ...
        'Nr. biți:', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, 'HorizontalAlignment', 'left');

    % Câmp editabil pentru introducerea numărului de biți pentru cuantizare, implicit este 8
    bitsInput = uicontrol('Parent', control_panel, 'Style', 'edit', 'Units', 'normalized', ...
        'Position', [0.67, 0.55, 0.07, 0.4], 'String', '8', 'BackgroundColor', 'white', ...
        'ForegroundColor', textColor, 'FontSize', fontSize);
```

- **Un câmp din care se poate selecta legea de compresie A sau μ (doar pentru PCM neuniform).**

Această secțiune introduce o etichetă text ("Lege compresie (A/μ):") și un meniu dropdown (popupmenu) numit lawSelector. Acest meniu oferă opțiunile 'A' și 'μ' pentru selectarea legii de compresie care va fi aplicată în cazul în care utilizatorul a ales procesarea PCM neuniformă.

```
% Lege compresie (pentru PCM neuniform)
      % Etichetă pentru selectarea legii de compresie
 uicontrol('Parent', control_panel, 'Style', 'text', 'Units', 'normalized', 'Position', [0.01, 0.05, 0.17, 0.3], ...
     'String', 'Lege compresie (A/μ):', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
     'FontSize', fontSize, 'HorizontalAlignment', 'left');

     % Meniu dropdown pentru selectarea legii de compresie (A sau μ) pentru PCM neuniform
 lawSelector = uicontrol('Parent', control_panel, 'Style', 'popupmenu', 'Units', 'normalized', ...
     'Position', [0.18, 0.05, 0.13, 0.4], 'String', {'A', 'μ'}, 'BackgroundColor', 'white', ...
     'ForegroundColor', textColor, 'FontSize', fontSize);
```

- **Un buton pentru generarea semnalelor selectate.**

Aici este creat un buton (pushbutton) cu eticheta "Generează". Atunci când utilizatorul apasă acest buton, este apelată funcția genereazaSemnal. Această funcție va prelua setările selectate (tip PCM, semnal de intrare, număr de biți, lege de compresie) și va genera semnalele corespunzătoare pentru vizualizare și procesare.

```
    % Buton generare semnal

        % Buton pentru a genera semnalul pe baza selecțiilor făcute
     uicontrol('Parent', control_panel, 'Style', 'pushbutton', 'Units', 'normalized', 'Position', [0.33, 0.05, 0.1, 0.4], ...
         'String', 'Generează', 'Callback', @genereazaSemnal, 'BackgroundColor', buttonColor, ...
         'ForegroundColor', textColor, 'FontSize', fontSize);
```

- **Vizualizarea semnalelor originale și a celor cuantizate în domeniul timp (ex. stairs) cu legendă. Vizualizarea semnalelor după fiecare pas pentru PCM neuniform.**

Această secțiune configurează crearea axelor pentru vizualizarea semnalelor în domeniul timp. Am ales să nu includ tot codul aferent creării tuturor celor șase grafice aici, pentru a menține concizia. În realitate, codul continuă în mod similar pentru a defini poziția, dimensiunea, culorile, titlurile și etichetele fiecărei axe.

Pentru PCM uniform, vor fi utilizate graficele pentru semnalul original (ax1) și cel cuantizat (ax2), unde cel cuantizat va fi afișat folosind funcția stairs.

Pentru PCM neuniform, se vor afișa suplimentar semnalul comprimat (ax_comp) și cel expandat (ax_exp), permițând vizualizarea fiecărui pas al procesului. Fiecare grafic este configurat cu titlu și etichete ale axelor pentru o interpretare ușoară

```
% --- AXE PENTRU VIZUALIZARE (PRIMELE 6 GRAFICE) ---
vertical_space = 0.07; % Spațiere verticală între grafice
horizontal_space = 0.04; % Spațiere orizontală între grafice
plot_width = 0.45;       % Lățimea graficelor
plot_height = 0.14;      % Înălțimea graficelor
y_top = 0.82;            % Poziția de sus pentru prima linie de grafice
title_y_offset = 1.05;
```

```
% Linia 1
% Axă semnal original
ax1_x = 0.03;
ax1_y = y_top - plot_height;
ax1 = axes('Units', 'normalized', 'Position', [ax1_x, ax1_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul original
title_obj = title(ax1, 'Semnal Original', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax1, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax1, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax1, 'on'); set(ax1, 'GridColor', textColor); % Activează grila
```

```
% Axă semnal comprimat (neuniform)
ax_comp_x = ax1_x + plot_width + horizontal_space;
ax_comp_y = ax1_y;
ax_comp = axes('Units', 'normalized', 'Position', [ax_comp_x, ax_comp_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul comprimat
title_obj = title(ax_comp, 'Semnal Comprimat', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_comp, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_comp, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_comp, 'on'); set(ax_comp, 'GridColor', textColor); % Activează grila
```

```
% Linia 2
% Axă semnal cuantizat
ax2_x = 0.03;
ax2_y = y_top - plot_height;
ax2 = axes('Units', 'normalized', 'Position', [ax2_x, ax2_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul cuantizat
title_obj = title(ax2, 'Semnal Cuantizat', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax2, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax2, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax2, 'on'); set(ax2, 'GridColor', textColor); % Activează grila
```

```
% Axă semnal expandat (neuniform)
ax_exp_x = ax2_x + plot_width + horizontal_space;
ax_exp_y = ax2_y;
ax_exp = axes('Units', 'normalized', 'Position', [ax_exp_x, ax_exp_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul expandat
title_obj = title(ax_exp, 'Semnal Expandat', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_exp, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_exp, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_exp, 'on'); set(ax_exp, 'GridColor', textColor); % Activează grila
```

- **Calcularea și vizualizarea erorii de cuantizare. Calcularea, vizualizarea și compararea erorii de cuantizare în cazul cuantizării neuniforme cu cea uniformă.**

Această secțiune creează două seturi de axe suplimentare. Primul set (ax3) este destinat vizualizării erorii de cuantizare în domeniul timp. Al doilea set (ax_err_comp) va fi utilizat pentru a compara eroarea de cuantizare (în termeni de RMS - Root Mean Square) între metoda PCM uniformă și cea neuniformă, afișând probabil un grafic bară pentru această comparație.

```
y_top = y_top - plot_height - vertical_space;

% Linia 3
% Axă eroare cuantizare

ax3_x = 0.03;
ax3_y = y_top - plot_height;
ax3 = axes('Units', 'normalized', 'Position', [ax3_x, ax3_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru eroarea de cuantizare
title_obj = title(ax3, 'Eroare Cuantizare', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax3, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax3, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax3, 'on'); set(ax3, 'GridColor', textColor); % Activează grila


% Axă comparatie erori

ax_err_comp_x = ax3_x + plot_width + horizontal_space;
ax_err_comp_y = ax3_y;
ax_err_comp = axes('Units', 'normalized', 'Position', [ax_err_comp_x, ax_err_comp_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru compararea erorilor
title_obj = title(ax_err_comp, 'Comparatie Eroare', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_err_comp, 'Tip PCM', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_err_comp, 'RMS Eroare', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_err_comp, 'on'); set(ax_err_comp, 'GridColor', textColor); % Activează grila
```

- **Calcule de putere și SNR. Calcularea și compararea SNR-ului între PCM uniform și neuniform cu același număr de biți.**

Această secțiune introduce un panou numit "Rezultate" unde vor fi afișate valorile numerice pentru RMS-ul erorii și SNR (Signal-to-Noise Ratio) atât pentru metoda PCM uniformă, cât și pentru cea neuniformă. De asemenea, creează două axe suplimentare (ax4 și ax_snr_comp) care vor fi utilizate pentru a vizualiza și compara valorile SNR obținute pentru cele două metode PCM.

```
% --- PANOURI TEXT PENTRU REZULTATE NUMERICE ---
    resultsPanel_y = 0.03; % Coborâm panoul de rezultate
    resultsPanel_height = 0.25;
    resultsPanel = uipanel('Title', 'Rezultate', 'Units', 'normalized', ...
        'Position', [0.52, resultsPanel_y, 0.46, resultsPanel_height], 'BackgroundColor', backgroundColor, ...
        'ForegroundColor', textColor, 'FontSize', fontSize); % Creează un panou pentru afișarea rezultatelor numerice



    y_rez_top = 0.9;
    rez_height = 1/(4+3*0.2); % Ajustat dinamic în funcție de numărul de linii
    rez_space = 0.2 * rez_height;


    % Text pentru RMS eroare uniforma
    uicontrol('Parent', resultsPanel, 'Style', 'text', 'Units', 'normalized', ...
        'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], 'String', ...
        'RMS Eroare Uniform:', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, ...
        'HorizontalAlignment', 'left'); % Afișează eticheta pentru RMS-ul erorii uniforme
    rmsErrorUniformText = uicontrol('Parent', resultsPanel, 'Style', 'text', ...
        'Units', 'normalized', 'Position', [0.5, y_rez_top - rez_height, 0.45, rez_height], 'String', '-', ...
        'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, 'HorizontalAlignment', 'left'); % Spațiu pentru afișarea valorii RMS-ului erorii uniforme
```

Aici se creează etichete text ("RMS Eroare Neuniform:" și "SNR Uniform (dB):") și spații adiacente (rmsErrorNeuniformText, snrUniformText) în panoul "Rezultate" pentru a prezenta valorile numerice ale RMS-ului erorii pentru PCM neuniform și ale SNR-ului pentru PCM uniform, după calcul.

```
% Text pentru RMS eroare neuniforma
uicontrol('Parent', resultsPanel, 'Style', 'text', 'Units', 'normalized', ...
    'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], 'String', ...
    'RMS Eroare Neuniform:', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, ...
    'HorizontalAlignment', 'left'); % Afișează eticheta pentru RMS-ul erorii neuniforme
rmsErrorNeuniformText = uicontrol('Parent', resultsPanel, 'Style', 'text', ...
    'Units', 'normalized', 'Position', [0.5, y_rez_top - rez_height, 0.45, rez_height], 'String', '-', ...
    'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, 'HorizontalAlignment', 'left');
% Spațiu pentru afișarea valorii RMS-ului erorii neuniforme

y_rez_top = y_rez_top - rez_height - rez_space;

% Text pentru SNR uniform
uicontrol('Parent', resultsPanel, 'Style', 'text', 'Units', 'normalized', ...
    'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], 'String', 'SNR Uniform (dB):', ...
    'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, ...
    'HorizontalAlignment', 'left'); % Afișează eticheta pentru SNR-ul uniform
snrUniformText = uicontrol('Parent', resultsPanel, 'Style', 'text', ...
    'Units', 'normalized', 'Position', [0.5, y_rez_top - rez_height, 0.45, rez_height], 'String', '-', ...
    'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, 'FontSize', fontSize, ...
    'HorizontalAlignment', 'left'); % Spațiu pentru afișarea valorii SNR-ului uniform
```

Similar, această secțiune adaugă o etichetă text ("SNR Neuniform (dB)") și un spațiu (snrNeuniformText) în panoul "Rezultate" pentru a afișa valoarea SNR-ului calculat pentru metoda PCM neuniformă.

```
% Text pentru SNR neuniform
uicontrol('Parent', resultsPanel, 'Style', 'text', 'Units', 'normalized', ...
    'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], 'String', ...
    'SNR Neuniform (dB):', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left'); % Afișează eticheta pentru SNR-ul neuniform
snrNeuniformText = uicontrol('Parent', resultsPanel, 'Style', 'text', ...
    'Units', 'normalized', 'Position', [0.5, y_rez_top - rez_height, 0.45, rez_height], ...
    'String', '-', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left'); % Spațiu pentru afișarea valorii SNR-ului neuniform
```

Această secțiune definește și creează două axe grafice separate, situate sub panoul cu rezultate numerice. Prima axă (ax4) este destinată vizualizării valorii SNR obținute, iar a doua axă (ax_snr_comp) este configurată pentru a permite compararea valorilor SNR între metoda PCM uniformă și cea neuniformă, probabil printr-un grafic bară. Fiecare axă este stilizată cu culoare de fundal, dimensiune a fontului, culoare a textului axelor, titlu și grilă.

```
% --- AXE PENTRU VIZUALIZARE ---
snr_height = 0.16; % Reducem înălțimea graficelor SNR
snr_y_pos = resultsPanel_y + resultsPanel_height + 0.01;

% Axă SNR
ax4_x = 0.03;
ax4_y = 0.05;
ax4 = axes('Units', 'normalized', 'Position', [ax4_x, ax4_y, 0.22, snr_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru afișarea SNR-ului
title_obj = title(ax4, 'SNR', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * 1.1, title_pos(3)]);
ylabel(ax4, 'SNR (dB)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax4, 'on'); set(ax4, 'GridColor', textColor); % Activează grila


% Axă comparatie SNR
ax_snr_comp_x = ax4_x + 0.22 + 0.03;
ax_snr_comp_y = 0.05;
ax_snr_comp = axes('Units', 'normalized', 'Position', [ax_snr_comp_x, ax_snr_comp_y, 0.22, snr_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, 'YColor', textColor); % Creează axa pentru compararea SNR-ului
title_obj = title(ax_snr_comp, 'Comparatie SNR', 'Color', textColor, 'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * 1.1, title_pos(3)]);
ylabel(ax_snr_comp, 'SNR (dB)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_snr_comp, 'on'); set(ax_snr_comp, 'GridColor', textColor); % Activează grila
```

- **Trimiterea semnalelor originale și a celor cuantizate la placa de sunet. (ex. sound, audioplayer)**

Aici sunt declarate variabile globale care vor fi utilizate în funcția de redare audio. semnal_fe reprezintă frecvența de eșantionare a semnalului audio, setată implicit la 8000 Hz. semnal_orig va stoca semnalul original, iar semnal_cuantizat va stoca semnalul după cuantizare. Variabila player va fi un obiect audioplayer utilizat pentru a reda semnalul cuantizat prin placa de sunet.

```
% Variabile globale audio
global semnal_fe semnal_orig semnal_cuantizat player;
semnal_fe = 8000;
```

- **Un buton care controlează evenimentele start/stop ale sunetelor generate.**

Prima parte creează butonul "Start/Stop" în panoul de control. Atunci când este apăsat, acesta apelează funcția playStop. Funcția playStop verifică dacă există un semnal cuantizat. Dacă nu există un obiect audioplayer creat sau dacă acesta nu este în stare de redare, se creează un nou obiect audioplayer cu semnalul cuantizat și frecvența de eșantionare, iar redarea este inițiată.

Dacă un player există și este în stare de redare, funcția oprește redarea curentă.

```
% Buton start/stop sunet
uicontrol('Parent', control_panel, 'Style', 'pushbutton', 'Units', 'normalized', 'Position', [0.45, 0.05, 0.1, 0.4], 'String', ...
    'Start/Stop', 'Callback', @playStop, 'BackgroundColor', buttonColor, ...
    'ForegroundColor', textColor, 'FontSize', fontSize);
```

```
% ---- FUNCȚIE PLAY/STOP AUDIO ----
function playStop(~, ~)
    if isempty(semnal_cuantizat), return; end
    if isempty(player) || strcmp(player.Running, 'off')
        player = audioplayer(semnal_cuantizat, semnal_fe);
        play(player);
    else
        stop(player);
    end
end

end
```

Descriere Generală a Funcționării Interfeței GUI pentru Utilizatori

Codul MATLAB definește o funcție numită script1_pcm_gui care construiește o interfață grafică (GUI) pentru explorarea cuantizării PCM uniforme și neuniforme a diferitelor tipuri de semnale audio.

GUI-ul permite utilizatorului să selecteze tipul de procesare PCM (uniform sau neuniform), să aleagă semnalul de intrare (sinusoidal, dreptunghiular, triunghiular, dinte de fierăstrău sau audio dintr-un fișier), să stabilească numărul de biți pentru cuantizare și, în cazul PCM neuniform, să selecteze legea de compresie (A sau μ).

După configurare, utilizatorul poate genera semnalul procesat, vizualiza semnalele originale și cuantizate (și semnalele intermediare pentru PCM neuniform) în domeniul timp, observa eroarea de cuantizare, calcula și compara valorile RMS ale erorii și SNR între cele două metode PCM, și reda/opri redarea semnalului cuantizat.

Elementele Interfeței Grafice și Funcționalitățile

1. **Selectarea Tipului de PCM**

- **Element:** Un meniu dropdown (popupmenu) etichetat "Tip PCM".
- **Funcționalitate:** Permite utilizatorului să aleagă între procesarea PCM "Uniform" și "Neuniform".

2. **Selectarea Semnalului de Intrare**

- **Element:** Un meniu dropdown (popupmenu) etichetat "Semnal intrare".
- **Funcționalitate:** Oferă opțiuni pentru selectarea semnalului de intrare: "Sinusoidal", "Dreptunghiular", "Triunghiular", "Dinte fierastrau", și "Audio" (dintr-un fișier .wav).

3. **Stabilirea Numărului de Biți pentru Cuantizare**

- **Element:** Un câmp de editare (edit) etichetat "Nr. biți".
- **Funcționalitate:** Permite utilizatorului să introducă numărul de biți utilizați pentru cuantizarea semnalului. Valoarea implicită este 8.

4. **Selectarea Legii de Compresie (pentru PCM Neuniform)**

- **Elemente:** Un meniu dropdown (popupmenu) etichetat "Lege compresie (A/μ)".
- **Funcționalitate:** Disponibil doar când este selectat PCM neuniform, permite alegerea legii de compresie "A" sau "μ".

5. **Generarea Semnalelor Selectate**

- **Element:** Un buton (pushbutton) etichetat "Generează".
- **Funcționalitate:** La apăsare, generează semnalele (original, comprimat, cuantizat, expandat) și calculează erorile și SNR-ul pe baza configurației curente. Aceste rezultate sunt apoi vizualizate în grafice și afișate numeric.

6. **Redarea/Oprirea Redării Audio**

- **Element:** Un buton (pushbutton) etichetat "Start/Stop".
- **Funcționalitate:** Controlează redarea semnalului cuantizat prin placa de sunet. Prima apăsare începe redarea, iar o apăsare ulterioară o oprește.

7. **Vizualizarea Semnalului în Domeniul Timp**

- **Element:** Șase zone de grafic (axes) etichetate implicit (Semnal Original, Semnal Comprimat, Semnal Cuantizat, Semnal Expandat, Eroare Cuantizare, Comparatie Eroare).
- **Funcționalitate:** Afișează forma de undă a semnalului original și a celui cuantizat (sub formă de trepte - stairs), precum și semnalul comprimat și expandat (pentru PCM neuniform) și eroarea de cuantizare în funcție de timp. Graficul "Comparatie Eroare" vizualizează eroarea RMS pentru PCM uniform și neuniform.

8. **Vizualizarea SNR**

- **Element:** Două zone de grafic (axes) situate sub panoul de rezultate numerice, etichetate implicit "SNR" și "Comparatie SNR".
- **Funcționalitate:** Graficul "SNR" afișează valoarea SNR-ului obținut, iar graficul "Comparatie SNR" compară valorile SNR pentru PCM uniform și neuniform.

9. **Afișarea Rezultatelor Numerice**

- **Element:** Un panou (uipanel) etichetat "Rezultate" care conține câmpuri text.

- **Funcționalitate:** Afișează valorile numerice ale RMS-ului erorii și ale SNR-ului pentru ambele metode PCM (uniform și neuniform).

Prezentare interfata grafica:

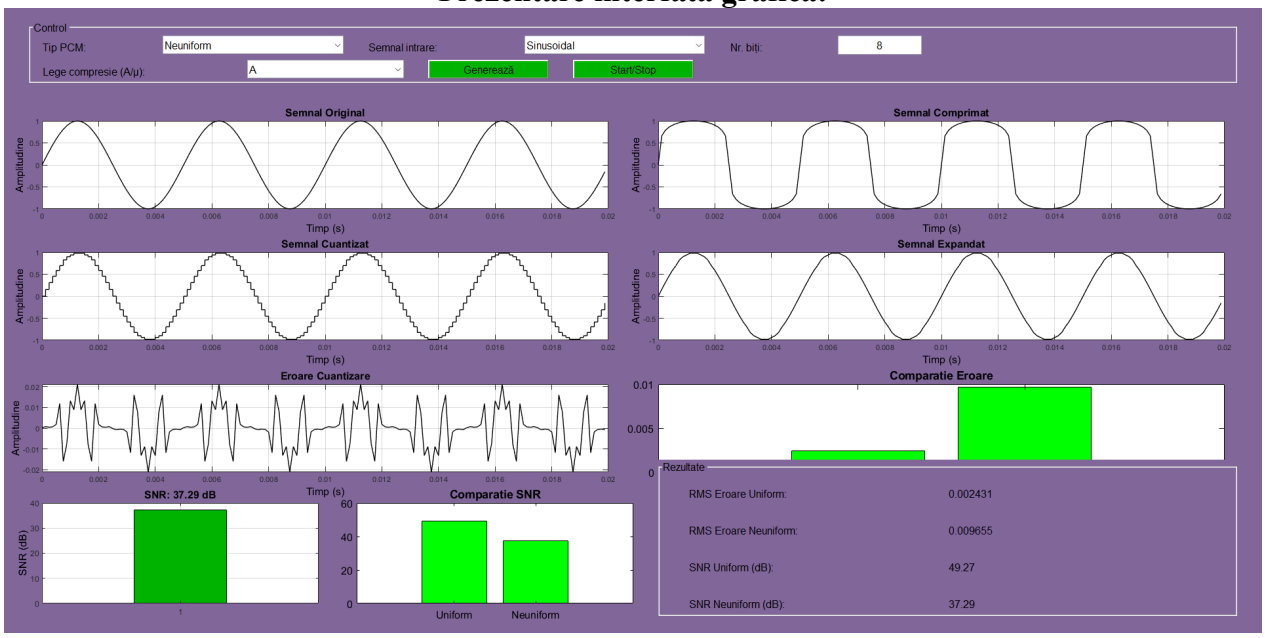

Figura 1.4. Prezentare Interfața Grafică pentru Analiza PCM Uniform și Neuniform

Figura prezintă interfața grafică dezvoltată pentru a demonstra și analiza procesul de cuantizare PCM (Pulse Code Modulation) atât în variantă uniformă, cât și neuniformă. Interfața oferă utilizatorului controale în partea superioară pentru a selecta tipul de PCM, semnalul de intrare, numărul de biți pentru cuantizare, și legea de compresie. Butoanele 'Generează' și 'Start/Stop' permit generarea semnalului procesat și redarea audio a semnalului cuantizat.

Exemplul 1: PCM Uniform

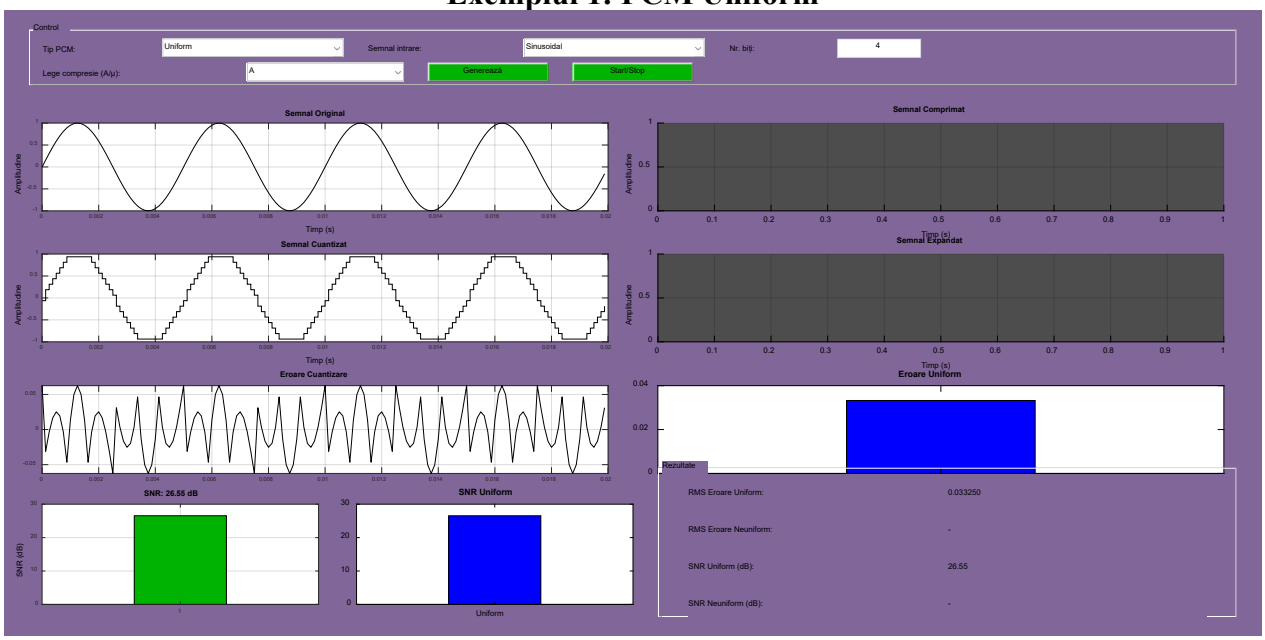

Figura 1.5. PCM Uniform cu Semnal Sinusoidal și 4 biți

Figura ilustrează cuantizarea PCM uniformă a unui semnal sinusoidal folosind 4 biți, rezultând în 2^4=16 nivele de cuantizare. În secțiunea de control, este selectat "Uniform" ca tip PCM, "Sinusoidal" ca semnal de intrare, și "4" ca număr de biți. Câmpul pentru legea de compresie este inactiv, fiind relevant doar pentru PCM neuniform.

Graficul din stânga sus prezintă semnalul sinusoidal original. Sub acesta, graficul "Semnal Cuantizat" arată versiunea discretizată în amplitudine a semnalului, cu 16 nivele. Se observă o aproximare a semnalului original, cu un aspect de trepte pronunțat.

Graficul "Eroare Cuantizare" afișează diferența dintre semnalul original și cel cuantizat. În partea de jos, graficul "SNR" indică un raport semnal-zgomot de 26.55 dB pentru acest caz uniform. Această valoare este de asemenea afișată în graficul "SNR Uniform". În dreapta jos, panoul

Tabelul "Rezultate" prezintă valoarea RMS a erorii pentru PCM uniform ca fiind 0.033250 și SNR-ul uniform ca 35.81 dB, sugerând un nivel semnificativ de zgomot de cuantizare. Graficele pentru semnal comprimat și expandat sunt inactive.

Exemplul 2: PCM Uniform

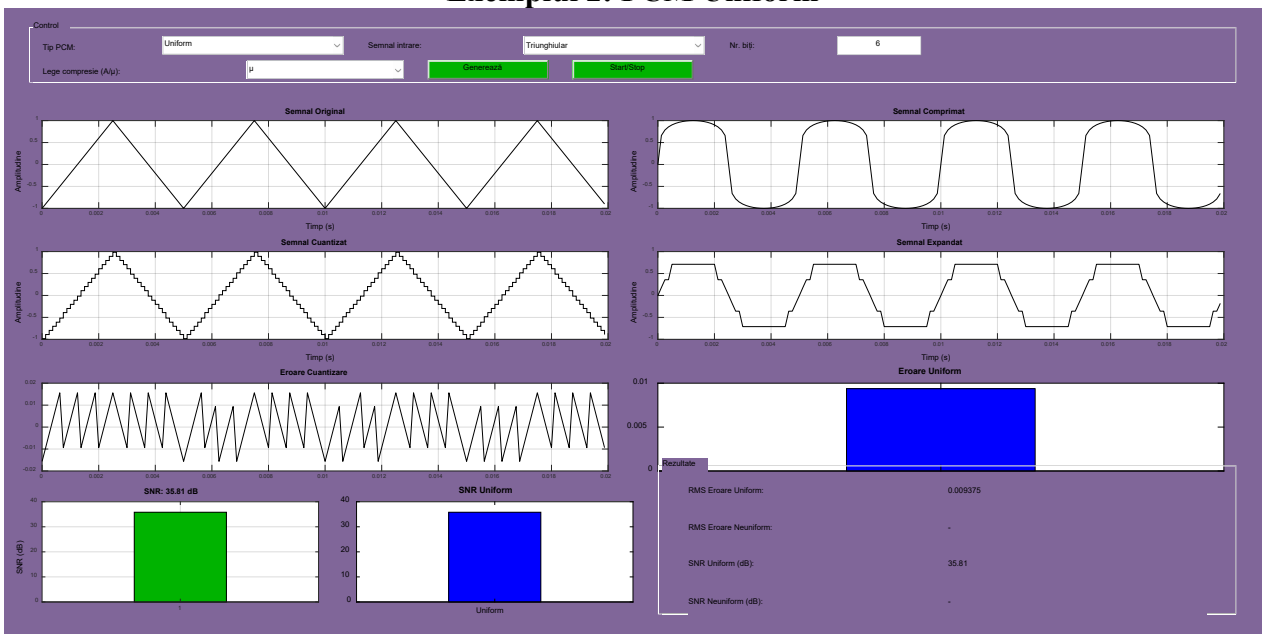

Figura 1.6. PCM Uniform cu Semnal Triunghiular și 6 biți

Figura ilustrează cuantizarea PCM uniformă a unui semnal triunghiular, utilizând 6 biți pentru reprezentare. În secțiunea de control, este selectat "Uniform" ca tip PCM, "Triunghiular" ca semnal de intrare, și "6" ca număr de biți. Câmpul pentru legea de compresie este inactiv, fiind relevant doar pentru PCM neuniform.

Graficul din stânga sus prezintă semnalul triunghiular original. Sub acesta, graficul "Semnal Cuantizat" arată versiunea discretizată în amplitudine a semnalului, cu 2^6=64 de nivele. Se observă o aproximare mai fină a semnalului original comparativ cu cazul cu 4 biți (Figura 1.5).

Graficul "Eroare Cuantizare" afișează diferența dintre semnalul original și cel cuantizat. În partea de jos, graficul "SNR" indică un raport semnal-zgomot de 35.81 dB pentru acest caz uniform. Această valoare este de asemenea afișată în graficul "SNR Uniform". În dreapta jos, panoul "Rezultate" prezintă valoarea RMS a erorii pentru PCM uniform ca fiind 0.009375 și SNR-ul uniform ca 35.81 dB. Graficele pentru semnal comprimat și expandat sunt inactive.

Exemplul 1: PCM Neuniform

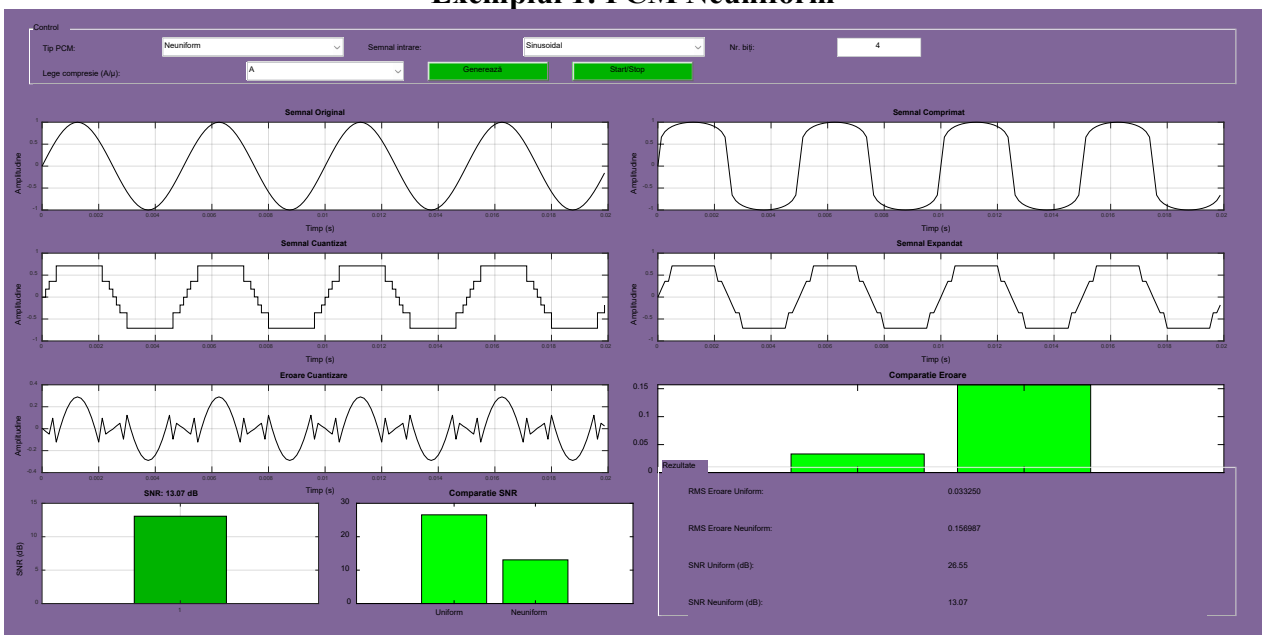

Figura 1.7. PCM Neuniform cu Semnal Sinusoidal, 4 biți și Legea A

Figura ilustrează rezultatele cuantizării PCM neuniforme aplicată unui semnal sinusoidal cu 4 biți, utilizând legea de compresie . În partea de sus, se observă selecția "Neuniform", "Sinusoidal", "4" biți și legea "A".

Graficul din stânga sus prezintă semnalul sinusoidal original. Alături, în dreapta sus, este afișat "Semnal Comprimat", unde se vede modificarea formei de undă datorită compresiei logaritmice A, aplatizând amplitudini mai mari și amplificând cele mici.

Sub semnalul original, "Semnal Cuantizat" arată semnalul comprimat cuantizat în 16 nivele. În dreapta, sub semnalul comprimat, "Semnal Expandat" prezintă semnalul rezultat după expansiune, care este o aproximare a semnalului original.

Graficul "Eroare Cuantizare" din stânga jos ilustrează diferența dintre semnalul original și cel expandat. În dreapta jos, "Comparatie Eroare" arată eroarea RMS pentru PCM uniform (valoare mică) comparativ cu cea pentru PCM neuniform (valoare mai mare).

Similar, graficul "Comparatie SNR" indică SNR-ul pentru PCM uniform (26.55 dB) și PCM neuniform (13.07 dB), sugerând un SNR mai scăzut în acest caz neuniform specific. Panoul "Rezultate" din dreapta confirmă numeric aceste valori: RMS eroare uniform = 0.033250, RMS eroare neuniform = 0.156987, SNR uniform = 26.55 dB, SNR neuniform = 13.07 dB.

Exemplul 2: PCM Neuniform

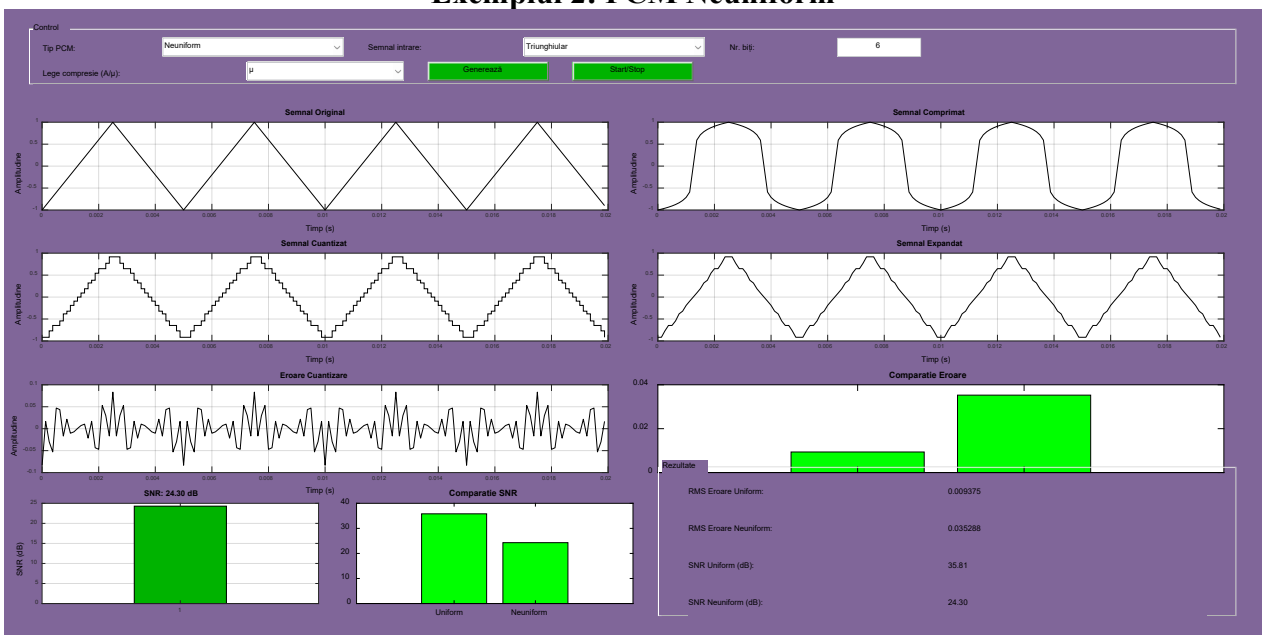

Figura 1.8. PCM Neuniform cu Semnal Triunghiular, 6 biți și Legea μ

Figura ilustrează rezultatele cuantizării PCM neuniforme aplicată unui semnal triunghiular cu 6 biți, utilizând legea de compresie μ. În partea de sus, se observă selecția "Neuniform", "Triunghiular", "6" biți și legea " μ ".

Graficul din stânga sus prezintă semnalul triunghiular original. Alături, în dreapta sus, este afișat "Semnal Comprimat", unde se vede modificarea formei de undă datorită compresiei conform legii μ, afectând pantele și amplitudinea. Sub semnalul original, "Semnal Cuantizat" arată semnalul comprimat cuantizat în 64 de nivele. În dreapta, sub semnalul comprimat, "Semnal Expandat" prezintă semnalul rezultat după expansiune, o aproximare a semnalului triunghiular original.

Graficul "Eroare Cuantizare" din stânga jos ilustrează diferența dintre semnalul original și cel expandat. În dreapta jos, "Comparatie Eroare" arată eroarea RMS pentru PCM uniform (valoare mai mică) comparativ cu cea pentru PCM neuniform (valoare mai mare). Similar, graficul "Comparatie SNR" indică SNR-ul pentru PCM uniform (35.81 dB) și PCM neuniform (24.30 dB), sugerând un SNR ușor mai scăzut în acest caz neuniform specific.

Panoul "Rezultate" din dreapta confirmă numeric aceste valori: RMS eroare uniform = 0.009375, RMS eroare neuniform = 0.035288, SNR uniform = 35.81 dB, SNR neuniform = 24.30 dB.

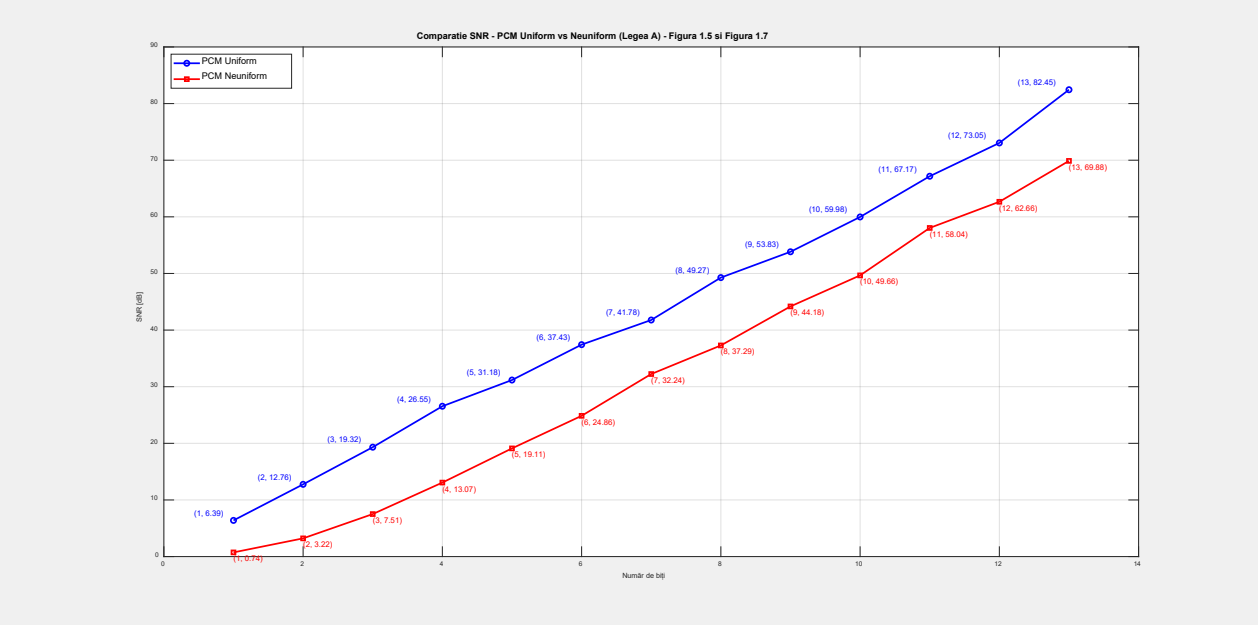

Figura 1.9: Comparație SNR - PCM Uniform vs. Neuniform (Legea A)

Această figură prezintă o comparație grafică a raportului semnal-zgomot (SNR) în decibeli (dB) pentru modulația prin coduri de impulsuri (PCM) uniformă și neuniformă, în funcție de numărul de biți utilizați pentru cuantizare. Ambele metode sunt aplicate unui semnal sinusoidal, iar compresia neuniformă este realizată conform legii A. Numărul de biți variază de la 1 la 13, reprezentând diferite niveluri de rezoluție de cuantizare.

Linia albastră, marcată cu cercuri, reprezintă performanța PCM uniform. Se observă o creștere generală a SNR-ului odată cu creșterea numărului de biți. Această creștere este relativ liniară, indicând o îmbunătățire directă a calității semnalului pe măsură ce se utilizează mai mulți biți pentru a reprezenta semnalul. Etichetele text de lângă puncte indică valorile exacte ale (număr de biți, SNR) pentru fiecare punct de pe curbă.

Linia roșie, marcată cu pătrate, reprezintă performanța PCM neuniform (cu legea A). Similar cu PCM uniform, SNR-ul crește odată cu numărul de biți. Cu toate acestea, la un număr mai mic de biți, SNR-ul este semnificativ mai mic decât în cazul PCM uniform. Pe măsură ce numărul de biți crește, SNR-ul pentru PCM neuniform se apropie de cel al PCM uniform, dar rămâne în general mai mic. Etichetele text roșii indică valorile (număr de biți, SNR) corespunzătoare pentru această curbă.

Secționarea codului și explicații:

```
% Numarul de biti
biti = 1:13;

% SNR pentru PCM uniform (semnal sinusoidal, lege A)
snr_uniform = [6.39, 12.76, 19.32, 26.55, 31.18, 37.43, 41.78, 49.27, 53.83, 59.98, 67.17, 73.05, 82.45];

% SNR pentru PCM neuniform (semnal sinusoidal, lege A)
snr_neuniform = [0.74, 3.22, 7.51, 13.07, 19.11, 24.86, 32.24, 37.29, 44.18, 49.66, 58.04, 62.66, 69.88];
```

Această secțiune inițializează vectorul biti, care definește intervalul de rezoluții de cuantizare ce vor fi analizate. Vectorul conține numere întregi de la 1 la 13, reprezentând numărul de biți utilizați în procesul PCM. Această varietate de valori permite evaluarea impactului rezoluției cuantizării asupra calității semnalului.

Mai sus sunt stocate valorile SNR (Signal-to-Noise Ratio) obținute din simulări. snr_uniform conține valorile SNR pentru PCM uniform, iar snr_neuniform pentru PCM neuniform (cu legea A), ambele corespunzătoare numărului de biți din vectorul biti.

```
% Creare figura
figure;
plot(biti, snr_uniform, '-o', 'LineWidth', 2, 'Color', 'b');
hold on;
plot(biti, snr_neuniform, '-s', 'LineWidth', 2, 'Color', 'r');
grid on;
```

Se creează o figură, se trasează curbele SNR pentru ambele tipuri de PCM, se menține figura pentru a permite suprapunerea graficelor și se activează grila pentru lizibilitate.

```
% Adăugare etichete text pentru PCM uniform
for i = 1:length(biti)
    text(biti(i), snr_uniform(i), sprintf('(%d, %.2f)', biti(i), snr_uniform(i)), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right', ...
        'FontSize', 12, 'Color', 'b');
end

% Adăugare etichete text pentru PCM neuniform
for i = 1:length(biti)
    text(biti(i), snr_neuniform(i), sprintf('(%d, %.2f)', biti(i), snr_neuniform(i)), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', ...
        'FontSize', 12, 'Color', 'r');
end
```

Aceste bucle for adaugă etichete text direct pe grafic, indicând coordonatele precise (număr de biți, SNR) pentru fiecare punct de date, facilitând interpretarea vizuală a rezultatelor.

```
% Setări titlu, etichete, legendă
xlabel('Număr de biți');
ylabel('SNR [dB]');
title('Comparatie SNR - PCM Uniform vs Neuniform (Legea A) - Figura 1.5 si Figura 1.7' ,'FontSize', 14);
legend('PCM Uniform', 'PCM Neuniform', 'Location', 'northwest', 'FontSize', 14);
```

În final, această secțiune setează etichetele axelor, titlul graficului și legenda, asigurând o prezentare clară și informativă a datelor.

Urmatoarea figură prezintă, similar cu figura anterioară, o comparație grafică a raportului semnal-zgomot (SNR) în decibeli (dB) pentru PCM uniform și neuniform, în funcție de numărul de biți utilizați pentru cuantizare (de la 1 la 13). Ambele metode sunt aplicate unui semnal sinusoidal. Curba albastră (cu cercuri) reprezintă PCM uniform, iar curba roșie (cu pătrate) reprezintă PCM neuniform.

Diferența cheie față de figura anterioară este că aici, compresia este realizată conform **legii μ** în loc de legea A. Ambele legi (A și μ) au formule diferite și sunt utilizate în standarde diferite (A-law în Europa, μ-law în America de Nord), ceea ce poate duce la performanțe ușor diferite, observabile prin compararea detaliată a valorilor SNR între cele două figuri.

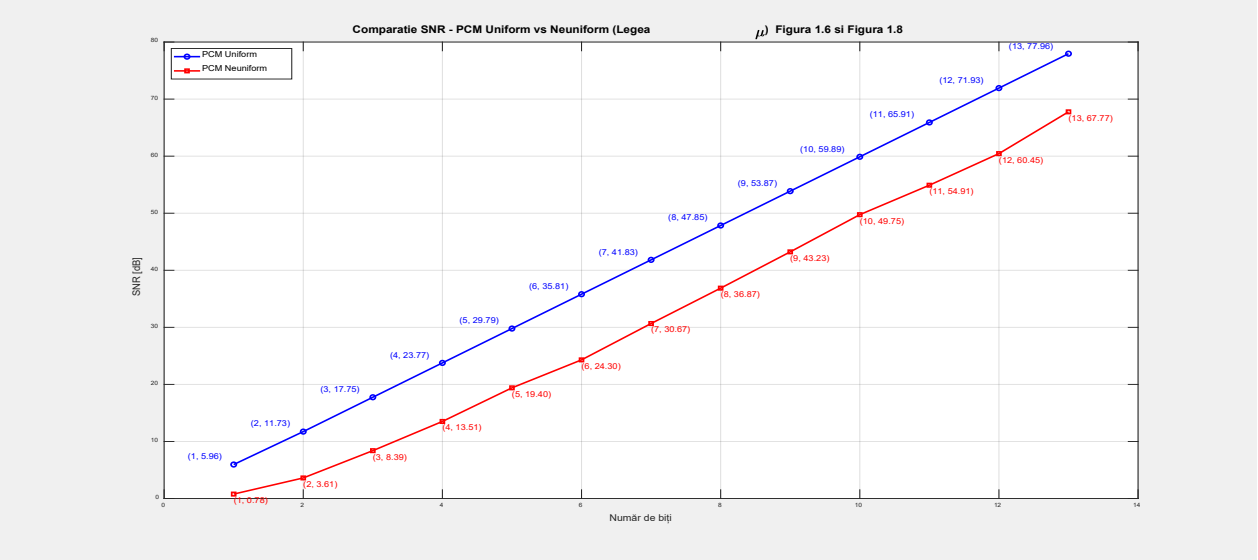

Figura 1.10: Comparație SNR - PCM Uniform vs. Neuniform (Legea μ)

În următorul exemplu este prezentat ca o **curiozitate,** dincolo de cele două exemple tipice cerute pentru PCM neuniform în cerințe. Am vrut să vizualizez rezultatele aplicării cuantizării PCM neuniforme unui fragment audio (din piesa "Talking to Myself" de Linkin Park). Deși documentația s-ar concentra pe semnale mai tipice pentru a demonstra principiile PCM neuniforme, încercarea unui semnal audio real oferă o perspectivă interesantă asupra efectelor cuantizării. În acest caz, am folosit 4 biți pentru cuantizare și legea de compresie μ.

Graficul din stânga sus prezintă semnalul audio original. Alături, în dreapta sus, este afișat "Semnal Comprimat", unde se observă modificarea dinamicii semnalului audio datorită compresiei conform legii μ, influențând amplitudini diferite în mod neuniform. Sub semnalul original, "Semnal Cuantizat" arată semnalul comprimat cuantizat în 2^4=16 nivele discrete. În dreapta, sub semnalul comprimat, "Semnal Expandat" prezintă semnalul rezultat după expansiune, o aproximare a semnalului audio original.

Graficul "Eroare Cuantizare" din stânga jos ilustrează diferența dintre semnalul audio original și cel expandat. În dreapta jos, "Comparatie Eroare" arată eroarea RMS pentru PCM uniform (0.029297) comparativ cu cea pentru PCM neuniform (0.036104), sugerând o eroare RMS ușor mai mare în cazul neuniform specific. Similar, graficul "SNR" din stânga jos indică SNR-ul pentru semnalul neuniform (14.13 dB).

Graficul "Comparatie SNR" compară SNR-ul pentru PCM uniform (15.94 dB) și PCM neuniform (14.13 dB), indicând un SNR ușor mai scăzut în acest caz neuniform.

Având în vedere utilizarea a doar 4 biți pentru cuantizare, se poate percepe o ușoară distorsiune audio.

Exemplu 3: PCM Neuniform – Curiozitate – Semnal Audio

Figura 1.11: Cuantizarea PCM Neuniformă a unui Fragment Audio ("Talking to Myself", Linkin Park) cu 4 Biți și Legea μ

Totodată este important de menționat că selecția specifică a tipului de PCM, a semnalului de intrare și a numărului de biți utilizati în figurile prezentate (1.5, 1.6, 1.7, 1.8, 1.9, 1.10 și 1.11) a fost realizată cu scopul experimental de a facilita vizualizarea caracteristicilor fundamentale ale procesului de cuantizare pentru fiecare tip de semnal. Aceste configurații particulare au fost alese pentru a **evidenția efectele cuantizării uniforme și neuniforme asupra formei de undă**, **erorii introduse și valorii SNR**. Cu toate acestea, interfața grafică oferă utilizatorului control deplin asupra acestor parametri, permițându-i să exploreze un spectru larg de variații, să compare direct cele două metode PCM și să observe impactul diferitelor setări asupra rezultatelor.

2. Script Doi – Modulația Delta

Descrierea teoretică și înțelegerea următoarelor aspecte: modulația/demodulația Delta fixă, modulația/demodulația Delta adaptivă Song și Jayant.

Modulația Delta este o tehnică de codare a vocii, o formă particulară a modulației PCM diferențiale (DPCM). În modulația DPCM, se cuantizează eroarea de predicie a eșantionului curent în loc de valoarea eșantionului curent, ceea ce reduce numărul de biți per eșantion și, implicit, debitul de transmisie. Modulația Delta cuantizează eroarea de predicție a eșantionului curent pe un singur bit, transmițând doar informația despre semnul erorii de predicție. Practic, modulația Delta este o tehnică de conversie analog/digitală și digital/analogică a semnalului vocal cu un singur bit per eșantion.

Utilizarea unui singur bit per eșantion duce la o degradare a calității semnalului vocal analogic reconstituit, manifestată prin zgomot de cuantizare mare și distorsionarea semnalului reconstituit. Din acest motiv, modulația Delta este utilizată în sistemele de transmisie/procesare a vocii unde calitatea semnalului recepționat nu este prioritară, dar se dorește un debit de transmisie cât mai redus. Pentru ca modulația Delta să fie eficientă, este necesară o corelație puternică între eșantioanele consecutive ale semnalului.

În modulația Delta liniară sau fixă, pasul de cuantizare este constant, în timp ce în modulația Delta adaptivă, pasul de cuantizare se modifică în funcție de dinamica semnalului. Modulațiile de tip DPCM, inclusiv modulația Delta, cuantizează eroarea (diferența) dintre valoarea prezisă a eșantionului curent și valoarea reală a acestuia, necesitând generarea unui semnal prezis pe baza eșantioanelor/valorilor anterioare ale semnalului.

Modulația / Demodulația Delta Fixă

În modulația Delta liniară, pasul de cuantizare este constant. Semnalul prezis este calculat pe baza eșantionului anterior și a pasului de cuantizare. Modulatorul Delta liniar include un comparator, un circuit Sample & Hold și un acumulator. Demodulatorul Delta liniar include un circuit de predicție și un filtru trece jos.

Modulația Delta este afectată de două tipuri de distorsiuni: distorsiunea de neurmărire pantă și distorsiunea de granularitate. Distorsiunea de neurmărire pantă apare când viteza de variație a semnalului analogic sursă este mai mare decât viteza de variație a semnalului prezis. Distorsiunea de granularitate apare când viteza de variație a semnalului analogic sursă este mai mică decât viteza de variație a semnalului prezis.

Modulația / Demodulația Delta Adaptivă Song și Jayant

Modulația Delta adaptivă este o tehnică de codare a semnalului care îmbunătățește modulația Delta liniară prin modificarea pasului de cuantizare în funcție de dinamica semnalului sursă. Această adaptare are ca scop reducerea zgomotului de cuantizare și a distorsiunilor, permițând în același timp o urmărire mai precisă a semnalului.

În modulația Delta adaptivă, pasul de cuantizare nu mai este fix, ci se modifica adaptiv in functie de dinamica semnalului sursa. Modulatorul adaptează pasul de cuantizare, folosind pași mici pentru porțiunile lent variabile ale semnalului și pași mari pentru cele rapid variabile. Această strategie reduce distorsiunile de granularitate și de urmărire a pantei, îmbunătățind calitatea generală a semnalului codat.

Algoritmul Song

Algoritmul Song este o metodă de modulație Delta adaptivă în care pasul de cuantizare se ajustează dinamic, în funcție de variațiile semnalului de intrare. Ajustarea se face prin incrementarea sau decrementarea pasului de cuantizare cu o valoare fixă, numită "cuantă elementară". Această cuantă servește ca unitate de bază pentru modificările treptate ale pasului de cuantizare, permițând o adaptare mai fină la semnalele cu variații lente sau rapide.

La baza algoritmului Song stă ideea de a corela ajustarea pasului de cuantizare cu comportamentul recent al semnalului. Mai precis, algoritmul examinează relația dintre bitul curent și bitul anterior din secvența codificată Delta, pentru a determina dacă pasul de cuantizare trebuie crescut sau redus.

Raționamentul este următorul:

- Dacă bitul curent este identic cu bitul anterior, se presupune că semnalul are o tendință constantă sau o variație lentă. În acest caz, pentru a reduce zgomotul granular, pasul de cuantizare este crescut cu o cuantă elementară.

- Dacă bitul curent diferă de bitul anterior, se consideră că semnalul se schimbă rapid. Pentru a urmări mai bine aceste variații, pasul de cuantizare este redus cu o cuantă elementară.

Algoritmul Jayant

Algoritmul Jayant este o metodă de modulație Delta adaptivă care ajustează dinamic pasul de cuantizare prin înmulțirea acestuia cu un factor variabil. Acest factor de scalare este determinat de valoarea bitului anterior transmis.

Ideea principală din spatele algoritmului Jayant este că, dacă semnalul se schimbă rapid, pasul de cuantizare ar trebui să fie mare pentru a urmări eficient aceste schimbări. În schimb, dacă semnalul variază lent (indicat de o secvență de biți identici), pasul de cuantizare poate fi redus pentru a reduce zgomotul de granularitate.

Spre deosebire de algoritmul Song, care adaugă sau scade o cuantă elementară, algoritmul Jayant multiplică pasul de cuantizare, permițând o adaptare mai rapidă și mai flexibilă la variațiile semnalului. Factorii de multiplicare sunt aleși empiric pentru a optimiza performanța în funcție de caracteristicile statistice ale semnalului de intrare.

Un aspect important de menționat este că algoritmul Jayant poate conduce la o gama mai largă de valori pentru pasul de cuantizare comparativ cu algoritmul Song. Această flexibilitate sporită permite o urmărire mai precisă și adaptivă a semnalului, în special în cazul variațiilor rapide. Totuși, această gamă extinsă poate complica atât reprezentarea, cât și analiza semnalului Delta, datorită variațiilor mai mari ale pasului de cuantizare.

Implementările în Matlab: modulația/demodulația Delta fixă, modulația/demodulația Delta adaptivă Song și Jayant.

Analiza Modulatiei / Demodulatiei Delta Fixa

- **delta_sin_laborator10_telefonie**

Inițializarea semnalului și a parametrilor

```
clear all;
close all;
clc;

A = 1.9;          % Amplitudinea semnalului
fs = 8e3;         % Frecventa de esantionare
f = 450;          % Frecventa semnalului
Te = 1/fs;        % Perioada de esantionare
t = 1:Te:10000*Te; % Baza de timp

signal = A * sin (2*pi*f*t); % Semnalul sinusoidal
```

Această secțiune inițializează mediul de lucru Matlab și definește parametrii de bază ai semnalului sinusoidal și ai procesului de eșantionare. Se setează amplitudinea semnalului (A), frecvența de eșantionare (fs), frecvența semnalului (f), perioada de eșantionare (Te) și se creează vectorul de timp (t). În final, se generează semnalul sinusoidal original (signal) care va fi supus procesului de modulație delta.

Calculul Pasului Delta Fix

```
%% Modulatie Delta Fixa
for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;     % Daca semnalul e mai mare decat aproximarea, iesirea e 1
        acc = acc + delta; % Actualizeaza aproximarea
    else
        delta_s(i) = 0;     % Altfel, iesirea e 0
        acc = acc - delta; % Actualizeaza aproximarea
    end
end
```

Se iterează prin fiecare eșantion al semnalului original. La fiecare pas, se compară valoarea curentă a semnalului (signal(i)) cu aproximarea sa curentă (acc). Dacă semnalul este mai mare, bitul modulat corespunzător (delta_s(i)) este setat la 1, iar aproximarea este incrementată cu pasul delta (delta). În caz contrar, bitul modulat este setat la 0, iar aproximarea este decrementată cu pasul delta.

Demodulația Delta

```
%% Demodulatie Delta
acc_r = 0;                    % Acumulatorul pentru demodulare
signal_r = zeros(1, length(signal)); % Semnalul demodulat
for i = 1:length(signal_r)
    if delta_s(i) == 1
        acc_r = acc_r + delta; % Reconstruieste semnalul
    else
        acc_r = acc_r - delta; % Reconstruieste semnalul
    end
    signal_r(i) = acc_r;       % Stocheaza valoarea semnalului demodulat
end
```

Aici are loc procesul de demodulație delta. Se inițializează un nou acumulator (acc_r) pentru reconstrucția semnalului și un vector (signal_r) pentru a stoca semnalul demodulat. Se parcurge semnalul modulat (delta_s). Dacă bitul curent este 1, se adaugă pasul delta (delta) la acumulator; dacă este 0, se scade pasul delta. Valoarea acumulatorului la fiecare pas este considerată valoarea eșantion a semnalului demodulat.

Filtrare

```
%% Filtru
f_cutoff = f + 50;
% f_cutoff = fc1;
signal_filt = lowpass(signal_r, f_cutoff, fs, 'Steepness', 0.9); % Filtru trece-jos
```

Această secțiune aplică un filtru trece-jos (lowpass) semnalului demodulat (signal_r). Scopul filtrului este de a reduce zgomotul de cuantizare și de a netezi semnalul reconstruit. Frecvența de tăiere (f_cutoff) este setată puțin peste frecvența semnalului original. Semnalul rezultat după filtrare este stocat în signal_filt.

Afișare și Analiză

```
%% Afisare
figure,
plot(t, signal, 'r-*'),       % Semnalul original (rosu, cu asterisc)
hold on,
plot(t, signal_r, 'b-+'),     % Semnalul demodulat (albastru, cu plus)
plot(t, signal_filt, 'g-o'),  % Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % Legenda

maxlag = round(fs/f);                                   % Intarzierea maxima pentru corelatie
[r, lags] = xcorr(signal, signal_filt, maxlag);        % Corelatia incrucisata
[n_corr, index] = max(r);                               % Maximul corelatiei
def = abs(lags(index));                                 % Defazajul
P_signal = rms(signal)^2;                               % Puterea semnalului

P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (sincronizat)
P_err_nosync = rms(signal_filt(maxlag:length(signal_filt)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (nesincronizat)


SNR_sync = 10*log10(P_signal / P_err_sync);             % SNR sincronizat
SNR_nosync = 10*log10(P_signal / P_err_nosync);           % SNR nesincronizat
```

Ultima secțiune importantă este dedicată vizualizării rezultatelor și analizei performanței. Se creează o figură în care sunt afișate pe același grafic semnalul original (signal), semnalul demodulat înainte de filtrare (signal_r) și semnalul demodulat după filtrare (signal_filt).

O legendă ajută la identificarea fiecărei curbe. În plus, se calculează corelația încrucișată între semnalul original și cel filtrat pentru a determina defazajul, se estimează puterea semnalului

original și puterea erorii (atât sincronizată, cât și nesincronizată), și în final, se calculează raportul semnal-zgomot (SNR) în ambele cazuri.

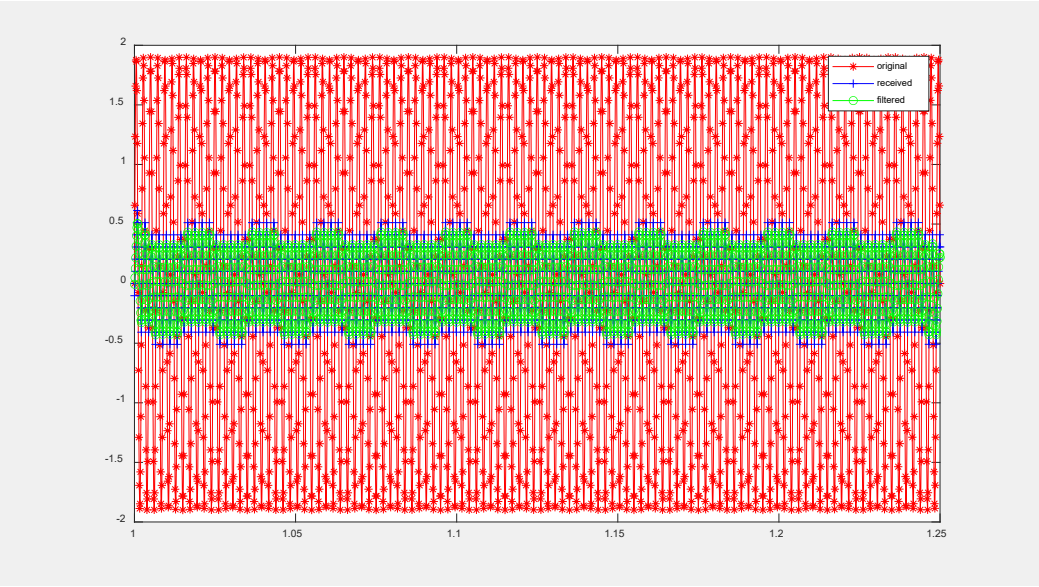

Figura 2.1: Semnal Sinusoidal: Original, Demodulat, Filtrat (Delta Fixă)

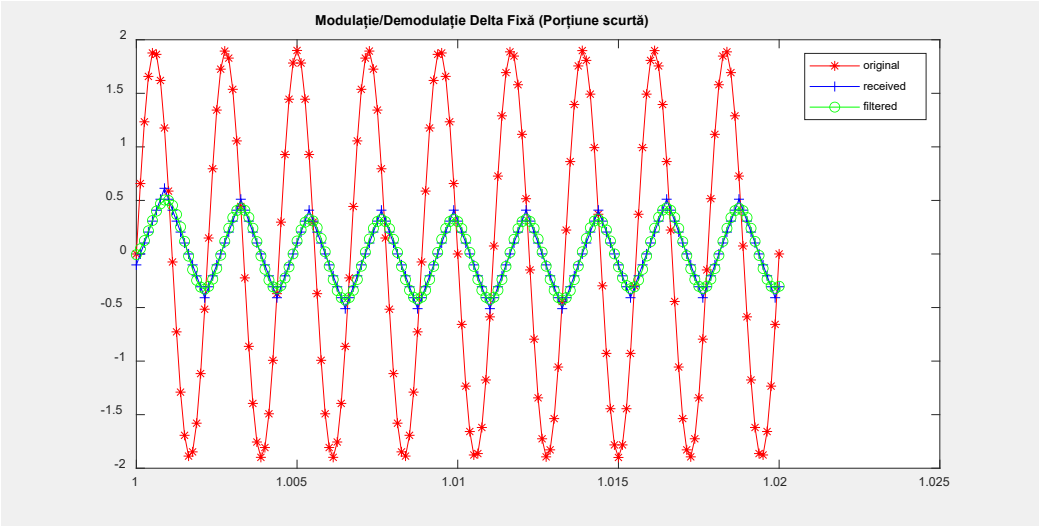

Figura 2.2: Portiunea Scurta pentru Semnal Sinusoidal: Original, Demodulat, Filtrat (Delta Fixă)

Figurile prezintă evoluția în timp a amplitudinii semnalului sinusoidal original (linia roșie cu steluțe), a semnalului demodulat înainte de filtrare (linia albastră cu plusuri) și a semnalului demodulat după aplicarea unui filtru trece-jos (linia verde cu cercuri). Se poate observa cum semnalul demodulat (albastru) este o aproximare "în trepte" a semnalului original, iar aplicarea filtrului (verde) netezește această aproximație, făcând-o să urmărească mai exact forma semnalului original.

SNR Sincronizat (Jayant): 1.8353 dB
SNR Nesincronizat (Jayant): 0.43468 dB

- **delta_vocal_laborator10_telefonie**

Încărcarea și pre-procesarea semnalului vocal

```
clear all;
close all;
clc;

[signal, fs] = audioread('Issaaaaa.wav'); % Citeste semnalul audio dintr-un fisier .wav
t = (0:length(signal)-1)/fs; % Genereaza vectorul de timp
signal = signal'; % Transpune semnalul (il face vector linie)
fc1 = 3400; % Frecventa de taiere a filtrului trece jos
s_f = lowpass(signal, fc1, fs, 'Steepness', 0.8); % Filtreaza semnalul audio cu un filtru trece jos
signal = s_f; % Actualizeaza semnalul cu cel filtrat
```

Această secțiune inițializează mediul de lucru și încarcă un semnal audio vocal din fișierul 'Issaaaaa.wav' folosind funcția audioread, obținând semnalul (signal) și frecvența de eșantionare (fs). Se generează apoi un vector de timp (t) corespunzător duratei semnalului. Semnalul este transpus pentru a fi un vector linie.

Ulterior, semnalul vocal este filtrat printr-un filtru trece-jos cu frecvența de tăiere fc1 (3400 Hz), tipică pentru limitarea benzii de frecvențe în telefonia vocală, iar semnalul filtrat îl înlocuiește pe cel original.

Calculul Pasului Delta Fix

```
%  panta semnalului original
deriv = diff(signal);                    % Derivata semnalului
[Minim, Maxim] = bounds(abs(deriv));     % Minimul si maximul valorilor absolute ale derivatei
delta = median([Minim, Maxim])*0.3;      % Pasul delta
acc = 0;                                 % Acumulatorul
delta_s = zeros(1,length(signal));       % Semnalul modulat delta
```

Similar cu cazul semnalului sinusoidal, această secțiune calculează pasul delta fix pentru modulație. Se estimează panta semnalului vocal filtrat prin calcularea derivatei sale discrete (deriv). Se găsesc apoi valorile minimă și maximă ale valorii absolute a acestei derivate, iar pasul delta (delta) este stabilit ca o fracțiune (0.3) din mediana acestor valori. De asemenea, se inițializează acumulatorul (acc) pentru modulație și vectorul delta_s pentru stocarea semnalului modulat delta.

Modulația Delta Fixă

```
%% Modulatie Delta Fixa


for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;     % Daca semnalul e mai mare decat aproximarea, iesirea e 1
        acc = acc + delta; % Actualizeaza aproximarea
    else
        delta_s(i) = 0;     % Altfel, iesirea e 0
        acc = acc - delta; % Actualizeaza aproximarea
    end
end
```

Această secțiune implementează modulația delta fixă pentru semnalul vocal. Se iterează prin fiecare eșantion al semnalului vocal filtrat (signal). La fiecare pas, se compară valoarea curentă a semnalului cu aproximarea sa curentă (acc). Dacă semnalul este mai mare, bitul modulat (delta_s(i)) este setat la 1, iar aproximarea este incrementată cu pasul delta (delta). În caz contrar, bitul modulat este setat la 0, iar aproximarea este decrementată cu pasul delta.

Demodulația Delta

```
%% Demodulatie Delta


acc_r = 0;                              % Acumulatorul pentru demodulare
signal_r = zeros(1, length(signal));    % Semnalul demodulat
for i = 1:length(signal_r)
    if delta_s(i) == 1
        acc_r = acc_r + delta; % Reconstruieste semnalul
    else
        acc_r = acc_r - delta; % Reconstruieste semnalul
    end
    signal_r(i) = acc_r;       % Stocheaza valoarea semnalului demodulat
end
```

Aici se realizează demodulația delta pentru a reconstrui semnalul vocal. Se inițializează un acumulator (acc_r) și un vector (signal_r) pentru semnalul demodulat. Se parcurge semnalul modulat (delta_s). Dacă bitul curent este 1, se adaugă pasul delta (delta) la acumulator; dacă este 0, se scade pasul delta. Valoarea acumulatorului la fiecare pas este stocată ca valoare a semnalului demodulat.

Filtrare la Demodulare

```
%% Filtru

% f_cutoff = f + 50; % Frecventa de taiere (nu mai e folosita)
f_cutoff = fc1; % Frecventa de taiere a filtrului trece jos
signal_filt = lowpass(signal_r, f_cutoff, fs, 'Steepness', 0.9); % Filtru trece-jos
```

Semnalul demodulat (signal_r) este trecut printr-un filtru trece-jos cu aceeași frecvență de tăiere (fc1) ca și la pre-procesarea semnalului original. Acest filtru ajută la netezirea semnalului reconstruit și la reducerea zgomotului de cuantizare introdus de procesul de modulație delta. Rezultatul filtrării este stocat în signal_filt.

Afișare și Analiză

```
%% Afisare


figure,
plot(t, signal, 'r-*'),       % Semnalul original (rosu, cu asterisc)
hold on,
plot(t, signal_r, 'b-+'),     % Semnalul demodulat (albastru, cu plus)
plot(t, signal_filt, 'g-o'),  % Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % Legenda
sound(signal, fs);           % Reda semnalul original
% sound(signal_filt, fs);      % Reda semnalul filtrat



% !!! de retinut ca la urmarirea semnalului avem distorsiuni de granularitate
```

```
% maxlag = round(fs/f);                      % Intarzierea maxima pentru corelatie
maxlag = round(fs/fc1);                    % Intarzierea maxima pentru corelatie
[r, lags] = xcorr(signal, signal_filt, maxlag); % Corelatia incrucisata
[n_corr, index] = max(r);                    % Maximul corelatiei
def = abs(lags(index));                      % Defazajul
P_signal = rms(signal)^2;                    % Puterea semnalului

P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (sincronizat)
P_err_nosync = rms(signal_filt(maxlag:length(signal_filt)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (nesincronizat)

SNR_sync = 10*log10(P_signal / P_err_sync);     % SNR sincronizat
SNR_nosync = 10*log10(P_signal / P_err_nosync);   % SNR nesincronizat
```

Această ultimă secțiune importantă afișează grafic semnalul vocal original filtrat, semnalul demodulat și semnalul demodulat filtrat. De asemenea, redă semnalul vocal original. În plus, se calculează corelația încrucișată pentru a evalua întârzierea, puterea semnalului original și puterea erorii pentru a determina raportul semnal-zgomot (SNR) în condiții sincronizate și nesincronizate.

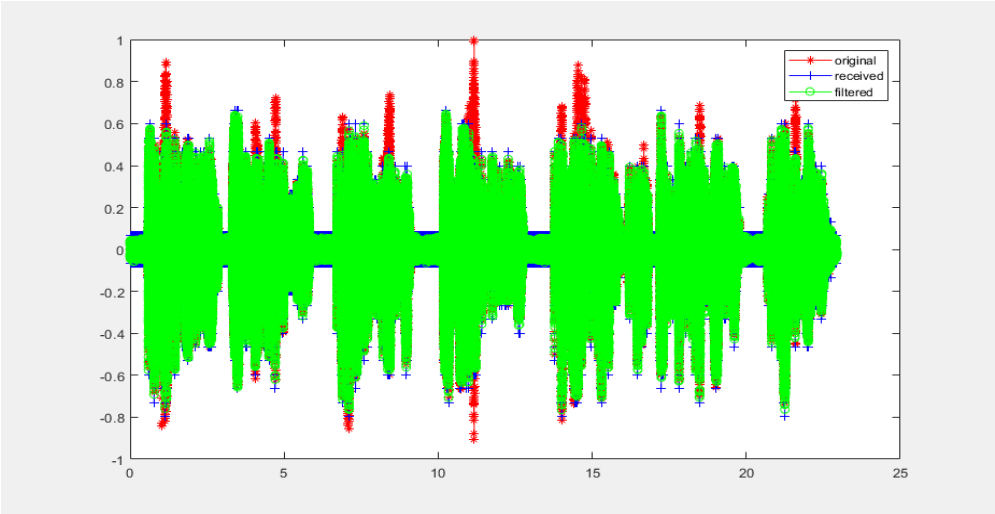

Figura 2.3: Modulația și Demodulația Delta Fixă a unui Semnal Vocal

Figura arată semnalul vocal original (roșu), aproximarea sa demodulată (albastru, în trepte) și versiunea netezită după filtrare (verde), care urmărește forma generală a semnalului vocal.

SNR Sincronizat (Jayant): 10.2355 dB
SNR Nesincronizat (Jayant): 9.9048 dB

Analiza Modulației / Demodulației Delta Adaptivă Song și Jayant

- **delta_sin_song_laborator10_telefonie**

Inițializarea semnalului și a parametrilor

```
clear all;
close all;
clc;

A = 1.9;          % Amplitudinea semnalului
fs = 8e3;         % Frecventa de esantionare
f = 450;          % Frecventa semnalului
Te = 1/fs;        % Perioada de esantionare
t = 1:Te:10000*Te; % Baza de timp
signal = A * sin (2*pi*f*t); % Semnalul sinusoidal
```

Această secțiune realizează inițializarea mediului de lucru Matlab și definește parametrii de bază pentru semnalul sinusoidal și procesul de eșantionare. Se stabilesc amplitudinea (A), frecvența de eșantionare (fs), frecvența semnalului (f), perioada de eșantionare (Te) și se creează vectorul de timp (t). Apoi, se generează semnalul sinusoidal original (signal).

Calculul Pasului Delta Inițial și Inițializări

```
% panta semnalului original
deriv = diff(signal);                    % Derivata semnalului
[Minim, Maxim] = bounds(abs(deriv));     % Minimul si maximul valorilor absolute ale derivatei
delta = median([Minim, Maxim])*0.3;      % Pasul delta initial
acc = 0;                                 % Acumulatorul
delta_s = zeros(1,length(signal));       % Semnalul modulat delta
delta_elem = 0.05;                       % Valoarea cu care se modifica pasul delta
```

Aici se calculează pasul delta inițial, similar cu abordarea pentru delta fixă, bazat pe panta semnalului sinusoidal.

Se calculează derivata (deriv), se găsesc valorile extreme ale acesteia, iar pasul delta inițial (delta) este o fracțiune din mediana lor. De asemenea, se inițializează acumulatorul (acc), vectorul pentru semnalul modulat delta (delta_s) și variabila delta_elem, care reprezintă valoarea cu care pasul delta va fi adaptat.

Modulația Delta Adaptivă Song

```
for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;                 % Iesirea modulata delta este 1
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta = delta + delta_elem; % Incrementeaza delta daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta = delta - delta_elem; % Decrementeaza delta daca iesirile sunt diferite
        end
        if delta < delta_elem
            delta = delta_elem;         % !!! delta nu devine prea mic
        end
        acc = acc + delta;             % Actualizeaza acumulatorul
    else
        delta_s(i) = 0;                 % Iesirea modulata delta este 0
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta = delta + delta_elem; % Incrementeaza delta daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta = delta - delta_elem; % Decrementeaza delta daca iesirile sunt diferite
        end
        if delta < delta_elem
            delta = delta_elem;         % !!! delta nu devine prea mic
        end
        acc = acc - delta;             % Actualizeaza acumulatorul
    end
end
```

Secțiunea implementează modulația delta adaptivă Song. Pentru fiecare eșantion al semnalului, se compară valoarea cu aproximarea (acc). Ieșirea modulației (delta_s(i)) este 1 dacă semnalul este mai mare și 0 altfel. Pasul delta (delta) este adaptat: crește cu delta_elem dacă bitul curent este același cu cel precedent și scade cu delta_elem dacă sunt diferiți. Se impune o limită inferioară pentru delta pentru a nu deveni prea mic. Apoi, acumulatorul (acc) este actualizat cu valoarea curentă a pasului delta.

Demodulația Delta Adaptivă Song

```
acc_r = 0;                                % Acumulatorul pentru demodulare
signal_r = zeros(1, length(signal));      % Semnalul demodulat
delta_r = median([Minim,Maxim])*0.3;       % Pasul delta initial pentru demodulare
for i = 1:length(signal_r)
    if delta_s(i) == 1
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta_r = delta_r + delta_elem; % Incrementeaza delta_r daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta_r = delta_r - delta_elem; % Decrementeaza delta_r daca iesirile sunt diferite
        end
        if delta_r < delta_elem
            delta_r = delta_elem;          % !!! delta_r nu devine prea mic
        end
        acc_r = acc_r + delta_r;          % Reconstruieste semnalul
    else
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta_r = delta_r + delta_elem; % Incrementeaza delta_r daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta_r = delta_r - delta_elem; % Decrementeaza delta_r daca iesirile sunt diferite
        end
        if delta_r < delta_elem
            delta_r = delta_elem;          % !!! delta_r nu devine prea mic
        end
        acc_r = acc_r - delta_r;          % Reconstruieste semnalul
    end
    signal_r(i) = acc_r;                    % Stocheaza valoarea semnalului demodulat
end
```

Această secțiune realizează demodulația adaptivă Song. Se inițializează acumulatorul pentru demodulare (acc_r), semnalul demodulat (signal_r) și pasul delta inițial pentru demodulare (delta_r). Similar cu modulația, pasul delta_r este adaptat în funcție de secvența biților modulați (delta_s).

Dacă bitul curent este același cu cel precedent, delta_r crește; dacă sunt diferiți, scade. Se aplică o limită inferioară pentru delta_r. Semnalul demodulat este reconstruit adăugând sau scăzând valoarea curentă a delta_r din acc_r.

Filtrare și Afișare

```
f_cutoff = f + 50; % Frecventa de taiere a filtrului trece jos
% f_cutoff = fc1;
signal_filt = lowpass(signal_r, f_cutoff, fs, 'Steepness', 0.9); % Filtru trece-jos
```

```
figure,
plot(t, signal, 'r-*'),% Semnalul original (rosu, cu asterisc)
hold on,
plot(t, signal_r, 'b-+'),% Semnalul demodulat (albastru, cu plus)
plot(t, signal_filt, 'g-o'),% Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % legenda
```

```
maxlag = round(fs/f);% Intarzierea maxima pentru corelatie
[r, lags] = xcorr(signal, signal_filt, maxlag);% Corelatia incrucisata
[n_corr, index] = max(r); % Maximul corelatiei
def = abs(lags(index)); % Defazajul
P_signal = rms(signal)^2; % Puterea semnalului

P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (sincronizat)
P_err_nosync = rms(signal_filt(maxlag:length(signal)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;
% Puterea erorii (nesincronizat)

SNR_sync = 10*log10(P_signal / P_err_sync); % SNR sincronizat
SNR_nosync = 10*log10(P_signal / P_err_nosync); % SNR nesincronizat
```

Această ultimă secțiune importantă aplică un filtru trece-jos (lowpass) semnalului demodulat (signal_r) pentru a reduce zgomotul. Apoi, afișează pe același grafic semnalul original, semnalul demodulat și semnalul filtrat. De asemenea, calculează și afișează SNR-ul (raportul semnal-zgomot) pentru a evalua performanța sistemului.

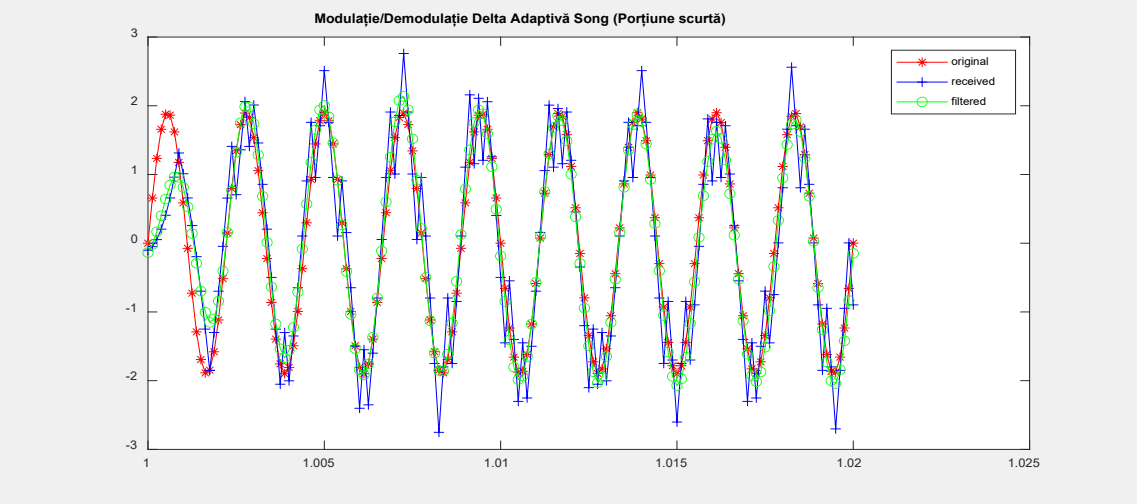

Figura 2.4: Porțiune scurtă Modulație/Demodulație Delta Adaptivă Song

Această porțiune mărită a graficului ilustrează mai clar semnalul sinusoidal original (roșu), aproximarea sa demodulată (albastru) care adaptează pasul, și semnalul demodulat filtrat (verde), care urmărește forma originală într-o manieră mai netedă. Afișarea pe un interval scurt de timp permite vizualizarea detaliată a modului în care algoritmul Song urmărește semnalul.

SNR Sincronizat (Jayant): 19.0493 dB
SNR Nesincronizat (Jayant): 19.0493 dB

- **delta_vocal_song_laborator10_telefonie**

Încărcarea și pre-procesarea semnalului vocal

```
clear all;
close all;
clc;


[signal, fs] = audioread('Issaaaaa.wav'); % Citeste semnalul audio
t = (0:length(signal)-1)/fs;              % Genereaza vectorul de timp
signal = signal';                        % Transpune semnalul (il face vector linie)
fc1 = 3400;                             % Frecventa de taiere a filtrului trece jos
s_f = lowpass(signal, fc1, fs, 'Steepness', 0.8); % Filtreaza semnalul audio
signal = s_f;                           % Actualizeaza semnalul cu cel filtrat
```

Această secțiune inițializează mediul de lucru și încarcă semnalul audio vocal din fișierul 'Issaaaaa.wav', obținând semnalul (signal) și frecvența de eșantionare (fs). Se creează un vector de timp (t). Semnalul este transpus pentru a fi un vector linie. Apoi, semnalul vocal este filtrat cu un filtru trece-jos având frecvența de tăiere fc1 (3400 Hz), iar semnalul filtrat înlocuiește semnalul original.

Calculul Pasului Delta Inițial și Inițializări

```
%  panta semnalului original
deriv = diff(signal);                    % Derivata semnalului
[Minim, Maxim] = bounds(abs(deriv));    % Minimul si maximul valorilor absolute ale derivatei
delta = median([Minim, Maxim])*0.3;     % Pasul delta initial
acc = 0;                                % Acumulatorul
delta_s = zeros(1,length(signal));      % Semnalul modulat delta
delta_elem = 0.5;                       % Valoarea cu care se modifica pasul delta
```

Aici se calculează pasul delta inițial bazat pe panta semnalului vocal filtrat. Se determină derivata (deriv) și valorile extreme ale acesteia, iar pasul delta inițial (delta) este o fracțiune din mediana lor. Se inițializează, de asemenea, acumulatorul (acc), vectorul pentru semnalul modulat delta (delta_s) și valoarea cu care pasul delta va fi adaptat (delta_elem).

Modulația Delta Adaptivă Song

```
for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;                 % Iesirea modulata delta este 1
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta = delta + delta_elem; % Incrementeaza delta daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta = delta - delta_elem; % Decrementeaza delta daca iesirile sunt diferite
        end
        if delta < delta_elem
            delta = delta_elem;         % !!! delta nu devine prea mic
        end
        acc = acc + delta;             % Actualizeaza acumulatorul
    else
        delta_s(i) = 0;                 % Iesirea modulata delta este 0
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta = delta + delta_elem; % Incrementeaza delta daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta = delta - delta_elem; % Decrementeaza delta daca iesirile sunt diferite
        end
        if delta < delta_elem
            delta = delta_elem;         % !!! ca delta nu devine prea mic
        end
        acc = acc - delta;             % Actualizeaza acumulatorul
    end
end
```

Această secțiune implementează modulația delta adaptivă Song pentru semnalul vocal. Se compară fiecare eșantion cu aproximarea (acc), iar ieșirea modulației (delta_s(i)) este 1 sau 0. Pasul delta (delta) este adaptat: crește dacă bitul curent este același cu cel precedent și scade dacă sunt diferiți. Se menține un prag minim pentru delta. Apoi, acumulatorul (acc) este actualizat.

Demodulația Delta Adaptivă Song

```
acc_r = 0;                              % Acumulatorul pentru demodulare
signal_r = zeros(1, length(signal));    % Semnalul demodulat
delta_r = median([Minim,Maxim])*0.3;     % Pasul delta initial pentru demodulare
for i = 1:length(signal_r)
    if delta_s(i) == 1
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta_r = delta_r + delta_elem; % Incrementeaza delta_r daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta_r = delta_r - delta_elem; % Decrementeaza delta_r daca iesirile sunt diferite
        end
        if delta_r < delta_elem
            delta_r = delta_elem;         % !!! delta_r nu devine prea mic
        end
        acc_r = acc_r + delta_r;         % Reconstruieste semnalul
    else
        if i > 1 && delta_s(i) == delta_s(i-1)
            delta_r = delta_r + delta_elem; % Incrementeaza delta_r daca iesirile precedente si curenta sunt la fel
        elseif i > 1
            delta_r = delta_r - delta_elem; % Decrementeaza delta_r daca iesirile sunt diferite
        end
        if delta_r < delta_elem
            delta_r = delta_elem;         % !!! delta_r nu devine prea mic
        end
        acc_r = acc_r - delta_r;         % Reconstruieste semnalul
    end
    signal_r(i) = acc_r;                   % Stocheaza valoarea semnalului demodulat
end
```

Această secțiune implementează demodulația adaptivă Song. Se inițializează acumulatorul (acc_r), semnalul demodulat (signal_r) și pasul delta inițial (delta_r). Pasul delta_r este adaptat în funcție de secvența biților modulați (delta_s). Semnalul demodulat este reconstruit adăugând sau scăzând valoarea curentă a delta_r din acc_r.

Filtrare și Afișare

```
figure,
start_index = find(t >= 1, 1, 'first');
end_index = find(t <= 1.02, 1, 'last');
plot(t(start_index:end_index), signal(start_index:end_index), 'r-*'),       % Semnalul original (roșu, cu asterisc)
hold on,
plot(t(start_index:end_index), signal_r(start_index:end_index), 'b-+'),     % Semnalul demodulat (albastru, cu plus)
plot(t(start_index:end_index), signal_filt(start_index:end_index), 'g-o'),  % Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % Legenda
title('Modulație/Demodulație Delta Adaptivă Song (Vocal - Porțiune scurtă)');


% !!! de retinut ca la urmarirea semnalului avem distorsiuni de granularitate


% maxlag = round(fs/f);                       % Intarzierea maxima pentru corelatie (nu mai e folosita)
maxlag = round(fs/fc1);                    % Intarzierea maxima pentru corelatie
[r, lags] = xcorr(signal, signal_filt, maxlag); % Corelatia incrucisata
[n_corr, index] = max(r);                  % Maximul corelatiei
def = abs(lags(index));                    % Defazajul
P_signal = rms(signal)^2;                  % Puterea semnalului

P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2; % Puterea erorii (sincronizat)
P_err_nosync = rms(signal_filt(maxlag:length(signal)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;       % Puterea erorii (nesincronizat)

SNR_sync = 10*log10(P_signal / P_err_sync);     % SNR sincronizat
SNR_nosync = 10*log10(P_signal / P_err_nosync);   % SNR nesincronizat
```

Această secțiune afișează grafic pe un interval scurt (pentru detalii) semnalul vocal original (roșu), cel demodulat (albastru) și cel filtrat (verde). Apoi, calculează defazajul prin corelație încrucișată și evaluează performanța prin raportul semnal-zgomot (SNR) în scenarii sincronizate și nesincronizate.

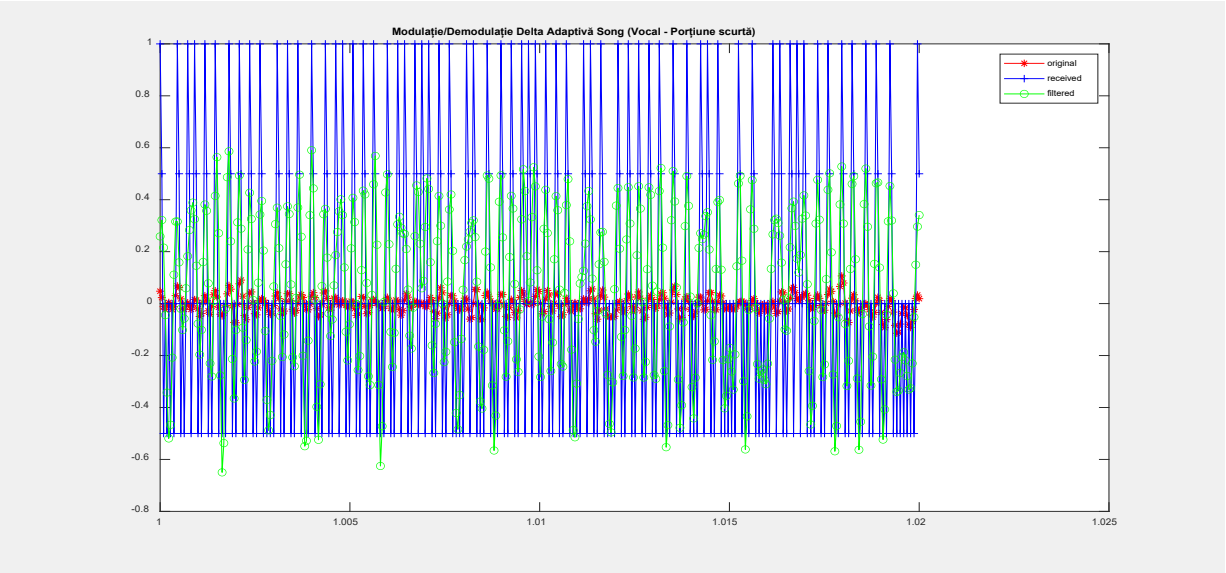

Figura 2.5: Portiune Scurta Modulatie/Demodulație Delta Adaptivă Song pentru Semnal Vocal

Această imagine prezintă o porțiune detaliată a semnalului vocal original (roșu), a semnalului demodulat folosind algoritmul adaptiv Song (albastru) și a semnalului demodulat după filtrare (verde). Se observă modul în care semnalul demodulat "în trepte" (albastru) urmărește semnalul original, iar filtrarea (verde) tinde să netezească aproximația. Afișarea pe un interval scurt de timp permite o analiză mai clară a formelor de undă.

SNR Sincronizat (Jayant): -4.3385 dB
SNR Nesincronizat (Jayant): -4.3385 dB

- **delta_sin_jayant_laborator10_telefonie**

Inițializarea Semnalului și a Parametrilor

```
clear all;
close all;
clc;

A = 1.9;          % Amplitudinea semnalului sinusoidal
fs = 8e3;         % Frecventa de esantionare
f = 450;          % Frecventa semnalului sinusoidal
Te = 1/fs;        % Perioada de esantionare
t = 1:Te:10000*Te; % Baza de timp
signal = A * sin (2*pi*f*t); % Generarea semnalului sinusoidal
```

Această secțiune inițializează mediul de lucru Matlab și definește parametrii de bază pentru semnalul sinusoidal (amplitudine A, frecvență f) și procesul de eșantionare (frecvență de eșantionare fs, perioadă Te). De asemenea, creează vectorul de timp t și generează semnalul sinusoidal original signal.

Inițializarea Parametrilor Modulației/Demodulației Delta adaptivă Jayant

```
% Inițializare
deriv = diff(signal);
[Minim, Maxim] = bounds(abs(deriv));
delta_initial = median([Minim, Maxim])*0.3; % Pasul delta initial
delta = delta_initial;
acc = 0;
delta_s = zeros(1,length(signal));
signal_r = zeros(1, length(signal));
acc_r = 0;
delta_r = delta_initial;
P = 2; % Factor de scalare pentru adaptarea pasului
```

În această secțiune, se calculează pasul delta inițial (delta_initial) pe baza pantei semnalului sinusoidal. Se inițializează pasul delta curent pentru modulație (delta), acumulatorul pentru modulație (acc), semnalul modulat delta (delta_s), semnalul demodulat (signal_r), acumulatorul pentru demodulație (acc_r) și pasul delta curent pentru demodulație (delta_r) cu valoarea inițială. Se definește, de asemenea, factorul de scalare P, care va fi utilizat pentru adaptarea pasului delta.

Modulația Delta Adaptivă Jayant

```
for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;
        acc = acc + delta;
        if i > 1
            if delta_s(i) == delta_s(i-1)
                delta = delta * P;
            else
                delta = delta / P;
            end
        end
            else
                delta_s(i) = 0;
                acc = acc - delta;
                if i > 1
                    if delta_s(i) == delta_s(i-1)
                        delta = delta * P;
                    else
                        delta = delta / P;
                    end
                end
            end
            % Limite pentru delta (optional)
            delta = max(delta, delta_initial/10);
            delta = min(delta, delta_initial*10);
        end
```

Această secțiune implementează modulația delta adaptivă Jayant. Pentru fiecare eșantion al semnalului, se compară valoarea cu aproximarea (acc). Dacă semnalul este mai mare, bitul modulat (delta_s(i)) este 1, iar aproximarea crește cu delta; altfel, bitul este 0, iar aproximarea scade cu delta. Pasul delta (delta) este adaptat: se înmulțește cu P dacă bitul curent este același cu cel precedent și se împarte la P dacă sunt diferiți. Se aplică limite pentru a menține pasul delta într-un interval rezonabil.

Demodulația Delta Adaptivă Jayant

```
for i = 1:length(signal)
    if delta_s(i) == 1
        acc_r = acc_r + delta_r;
    else
        acc_r = acc_r - delta_r;
    end
    signal_r(i) = acc_r;
    if i > 1
        if delta_s(i) == delta_s(i-1)
            delta_r = delta_r * P;
        else
            delta_r = delta_r / P;
        end
    end
    % Limite pentru delta_r (optional)
    delta_r = max(delta_r, delta_initial/10);
    delta_r = min(delta_r, delta_initial*10);
end
```

Această secțiune realizează demodulația delta adaptivă Jayant. Se parcurge semnalul modulat (delta_s). Dacă bitul este 1, se adaugă delta_r la aproximarea semnalului (acc_r); dacă este 0, se scade delta_r. Semnalul demodulat este stocat în signal_r. Similar cu modulația, pasul delta pentru demodulație (delta_r) este adaptat în funcție de secvența biților modulați, folosind același factor de scalare P și aceleași limite.

Filtrare și Afișare

```
figure,
start_index = find(t >= 1, 1, 'first');
end_index = find(t <= 1.02, 1, 'last');
plot(t(start_index:end_index), signal(start_index:end_index), 'r-*'),    % Semnalul original (roșu, cu asterisc)
hold on,
plot(t(start_index:end_index), signal_r(start_index:end_index), 'b-+'),  % Semnalul demodulat (albastru, cu plus)
plot(t(start_index:end_index), signal_filt(start_index:end_index), 'g-o'), % Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % Legenda
title('Modulație/Demodulație Delta Adaptivă Jayant (Porțiune scurtă)');

maxlag = round(fs/f);
[r, lags] = xcorr(signal, signal_filt, maxlag);
[n_corr, index] = max(r);
def = abs(lags(index));
P_signal = rms(signal)^2;
P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2;
P_err_nosync = rms(signal_filt(maxlag:length(signal_filt)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;
SNR_sync = 10*log10(P_signal / P_err_sync);
SNR_nosync = 10*log10(P_signal / P_err_nosync);
```

Această ultimă secțiune aplică un filtru trece-jos (lowpass) semnalului demodulat (signal_r) pentru a reduce zgomotul. Apoi, afișează pe același grafic semnalul original, semnalul demodulat și semnalul filtrat. De asemenea, calculează corelația încrucișată pentru a determina defazajul și rapoartele semnal-zgomot (SNR) sincronizat și nesincronizat, afișându-le în fereastra de comandă.

SNR Sincronizat (Jayant): 10.6259 dB
SNR Nesincronizat (Jayant): 8.0981 dB

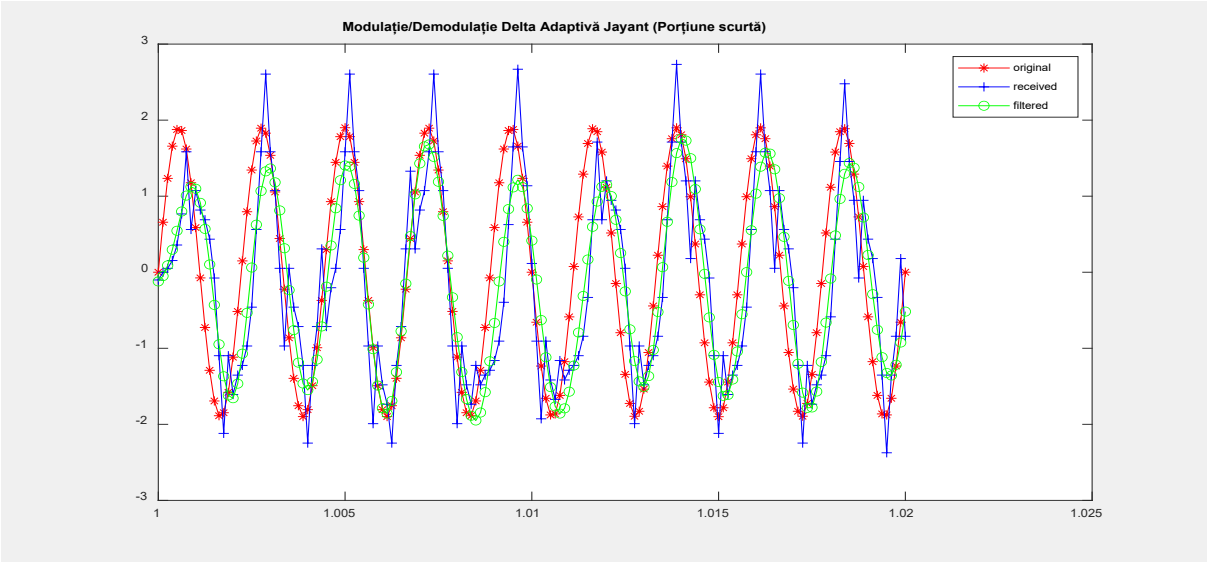

Figura 2.6: Portiunea scurta pentru Modulație/Demodulație Delta Adaptivă Jayant

- **delta_vocal_jayant_laborator10_telefonie**

Încărcarea și pre-procesarea semnalului vocal

```
clear all;
close all;
clc;

[signal, fs] = audioread('Issaaaaa.wav'); % Citeste semnalul audio
t = (0:length(signal)-1)/fs;% Genereaza vectorul de timp
signal = signal';  % Transpune semnalul (il face vector linie)
fc1 = 3400; % Frecventa de taiere a filtrului trece jos
s_f = lowpass(signal, fc1, fs, 'Steepness', 0.8); % Filtreaza semnalul audio
signal = s_f;% Actualizeaza semnalul cu cel filtrat
```

Această secțiune inițializează mediul de lucru, citește un semnal audio vocal din fișierul 'Issaaaaa.wav' și generează vectorul de timp corespunzător. Semnalul este transpus într-un vector linie și apoi filtrat printr-un filtru trece-jos cu frecvența de tăiere de 3400 Hz, specifică pentru aplicații de telefonie vocală. Semnalul filtrat este apoi utilizat pentru procesare.

Inițializarea parametrilor modulației/demodulației Delta adaptivă Jayant

```
% Inițializare
deriv = diff(signal);
[Minim, Maxim] = bounds(abs(deriv));
delta_initial = median([Minim, Maxim])*0.3; % Pasul delta initial
delta = delta_initial;
acc = 0;
delta_s = zeros(1,length(signal));
signal_r = zeros(1, length(signal));
acc_r = 0;
delta_r = delta_initial;
P = 2; % Factor de scalare pentru adaptarea pasului
```

Aici se calculează pasul delta inițial pe baza derivatei semnalului vocal filtrat. Se inițializează pasul delta curent pentru modulație (delta), acumulatorul pentru modulație (acc), semnalul modulat delta (delta_s), semnalul demodulat (signal_r), acumulatorul pentru demodulație (acc_r) și pasul delta curent pentru demodulație (delta_r) cu valoarea inițială. Se definește, de asemenea, factorul de scalare P=2, utilizat în algoritmul de adaptare Jayant.

Modulația Delta Adaptivă Jayant

```
%% Modulatie Delta Adaptiva Jayant

for i = 1:length(signal)
    if signal(i) > acc
        delta_s(i) = 1;
        acc = acc + delta;
        if i > 1
            if delta_s(i) == delta_s(i-1)
                delta = delta * P;
            else
                delta = delta / P;
            end
        end
    else
        delta_s(i) = 0;
        acc = acc - delta;
        if i > 1
            if delta_s(i) == delta_s(i-1)
                delta = delta * P;
            else
                delta = delta / P;
            end
        end
    end
    % Limite pentru delta (optional)
    delta = max(delta, delta_initial/10);
    delta = min(delta, delta_initial*10);
end
```

Această secțiune implementează algoritmul de modulație delta adaptivă Jayant. Pentru fiecare eșantion al semnalului vocal, se compară cu aproximarea curentă (acc). Bitul modulat (delta_s(i)) este setat la 1 dacă semnalul este mai mare și la 0 altfel.

Pasul delta (delta) este adaptat în funcție de secvența ultimelor doi biți modulați: crește dacă sunt identici și scade dacă sunt diferiți, folosind factorul de scalare P. Se aplică limite pentru a menține pasul delta într-un interval rezonabil.

Demodulația Delta Adaptivă Jayant

```
for i = 1:length(signal)
    if delta_s(i) == 1
        acc_r = acc_r + delta_r;
    else
        acc_r = acc_r - delta_r;
    end
    signal_r(i) = acc_r;
    if i > 1
        if delta_s(i) == delta_s(i-1)
            delta_r = delta_r * P;
        else
            delta_r = delta_r / P;
        end
    end
    % Limite pentru delta_r (optional)
    delta_r = max(delta_r, delta_initial/10);
    delta_r = min(delta_r, delta_initial*10);
end
```

Aici se realizează demodulația delta adaptivă Jayant. Semnalul demodulat (signal_r) este reconstruit pe baza secvenței de biți modulate (delta_s) și a pasului delta adaptiv pentru demodulare (delta_r). Similar cu modulația, pasul delta_r este adaptat în funcție de ultimii doi biți primiți, folosind același factor de scalare P și aceleași limite.

Filtrare

```
f_cutoff = fc1;
signal_filt = lowpass(signal_r, f_cutoff, fs, 'Steepness', 0.9);
```

Semnalul demodulat (signal_r) este trecut printr-un filtru trece-jos cu aceeași frecvență de tăiere (fc1) ca și la pre-procesarea semnalului original. Scopul este de a reduce zgomotul de cuantizare introdus de procesul de modulație/demodulație delta.

Afișare și Calcul SNR (Porțiune scurtă)

```
figure,
start_index = find(t >= 1, 1, 'first');
end_index = find(t <= 1.02, 1, 'last');
plot(t(start_index:end_index), signal(start_index:end_index), 'r-*'),    % Semnalul original (roșu, cu asterisc)
hold on,
plot(t(start_index:end_index), signal_r(start_index:end_index), 'b-+'),  % Semnalul demodulat (albastru, cu plus)
plot(t(start_index:end_index), signal_filt(start_index:end_index), 'g-o'), % Semnalul filtrat (verde, cu cerc)
legend('original', 'received', 'filtered'); % Legenda
title('Modulație/Demodulație Delta Adaptivă Jayant (Porțiune scurtă)');
```

Sectiunea de sus afișează grafic o porțiune scurtă (între t=1 și t=1.02 secunde) a semnalului vocal original (roșu), a semnalului demodulat (albastru) și a semnalului demodulat filtrat (verde).

```
maxlag = round(fs/fc1);
[r, lags] = xcorr(signal, signal_filt, maxlag);
[n_corr, index] = max(r);
def = abs(lags(index));
P_signal = rms(signal)^2;
P_err_sync = rms(signal_filt(maxlag+def:length(signal_filt)-maxlag+def) - signal(maxlag:length(signal)-maxlag))^2;
P_err_nosync = rms(signal_filt(maxlag:length(signal_filt)-maxlag) - signal(maxlag:length(signal)-maxlag))^2;
SNR_sync = 10*log10(P_signal / P_err_sync);
SNR_nosync = 10*log10(P_signal / P_err_nosync);

disp(['SNR Sincronizat (Jayant - Vocal): ', num2str(SNR_sync), ' dB']);
disp(['SNR Nesincronizat (Jayant - Vocal): ', num2str(SNR_nosync), ' dB']);
```

Se calculează, de asemenea, corelația încrucișată pentru a determina defazajul și se evaluează performanța sistemului prin calculul și afișarea rapoartelor semnal-zgomot (SNR) sincronizat și nesincronizat.

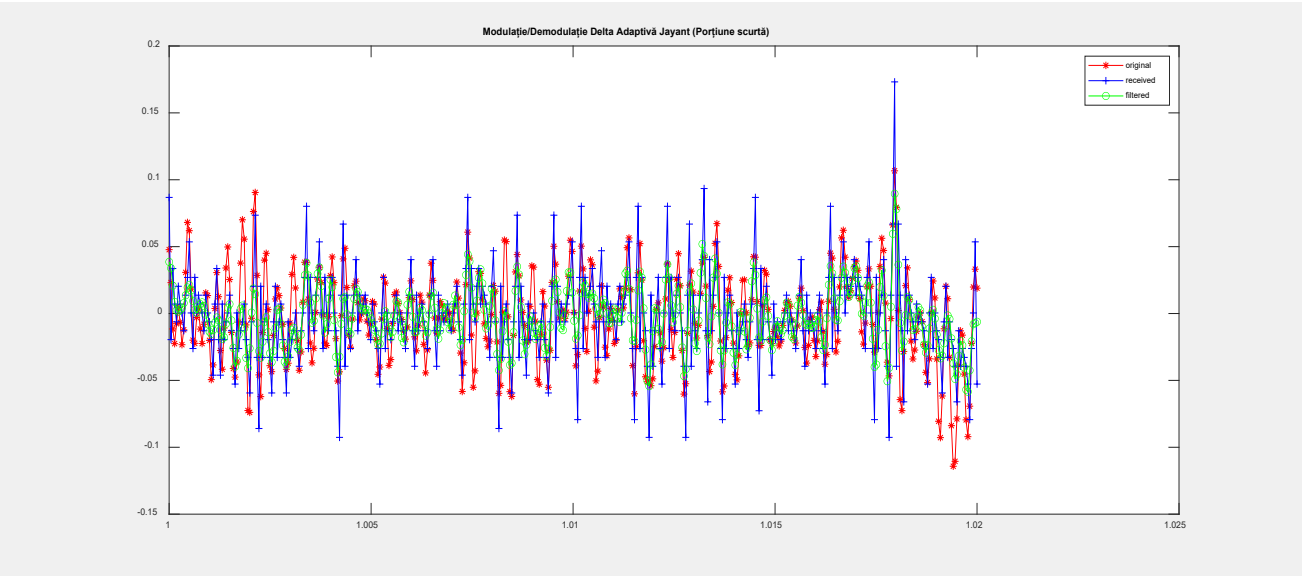

Figura 2.7: Rezultat pentru Delta Adaptivă Jayant pentru Semnal Vocal

Această imagine prezintă o porțiune detaliată a semnalului vocal original (roșu), a semnalului demodulat folosind algoritmul adaptiv Jayant (albastru) și a semnalului demodulat după filtrare (verde). Se observă modul în care pasul delta variabil al algoritmului Jayant încearcă să urmărească semnalul vocal, iar filtrarea aplicată semnalului demodulat are un efect de netezire.

Implementarea unei interfețe grafice

```
function script2_delta_gui()
    % ---- SETĂRI VIZUALE ----
    backgroundColor = [0.5 0.4 0.6]; % fundal mov
    textColor = [0 0 0];             % text negru
    buttonColor = [0 0.7 0];         % butoane verzi
    fontSize = 12;                   % dimensiune font redusa
    lineWidth = 1;


    % ---- FIGURA PRINCIPALĂ ----
    % creaza figura principala a GUI-ului
    f = figure('Name', 'Delta Modulation GUI', 'Units', ...
        'normalized', 'Position', [0.05, 0.05, 0.9, 0.85], 'Color', backgroundColor);

    % --- CONTROALE UI DE SUS ---
    control_panel = uipanel('Parent', f, 'Title', 'Control', ...
        'Units', 'normalized', 'Position', [0.02, 0.88, 0.96, 0.1], ...
        'BackgroundColor', backgroundColor, ...
        'ForegroundColor', textColor, 'FontSize', fontSize);
```

Această secțiune inițializează aspectul vizual al GUI-ului (culori, font, grosime linii) și creează fereastra principală ('Delta Modulation GUI'). În partea de sus a ferestrei, adaugă un panou 'Control' care va conține elementele de interacțiune.

- **Crearea câmpului de selecție pentru semnalul de intrare.**

```
% Semnal intrare
% Etichetă pentru selectarea semnalului de intrare
uicontrol('Parent', control_panel, 'Style', 'text', 'Units', ...
    'normalized', 'Position', [0.3, 0.55, 0.13, 0.3], 'String', 'Semnal intrare:', ...
    'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left');


% Meniu dropdown pentru selectarea tipului de semnal de intrare
signalType = uicontrol('Parent', control_panel, 'Style', ...
    'popupmenu', 'Units', 'normalized', ...
    'Position', [0.43, 0.55, 0.15, 0.4], 'String', {'Sinusoidal', ...
    'Dreptunghiular', 'Triunghiular', 'Dinte fierastrau', 'Audio'}, ...
    'BackgroundColor', 'white', 'ForegroundColor', textColor, 'FontSize', fontSize);
```

În interiorul panoului de control, această secțiune adaugă două elemente UI: o etichetă statică (uicontrol cu Style 'text') care afișează textul "Semnal intrare:", și un meniu dropdown (uicontrol cu Style 'popupmenu') numit signalType.

Acest meniu permite utilizatorului să aleagă dintre diferite tipuri de semnale de intrare: sinusoidal, dreptunghiular, triunghiular, dinte de fierăstrău și audio. Proprietățile setate pentru meniul dropdown includ lista de opțiuni ('String'), culoarea de fundal (alb) și culoarea textului.

Poziția și dimensiunea elementelor sunt specificate folosind unități normalizate, astfel încât să se adapteze la dimensiunea panoului de control.

- **Crearea câmpului de selecție pentru tipul de modulație Delta.**

```
% Tip Modulatie Delta


% Eticheta pentru selectarea tipului de modulatie Delta
uicontrol('Parent', control_panel, 'Style', 'text', 'Units', ...
    'normalized', 'Position', [0.01, 0.55, 0.12, 0.3], 'String', 'Tip Delta:', ...
    'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left');

% Meniu dropdown pentru selectarea tipului de modulatie Delta (Fixa/Song/Jayant)
deltaType = uicontrol('Parent', control_panel, 'Style', ...
    'popupmenu', 'Units', 'normalized', 'Position', [0.13, 0.55, 0.15, 0.4], ...
    'String', {'Fixa', 'Song', 'Jayant'}, 'BackgroundColor', ...
    'white', 'ForegroundColor', textColor, 'FontSize', fontSize);
```

Această parte adaugă o etichetă "Tip Delta" și un meniu dropdown numit deltaType. Acest meniu oferă utilizatorului posibilitatea de a alege între trei tipuri de modulație Delta: "Fixa", "Song" și "Jayant". La fel ca și pentru selecția semnalului, se definesc proprietăți pentru stil, poziție, dimensiune, opțiuni afișate și culori.

- **Crearea câmpului pentru stabilirea pasului de cuantizare (delta).**

```
% Pas cuantizare (delta)
% Etichetă pentru introducerea pasului de cuantizare
uicontrol('Parent', control_panel, 'Style', 'text', 'Units', ...
    'normalized', 'Position', [0.6, 0.55, 0.12, 0.3], 'String', ...
    'Pas delta:', 'BackgroundColor', backgroundColor, 'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left');

% Câmp editabil pentru introducerea pasului de cuantizare
deltaInput = uicontrol('Parent', control_panel, 'Style', ...
    'edit', 'Units', 'normalized', ...
    'Position', [0.72, 0.55, 0.07, 0.4], 'String', '0.1', 'BackgroundColor', 'white', ...
    'ForegroundColor', textColor, 'FontSize', fontSize);
```

Această secțiune introduce o etichetă statică "Pas delta" și un câmp editabil (uicontrol cu Style 'edit') numit deltaInput. Acest câmp permite utilizatorului să introducă o valoare numerică ce reprezintă pasul de cuantizare (delta) utilizat în procesul de modulație Delta. Valoarea implicită afișată în acest câmp este '0.1'.

- **Crearea câmpului pentru pasul de cuantizare elementar (pentru Delta adaptivă).**

```
% Pas cuantizare elementar (pentru Delta adaptiva)
% Eticheta pentru pasul de cuantizare elementar
uicontrol('Parent', control_panel, 'Style', 'text', ...
    'Units', 'normalized', 'Position', [0.01, 0.05, 0.17, 0.3], ...
    'String', 'Pas delta elementar:', 'BackgroundColor', backgroundColor, ...
    'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left');

% Camp editabil pentru pasul de cuantizare elementar
deltaElemInput = uicontrol('Parent', control_panel, 'Style', ...
    'edit', 'Units', 'normalized', ...
    'Position', [0.18, 0.05, 0.13, 0.4], 'String', '0.05', ...
    'BackgroundColor', 'white', ...
    'ForegroundColor', textColor, 'FontSize', fontSize);
deltaElemInput.Visible = 'off'; % Initial ascuns
```

Pentru modulația Delta adaptivă (tipurile Song și Jayant), această secțiune adaugă o etichetă "Pas delta elementar:" și un câmp editabil deltaElemInput. Acest câmp permite utilizatorului să specifice pasul de cuantizare de bază care va fi adaptat în funcție de algoritmul specific tipului de modulație adaptivă selectat.

Inițial, acest câmp este setat să fie invizibil (deltaElemInput.Visible = 'off';), deoarece este relevant doar când se selectează un tip de modulație Delta adaptivă. Valoarea implicită este '0.05'.

- **Crearea câmpului pentru stabilirea frecvenței de tăiere a filtrului**

```
% Frecventa taiere filtru
% Eticheta pentru introducerea frecventei de taiere a filtrului
uicontrol('Parent', control_panel, 'Style', 'text', 'Units', ...
    'normalized', 'Position', [0.33, 0.05, 0.15, 0.3], ...
    'String', 'Frecventa filtru:', 'BackgroundColor', backgroundColor, ...
    'ForegroundColor', textColor, ...
    'FontSize', fontSize, 'HorizontalAlignment', 'left');

% Camp editabil pentru frecventa de taiere a filtrului
filterFreqInput = uicontrol('Parent', control_panel, 'Style', ...
    'edit', 'Units', 'normalized', ...
    'Position', [0.48, 0.05, 0.1, 0.4], 'String', '3400', ...
    'BackgroundColor', 'white', ...
    'ForegroundColor', textColor, 'FontSize', fontSize);
```

Această parte a codului creează o etichetă "Frecventa filtru" și un câmp editabil filterFreqInput. Utilizatorul poate introduce în acest câmp valoarea frecvenței de tăiere pentru filtrul low-pass care va fi aplicat semnalului demodulat. Valoarea implicită este setată la '3400'.

- **Crearea butonului pentru generarea semnalelor selectate.**

```
% Buton generare semnal
% Buton pentru a genera semnalul si a efectua modulatia Delta
uicontrol('Parent', control_panel, 'Style', 'pushbutton', 'Units', ...
    'normalized', 'Position', [0.6, 0.05, 0.1, 0.4], ...
    'String', 'Genereaza', 'Callback', @genereazaSemnal, ...
    'BackgroundColor', buttonColor, ...
    'ForegroundColor', textColor, 'FontSize', fontSize);
```

Această secțiune adaugă un buton cu eticheta "Genereaza". Atunci când utilizatorul apasă acest buton, este declanată funcția de callback @genereazaSemnal.

Această funcție este responsabilă de citirea valorilor selectate din controalele GUI (tipul de modulație, semnalul de intrare, pasul delta, frecvența filtrului), generarea semnalului de intrare, aplicarea modulației Delta, demodularea semnalului, aplicarea filtrului și vizualizarea rezultatelor. Culoarea de fundal a butonului este setată la verde (buttonColor).

- **Crearea axelor pentru vizualizarea semnalelor în domeniul timp.**

```
% --- AXE PENTRU VIZUALIZARE ---

vertical_space = 0.07; % Spațiere verticală între grafice
horizontal_space = 0.04; % Spațiere orizontală între grafice
plot_width = 0.45;       % Lățimea graficelor
plot_height = 0.14;      % Înălțimea graficelor
y_top = 0.80;            % Poziția de sus pentru prima linie de grafice
title_y_offset = 1.05;
```

Această primă parte definește variabile pentru a controla spațierea verticală și orizontală dintre grafice, lățimea și înălțimea fiecărui grafic, poziția de start pe verticală pentru prima linie de grafice și un decalaj pentru poziția titlurilor graficelor.

```
% Linia 1
% Axă semnal original
ax1_x = 0.03;
ax1_y = y_top - plot_height;
ax1 = axes('Units', 'normalized', 'Position', [ax1_x, ax1_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, 'XColor', textColor, ...
    'YColor', textColor); % Creează axa pentru semnalul original
title_obj = title(ax1, 'Semnal Original', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax1, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax1, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax1, 'on');
set(ax1, 'GridColor', textColor); % Activează grila
```

Secțiunea creează prima axă (ax1) pentru vizualizarea semnalului original. Se specifică poziția și dimensiunea axei, culoarea de fundal, dimensiunea fontului și culorile pentru axele X și Y. Apoi, se setează titlul graficului ('Semnal Original'), etichetele pentru axele X ('Timp (s)') și Y ('Amplitudine'), și se activează grila.

```
% Axă semnal modulat
ax_mod_x = ax1_x + plot_width + horizontal_space;
ax_mod_y = ax1_y;
ax_mod = axes('Units', 'normalized', 'Position', [ax_mod_x, ax_mod_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, ...
    'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul modulat
title_obj = title(ax_mod, 'Semnal Modulat', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_mod, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_mod, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_mod, 'on');
set(ax_mod, 'GridColor', textColor); % Activează grila
y_top = y_top - plot_height - vertical_space;
```

Aici se creează a doua axă (ax_mod) pentru vizualizarea semnalului modulat. Poziția sa este calculată în funcție de poziția primei axe, lățimea graficului și spațierea orizontală. Similar cu prima axă, se setează proprietățile pentru aspect, titlu, etichete și grilă.

```
% Linia 2
% Axă semnal demodulat
ax2_x = 0.03;
ax2_y = y_top - plot_height;
ax2 = axes('Units', 'normalized', 'Position', [ax2_x, ax2_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, ...
    'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul demodulat
title_obj = title(ax2, 'Semnal Demodulat', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax2, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax2, 'Amplitudine', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax2, 'on');
set(ax2, 'GridColor', textColor); % Activează grila
```

Această parte ajustează poziția verticală pentru următoarea linie de grafice și apoi creează a treia axă (ax2) pentru vizualizarea semnalului demodulat. Din nou, se definesc poziția, dimensiunea, aspectul, titlul și etichetele axelor.

```
% Axă semnal filtrat
ax_filt_x = ax2_x + plot_width + horizontal_space;
ax_filt_y = ax2_y;
ax_filt = axes('Units', 'normalized', 'Position', [ax_filt_x, ax_filt_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, ...
    'XColor', textColor, 'YColor', textColor); % Creează axa pentru semnalul filtrat
title_obj = title(ax_filt, 'Semnal Filtru', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_filt, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_filt, 'Amplitudine', 'Color', textColor, ...
    'FontSize', fontSize); % Eticheta axei y
grid(ax_filt, 'on');
set(ax_filt, 'GridColor', textColor); % Activează grila
y_top = y_top - plot_height - vertical_space;
```

Secțiunea creează a patra axă (ax_filt) pentru a vizualiza semnalul după aplicarea filtrului. Se definește poziția, dimensiunea și aspectul vizual (culoare, font, culori axe, grilă), împreună cu titlul ('Semnal Filtru') și etichetele axelor.

```
% Linia 3
% Axă eroare mod/demod
ax3_x = 0.03;
ax3_y = y_top - plot_height;
ax3 = axes('Units', 'normalized', 'Position', [ax3_x, ax3_y, plot_width, plot_height], ...
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, ...
    'XColor', textColor, 'YColor', textColor); % Creează axa pentru eroarea de modulare/demodulare
title_obj = title(ax3, 'Eroare Mod/Demod', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax3, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax3, 'Amplitudine', 'Color', textColor, ...
    'FontSize', fontSize); % Eticheta axei y
grid(ax3, 'on');
set(ax3, 'GridColor', textColor); % Activează grila
```

Aici, se ajustează poziția verticală pentru a treia linie de grafice și se creează a cincea axă (ax3) pentru a vizualiza eroarea dintre semnalul original și cel demodulat. Se setează poziția, dimensiunea, aspectul, titlul ('Eroare Mod/Demod') și etichetele axelor.

```
% Axă SNR
ax_snr_x = ax3_x + plot_width + horizontal_space;
ax_snr_y = ax3_y;
ax_snr = axes('Units', 'normalized', 'Position', [ax_snr_x, ax_snr_y, plot_width, plot_height], ..
    'Color', [0.3 0.3 0.3], 'FontSize', fontSize, ...
    'XColor', textColor, 'YColor', textColor); % Creează axa pentru SNR
title_obj = title(ax_snr, 'SNR', 'Color', textColor, ...
    'FontSize', fontSize); % Setează titlul graficului
title_pos = get(title_obj, 'Position');
set(title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset, title_pos(3)]);
xlabel(ax_snr, 'Timp (s)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei x
ylabel(ax_snr, 'SNR (dB)', 'Color', textColor, 'FontSize', fontSize); % Eticheta axei y
grid(ax_snr, 'on');
set(ax_snr, 'GridColor', textColor); % Activează grila
```

În ultimă parte pentru vizualizări, se creează a șasea axă (ax_snr) pentru a afișa valoarea Signal-to-Noise Ratio (SNR). Se configurează poziția, dimensiunea, aspectul, titlul ('SNR') și etichetele axelor (pe Y este 'SNR (dB)').

De notat că pe axa X este pusă eticheta 'Timp (s)', deși de obicei SNR este o singură valoare, deci axa X ar putea fi interpretată diferit în contextul plotării SNR.

Exemple pentru modulatia Delta Fixa

Exemplul 1:

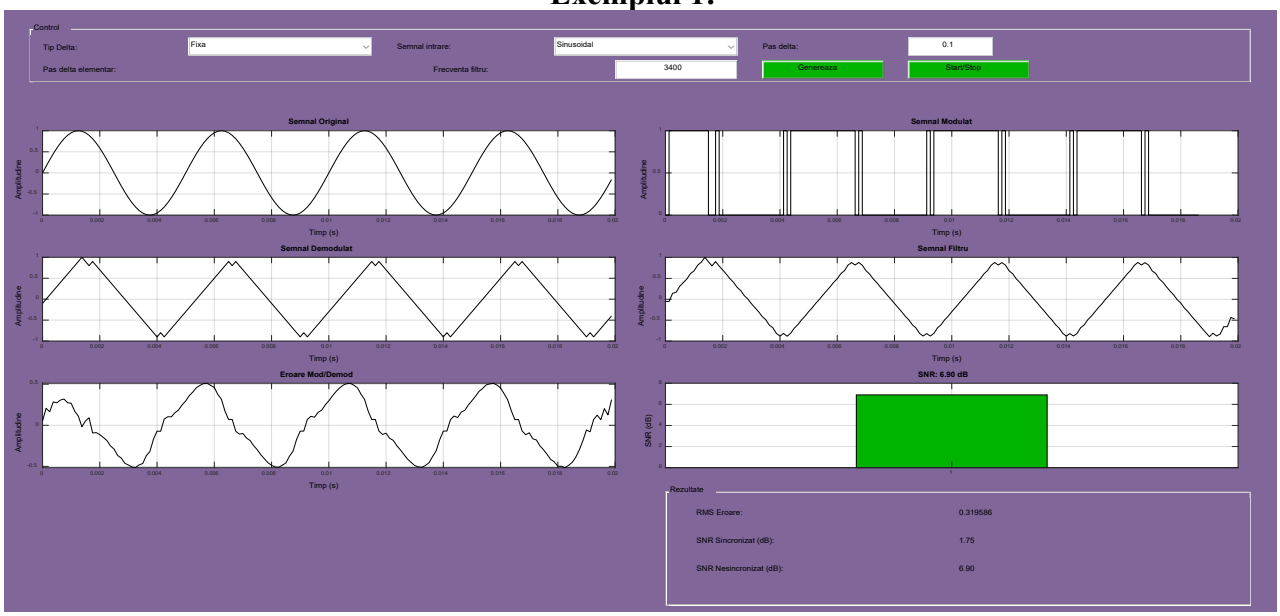

Figura 2.8: Modulație Delta Fixă a unui Semnal Sinusoidal

În această simulare, s-a aplicat modulația Delta fixă unui semnal sinusoidal, utilizând un pas de cuantizare (delta) de 0.1 și o frecvență de tăiere a filtrului de 3400 Hz.

Graficele rezultate ilustrează etapele procesului: semnalul original sinusoidal, semnalul modulat (o secvență binară), semnalul demodulat (o aproximare triunghiulară), semnalul filtrat (o reconstrucție mai netedă), eroarea dintre semnalul original și cel filtrat, și valoarea SNR de 6.90 dB. Panoul de rezultate numerice indică o eroare RMS de aproximativ 0.03195 și valori SNR de 1.75 dB (sincronizat) și 6.90 dB (nesincronizat).

Exemplul 2 – Audio:

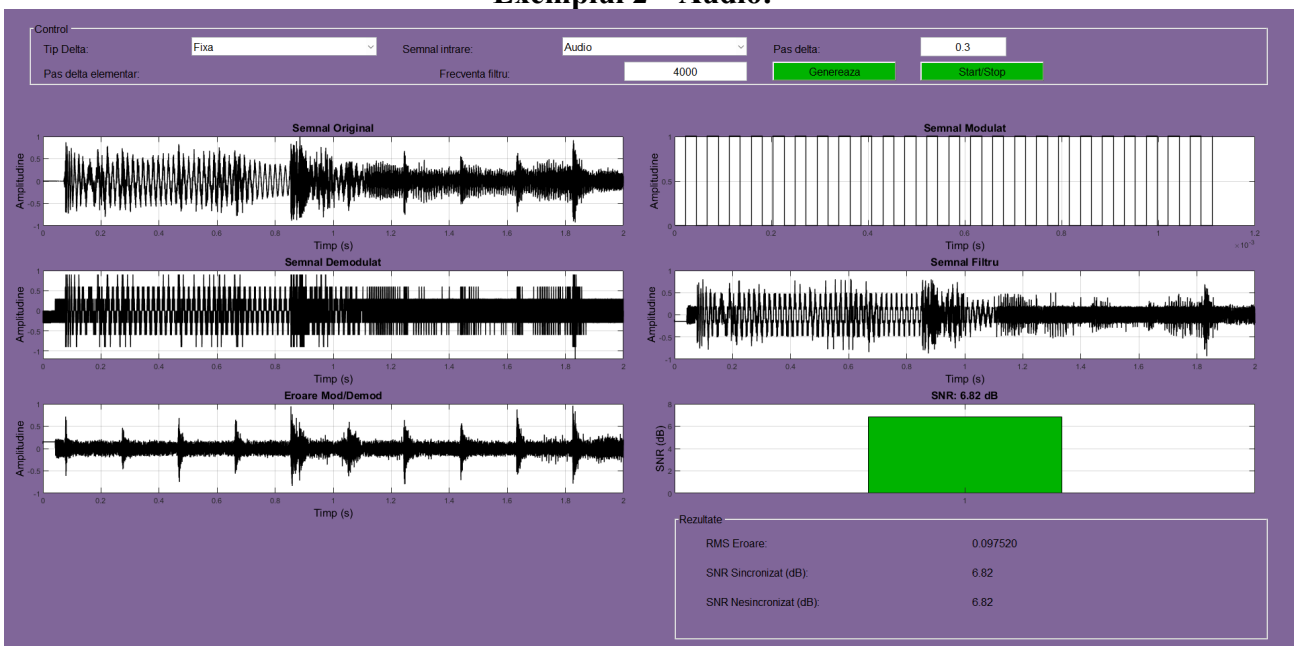

Figura 2.9: Modulație Delta Fixă Aplicată unui Semnal Audio

În această simulare, s-a utilizat modulația Delta fixă pentru a procesa un semnal audio, cu un pas de cuantizare (delta) setat la 0.3 și o frecvență de tăiere a filtrului de 4000 Hz.

Graficele afișează: semnalul audio original (variabil și complex), semnalul modulat (o undă pătrată binară care indică creșterile și descreșterile semnalului audio), semnalul demodulat (o aproximare "în trepte" a semnalului audio), semnalul filtrat (o tentativă de reconstrucție mai lină a semnalului audio), eroarea dintre semnalul audio original și cel filtrat, și un SNR de 6.62 dB.

Rezultatele numerice indică o eroare RMS de 0.097520, iar valorile SNR sincronizat și nesincronizat sunt ambele de 6.82 dB.

Se poate observa că modulația Delta urmărește variațiile amplitudinii semnalului audio, generând un semnal binar corespunzător. Demodularea produce o versiune brută a semnalului audio, iar filtrarea încearcă să îmbunătățească calitatea acestuia.

Valoarea SNR oferă o măsură a fidelității cu care semnalul audio este reprodus după procesul de modulație și demodulare. În acest caz, SNR-ul este relativ scăzut, sugerând o diferență semnificativă între semnalul original și cel reconstruit.

Exemple pentru Modulatia Delta Adaptivă Song

Exemplul 1:

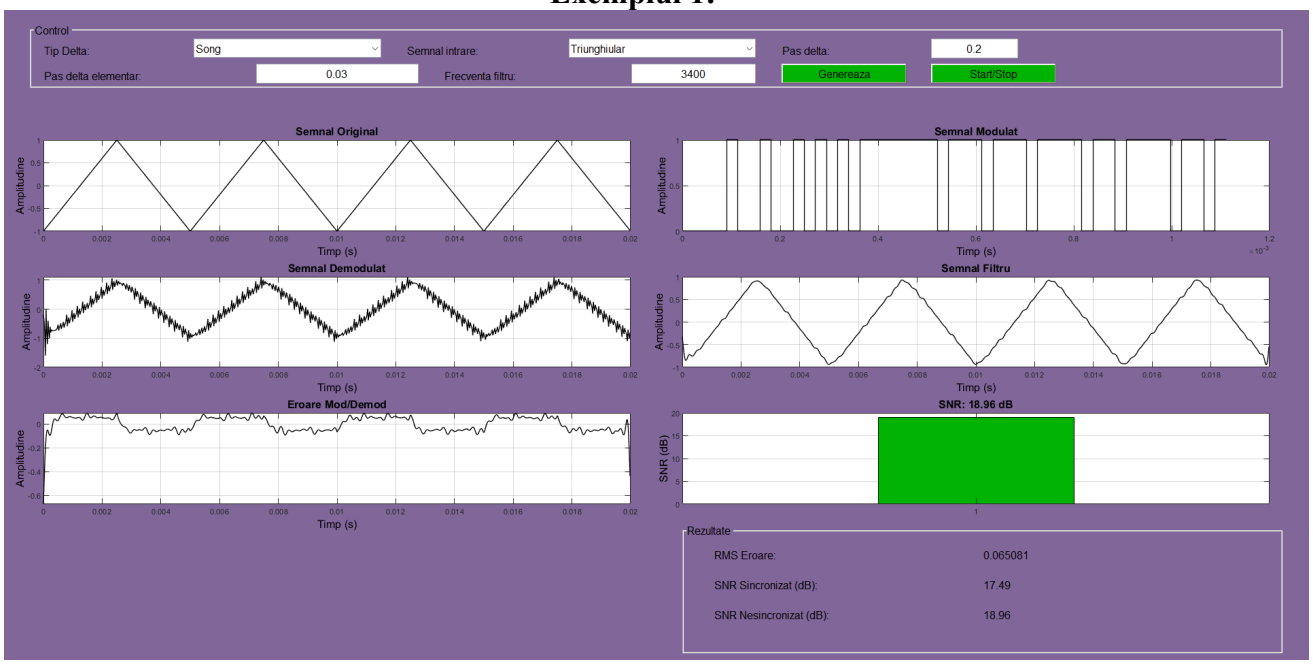

Figura 2.10: Modulație Delta Adaptivă Song Aplicată unui Semnal Triunghiular

În această simulare, s-a utilizat modulația Delta adaptivă de tip Song pentru a procesa un semnal triunghiular. Parametrii selectați includ un pas delta inițial de 0.2 și un pas delta elementar de 0.03, cu o frecvență de tăiere a filtrului de 3400 Hz.

Graficele afișează: semnalul original triunghiular, semnalul modulat (o secvență binară care reflectă adaptarea pasului), semnalul demodulat (o aproximare "în trepte" a semnalului triunghiular), semnalul filtrat (o reconstrucție mai lină), eroarea dintre semnalul original și cel filtrat este de 0.065081. Rezultatele numerice indică o eroare RMS de 0.065081, un SNR sincronizat de 17.49 dB și un SNR nesincronizat de 18.96 dB.

Se observă că, spre deosebire de modulația Delta fixă, în cea adaptivă Song, pasul de cuantizare variază în timp, încercând să urmărească mai bine panta semnalului triunghiular. Acest lucru se reflectă într-un semnal modulat cu o densitate variabilă a tranzițiilor.

Semnalul filtrat pare să apropie mai bine forma triunghiulară originală, iar valoarea SNR de aproape 19 dB sugerează o calitate a reconstrucției superioară comparativ cu exemplul cu modulație Delta fixă.

Exemplul 2 – Audio:

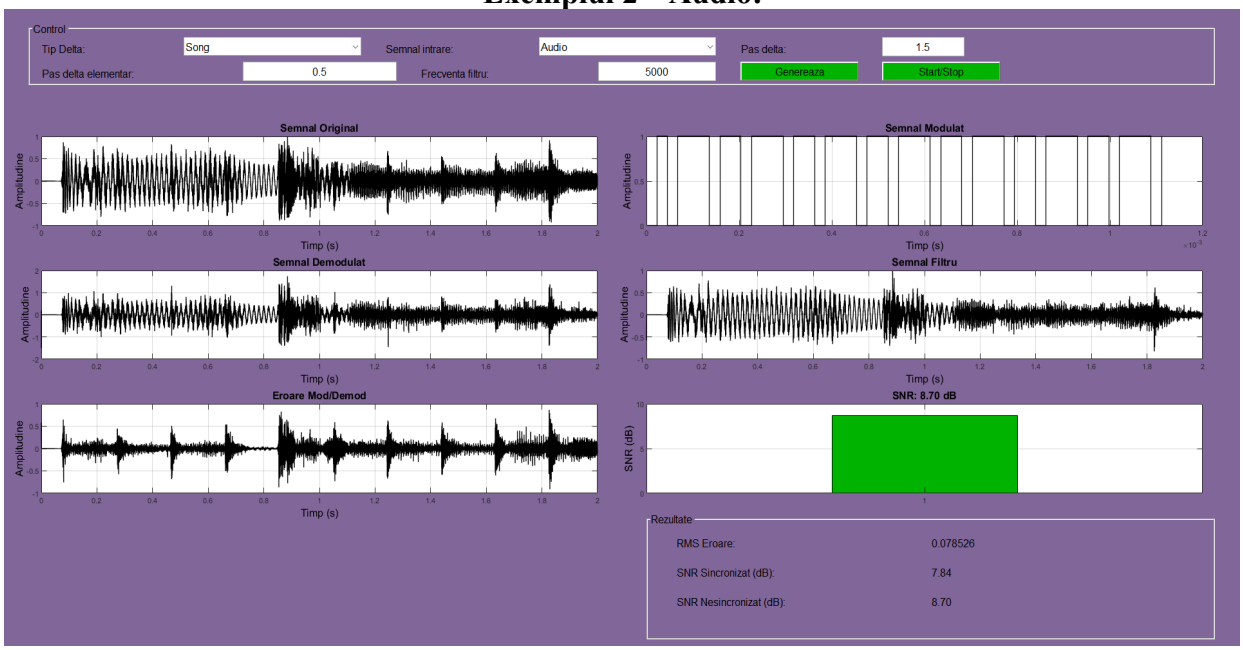

Figura 2.11: Modulație Delta Adaptivă Song Aplicată unui Semnal Audio

În această simulare, s-a aplicat modulația Delta adaptivă de tip Song unui semnal audio, cu un pas delta inițial de 1.5 și un pas delta elementar de 0.5, iar frecvența de tăiere a filtrului a fost setată la 5000 Hz.

Graficele vizualizează: semnalul audio original, semnalul modulat (o undă pătrată binară cu o densitate a tranzițiilor variabilă), semnalul demodulat (aproximarea în trepte a semnalului audio), semnalul filtrat (reconstrucția netezită a semnalului audio), eroarea dintre semnalul original și cel filtrat, și un SNR de 8.70 dB.

Rezultatele numerice indică o eroare RMS de 0.078526, un SNR sincronizat de 7.84 dB și un SNR nesincronizat de 8.70 dB.

Exemple pentru Modulatia Delta Adaptivă Jayant

Exemplul 1:

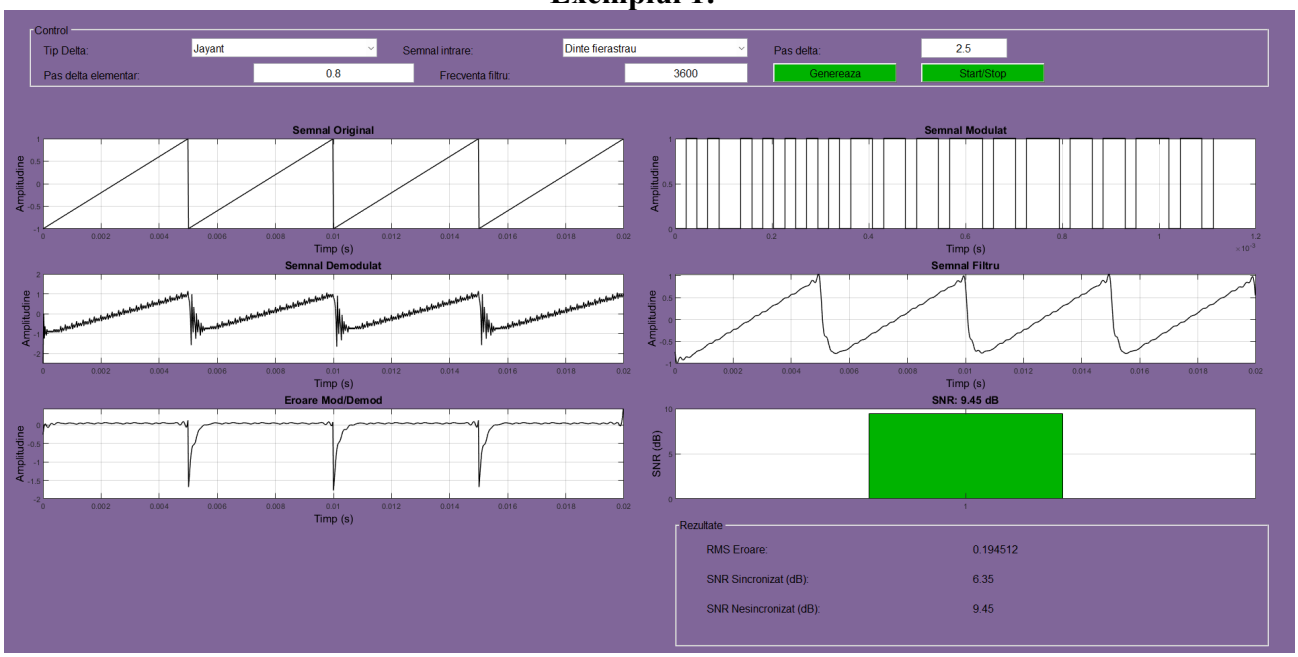

Figura 2.12: Modulație Delta Adaptivă Jayant Aplicată unui Semnal Dinte de Fierăstrău

În această simulare, s-a utilizat modulația Delta adaptivă de tip Jayant pentru a procesa un semnal dinte de fierăstrău, cu un pas delta inițial de 2.5 și un pas delta elementar de 0.8, iar frecvența de tăiere a filtrului a fost setată la 3600 Hz.

Graficele ilustrează: semnalul original dinte de fierăstrău, semnalul modulat (o secvență binară care reflectă adaptarea pasului specifică algoritmului Jayant), semnalul demodulat (o aproximare "în trepte" a semnalului dinte de fierăstrău), semnalul filtrat (o reconstrucție mai netedă), eroarea dintre semnalul original și cel filtrat, și un SNR de 9.45 dB.

Rezultatele numerice indică o eroare RMS de 0.194512, un SNR sincronizat de 6.35 dB și un SNR nesincronizat de 9.45 dB.

Exemplul 2 – Audio:

Figura 2.13: Modulație Delta Adaptivă Jayant Aplicată unui Semnal Audio

În această simulare, s-a aplicat modulația Delta adaptivă de tip Jayant unui semnal audio, cu un pas delta inițial de 5.5 și un pas delta elementar de 1.8, iar frecvența de tăiere a filtrului a fost setată la 5200 Hz.

Graficele afișează: semnalul audio original, semnalul modulat (o secvență binară cu o densitate a tranzițiilor variabilă, influențată de algoritmul Jayant), semnalul demodulat (aproximarea în trepte a semnalului audio), semnalul filtrat (reconstrucția netezită), eroarea dintre semnalul original și cel filtrat, și un SNR de 6.65 dB.

Rezultatele numerice indică o eroare RMS de 0.097196, iar valorile SNR sincronizat și nesincronizat sunt ambele de 6.65 dB.

Se observă modul în care algoritmul adaptiv Jayant ajustează pasul de cuantizare în funcție de istoricul semnalului modulat, încercând să urmărească cât mai bine variațiile amplitudinii semnalului audio. Semnalul filtrat reprezintă reconstrucția semnalului audio, iar valoarea SNR de 6.65 dB oferă o măsură a calității acestei reconstrucții.

Comparativ cu utilizarea modulației Song pe un semnal audio, se pot observa diferențe în semnalul modulat și în valoarea SNR, reflectând caracteristicile specifice ale algoritmului de adaptare Jayant.

Studiul SNR-ului pentru modulația Delta fixă pentru un semnal periodic și un semnal audio pe baza valorii pasului de cuantizare.

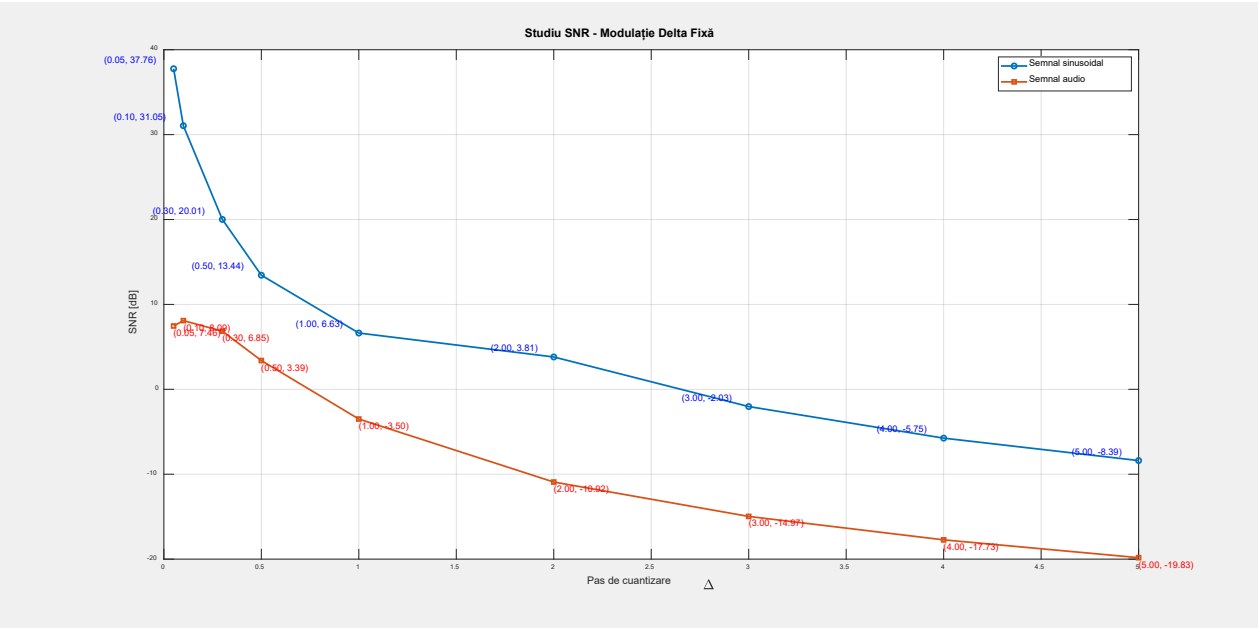

Figura 2.14: Dependența SNR de Pasul de Cuantizare pentru Modulația Delta Fixă aplicată unui Semnal Sinusoidal și Audio

Graficul ilustrează dependența raportului semnal-zgomot (SNR) de valoarea pasului de cuantizare (Δ) pentru modulația Delta fixă, comparând răspunsul pentru un semnal sinusoidal și un semnal audio. Ambele semnale au fost procesate cu o frecvență de tăiere a filtrului de 3400 Hz.

Pentru **semnalul sinusoidal** (curba albastră), se observă o tendință generală de scădere a SNR-ului pe măsură ce pasul de cuantizare crește. La valori mici ale pasului Δ (0.05 și 0.1), se obțin valori SNR ridicate (37.76 dB și respectiv 31.05 dB), indicând o reprezentare mai fidelă a semnalului. Pe măsură ce Δ crește, SNR-ul scade semnificativ, devenind negativ pentru valori mai mari de 2. Acest lucru sugerează că un pas de cuantizare mic este preferabil pentru un semnal sinusoidal în contextul modulației Delta fixe pentru a minimiza zgomotul de cuantizare.

Pentru **semnalul audio** (curba portocalie), dependența SNR de pasul de cuantizare este diferită. Inițial, SNR-ul crește ușor de la Δ=0.05 (7.46 dB) la Δ=0.1 (8.09 dB), sugerând că o creștere mică a pasului poate îmbunătăți temporar SNR-ul. Ulterior, similar cu semnalul sinusoidal, SNR-ul scade odată cu creșterea lui Δ, devenind negativ pentru valori mai mari de 1. Valorile SNR pentru semnalul audio sunt în general mai scăzute decât cele pentru semnalul sinusoidal în același interval al pasului de cuantizare, indicând o performanță mai slabă a modulației Delta fixe pentru semnale audio în comparație cu semnale periodice simple ca sinusoida, cel puțin în acest interval de parametri.

Secționarea codului și explicații:

```
% Valorile pasului delta
delta = [0.05, 0.1, 0.3, 0.5, 1, 2, 3, 4, 5];
```

Această secțiune definește un vector numit delta care conține diferitele valori ale pasului de cuantizare (Δ) pentru care ați efectuat studiul SNR.

```
% SNR pentru semnal sinusoidal
snr_sinus = [37.76, 31.05, 20.01, 13.44, 6.63, 3.81, -2.03, -5.75, -8.39];

% SNR pentru semnal audio
snr_audio = [7.46, 8.09, 6.85, 3.39, -3.5, -10.92, -14.97, -17.73, -19.83];
```

Aici sunt definiți doi vectori: snr_sinus care stochează valorile SNR obținute pentru semnalul sinusoidal corespunzătoare valorilor pasului delta din vectorul delta, și snr_audio care stochează valorile SNR pentru semnalul audio, de asemenea corespunzătoare valorilor pasului delta.

```
% Graficul
figure;
plot(delta, snr_sinus, '-o', 'LineWidth', 2);
hold on;
plot(delta, snr_audio, '-s', 'LineWidth', 2);
grid on;
```

Această secțiune generează graficul principal, trasând curbele SNR în funcție de pasul delta pentru semnalul sinusoidal (linie cu cercuri) și semnalul audio (linie cu pătrate), și activează grila.

```
% Adăugare etichete text pentru fiecare punct (sinus)
for i = 1:length(delta)
    text(delta(i), snr_sinus(i), sprintf('(%.2f, %.2f)', delta(i), snr_sinus(i)), ...
        'VerticalAlignment', 'bottom', 'HorizontalAlignment', 'right', ...
        'FontSize', 12, 'Color', 'b');
end

% Adăugare etichete text pentru fiecare punct (audio)
for i = 1:length(delta)
    text(delta(i), snr_audio(i), sprintf('(%.2f, %.2f)', delta(i), snr_audio(i)), ...
        'VerticalAlignment', 'top', 'HorizontalAlignment', 'left', ...
        'FontSize', 12, 'Color', 'r');
end
```

Aici cele două bucle for adaugă etichete text lângă fiecare punct de pe grafic, indicând coordonatele (pas delta, SNR). Pentru semnalul sinusoidal, textul este albastru ('Color', 'b') și aliniat în partea de jos dreapta. Pentru semnalul audio, textul este roșu ('Color', 'r') și aliniat în partea de sus stânga.

```
% Legenda și titluri
xlabel('Pas de cuantizare \Delta');
ylabel('SNR [dB]');
title('Studiu SNR - Modulație Delta fixă');
legend('Semnal sinusoidal', 'Semnal audio');
```

Ultimă parte setează eticheta axei x, eticheta axei y, titlul graficului și legenda care identifică cele două curbe.

În concluzie, un pas de cuantizare mic tinde să ofere un SNR mai bun pentru semnalul sinusoidal în modulația Delta fixă. Pentru semnalul audio, relația este mai complexă, dar în general, creșterea pasului de cuantizare duce la o degradare a SNR-ului după un punct inițial.

Graficul permite vizualizarea directă a compromisului dintre pasul de cuantizare și calitatea semnalului cuantizat (măsurată prin SNR) pentru ambele tipuri de semnale.

3. Script Trei - PCM neuniform folosind tabelele de codare
(pentru un punct bonus)

Explicarea interfeței PCM neuniform

Acest script MATLAB creează o interfață grafică utilizator (GUI) pentru a demonstra PCM neuniform, utilizând tabelele de codare definite pentru legea de compresie μ și pentru legea de compresie A.

GUI permite generarea diferitelor tipuri de semnale, specificarea numărului de biți pentru cuantizare și alegerea legii de compresie. Rezultatele cuantizării sunt vizualizate grafic, iar performanța este evaluată prin eroarea RMS, raportul semnal-zgomot (SNR), totodată este afișată și rata biților.

Implementarea unei interfețe grafice

Codul MATLAB furnizat în **anexa** de la sfârșitul documentației implementează o interfață grafică (GUI) pentru a demonstra și analiza modularea PCM (Pulse Code Modulation) neuniformă.

Aplicația permite utilizatorului să genereze diferite tipuri de semnale (sinusoidal, dreptunghiular, triunghiular, dinte de fierăstrău, audio), să aplice cuantizarea neuniformă folosind legile de compresie μ sau A, și să vizualizeze atât semnalul original cât și cel cuantizat, eroarea de cuantizare și SNR-ul (Signal-to-Noise Ratio). De asemenea, sunt afișate valorile numerice ale RMS-ului erorii, SNR-ului și ratei de biți.

Componente și Funcționalități Cheie

1. **Interfață Grafică Utilizator (GUI)**

- Codul începe prin a defini parametrii vizuali ai GUI-ului (culori, fonturi, etc.) pentru a asigura o prezentare clară și consistentă.
- Este creată o fereastră principală care conține un panou de control unde utilizatorul poate selecta tipul semnalului, numărul de biți pentru cuantizare și legea de compresie (μ-law sau A-law).
- GUI-ul include, de asemenea, axe pentru afișarea grafică a semnalelor și un panou text pentru afișarea rezultatelor numerice.

2. **Generarea Semnalului**

- Funcția generezaSemnal este crucială. Aceasta preia inputurile utilizatorului din GUI.
- Generează semnale sinusoidale, dreptunghiulare, triunghiulare, dinte de fierăstrău sau permite încărcarea unui fișier audio.

3. **Cuantizare Neuniformă**

- Implementează cuantizarea neuniformă folosind tabelele de codare standard pentru μ-law și A-law.
- Codul normalizează semnalul de intrare, determină segmentul corespunzător conform tabelei, cuantizează valoarea și aplică inversarea biților (pentru A-law) conform standardului ITU-T G.711.

4. **Calculul și Afișarea Rezultatelor**

- Calculează eroarea de cuantizare (diferența dintre semnalul original și cel cuantizat), RMS-ul erorii și SNR-ul.
- Afișează grafic semnalele (original, cuantizat, eroare) și SNR-ul.
- Actualizează panoul de rezultate numerice cu valorile calculate.

5. **Redare Audio**

- Funcția playStop permite utilizatorului să asculte semnalul cuantizat, oferind o evaluare subiectivă a calității audio după cuantizare.

- **Implementarea PCM Neuniform:** Codul implementează clar PCM neuniform folosind tabelele de codare pentru μ-law și A-law, așa cum se specifică în standardul G.711.

- **Analiza Semnalului Sinusoidal:** Codul este optimizat pentru a oferi o analiză detaliată și vizualizare pentru orice semnal, inclusiv sinusoidal. Prin modificarea parametrilor (număr de biți, legea de compresie), se pot genera multiple exemple și observa efectul acestora asupra erorii de cuantizare și SNR.

Descrierea și interpretarea rezultatelor cu semnal sinusoidal pentru minim 2 exemple.

Exemplul 1:

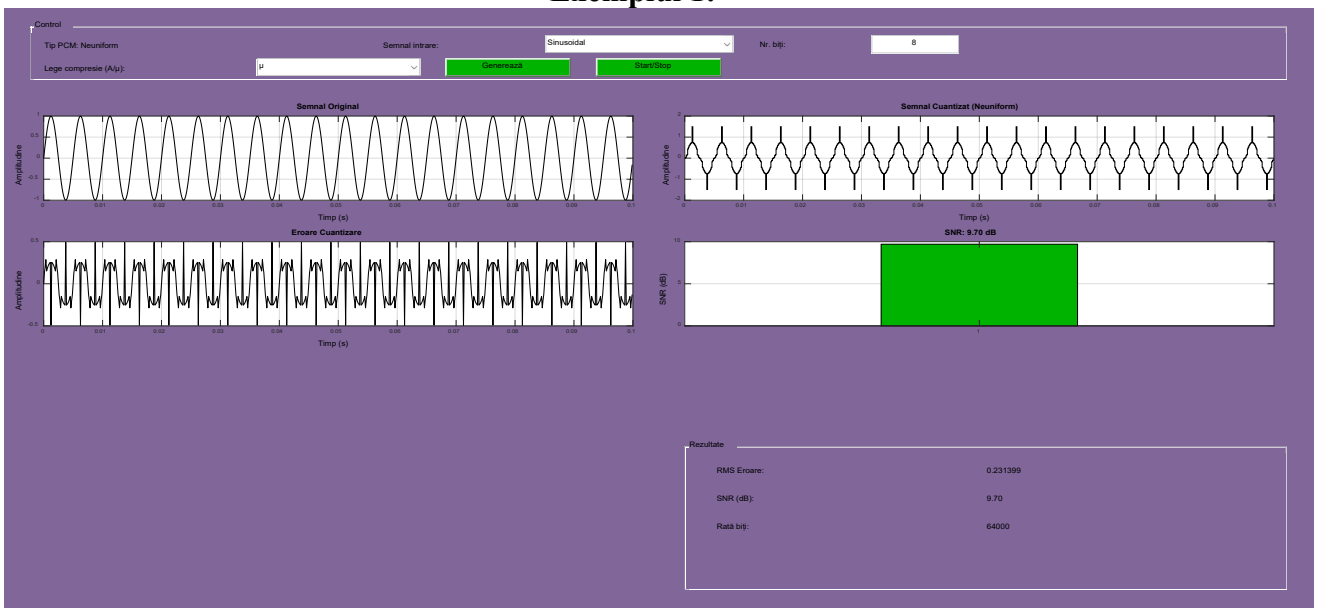

Figura 3.1. Cuantizarea Neuniformă a unui Semnal Sinusoidal cu 8 biți și Legea μ

Figura prezintă rezultatele cuantizării neuniforme a unui semnal sinusoidal, proces realizat cu 8 biți și legea de compresie μ.

Interfața grafică din partea superioară arată selecția parametrilor de procesare, respectiv tipul semnalului ("Sinusoidal"), numărul de biți ("8") și legea de compresie ("μ").

Graficul din stânga sus ilustrează semnalul sinusoidal original, oferind referința pentru evaluarea modificărilor introduse de cuantizare. În contrast, graficul din dreapta sus prezintă semnalul cuantizat, unde se observă aproximarea discretă a semnalului original prin trepte verticale, a căror distanță inegală reflectă compresia amplitudinii specifică legii de compresie μ.

Eroarea de cuantizare, reprezentată în graficul din stânga jos, indică diferența dintre semnalele original și cuantizat, cu amplitudine mai mare în zonele de variație rapidă a semnalului.

Raportul semnal-zgomot (SNR), afișat în graficul din dreapta jos și confirmat numeric în panoul de rezultate, măsoară calitatea semnalului cuantizat, sugerând o calitate acceptabilă cu valoarea de 9.70 dB. Panoul de rezultate din dreapta jos furnizează, de asemenea, valoarea RMS a erorii (0.231999) și rata de biți (64000), completând analiza procesului de cuantizare neuniformă și a impactului său asupra semnalului sinusoidal.

Exemplul 2:

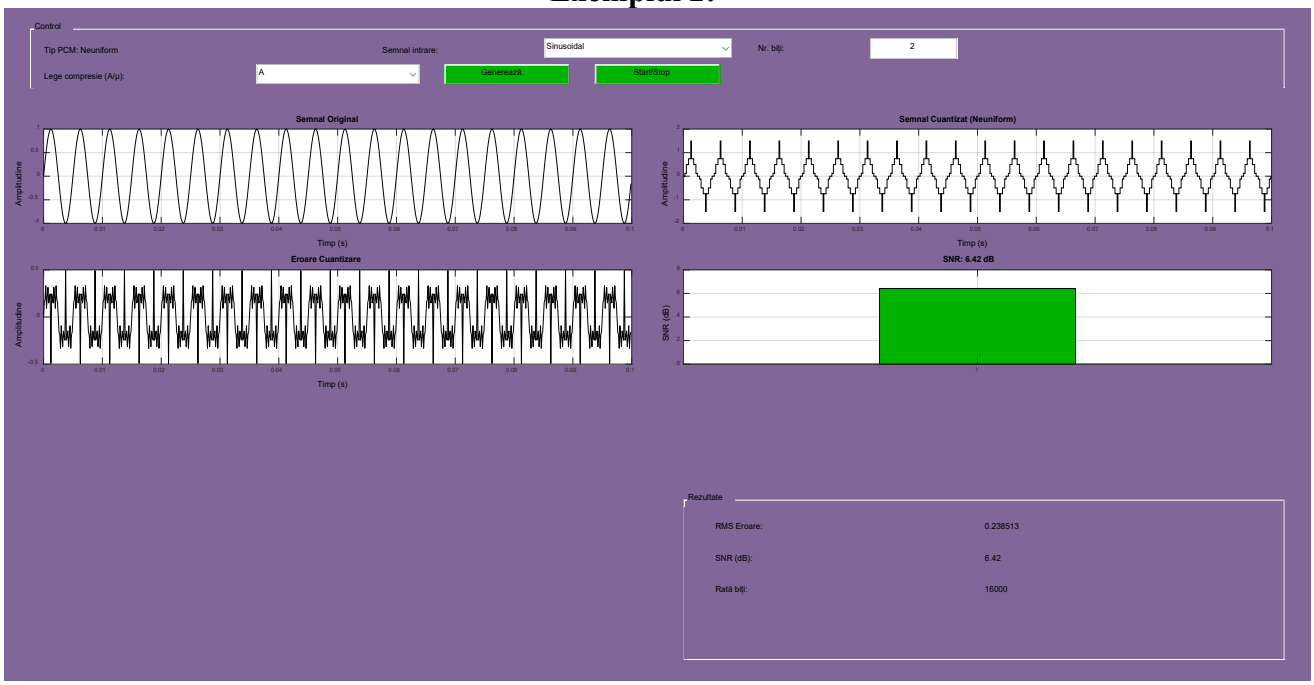

Figura 3.2. Cuantizarea Neuniformă a unui Semnal Sinusoidal cu 2 Biți și Legea A
(Fără Filtru, SNR 20 dB)

Această figură ilustrează cuantizarea neuniformă a unui semnal sinusoidal utilizând doar 2 biți și legea de compresie A. În panoul de control din partea superioară, se observă selecția "Sinusoidal" pentru tipul semnalului, "2" pentru numărul de biți și "A" pentru legea de compresie. Reducerea drastică a numărului de biți are un impact semnificativ asupra calității semnalului cuantizat.

Graficul din stânga sus prezintă semnalul sinusoidal original, neschimbat. În contrast, semnalul cuantizat, afișat în graficul din dreapta sus, este acum o aproximare foarte grosieră a sinusoidei, cu doar câteva niveluri distincte vizibile datorită rezoluției extrem de scăzute de 2 biți

(2^2 = 4 niveluri). Această cuantizare grosieră introduce o eroare semnificativă, așa cum se poate vedea în graficul erorii de cuantizare din stânga jos.

Eroarea are o amplitudine relativ mare și o formă strâns legată de semnalul original, indicând o distorsiune considerabilă.

Graficul din dreapta jos arată SNR-ul rezultat, care este de 6.42 dB. Această valoare scăzută a SNR-ului indică un raport mic între puterea semnalului util și puterea zgomotului de cuantizare, reflectând calitatea slabă a semnalului cuantizat.

Panoul cu rezultate numerice confirmă această valoare a SNR-ului (6.42 dB), alături de o valoare relativ mare a RMS-ului erorii (0.238513) și o rată de biți de 16000.

Exemplul 3:

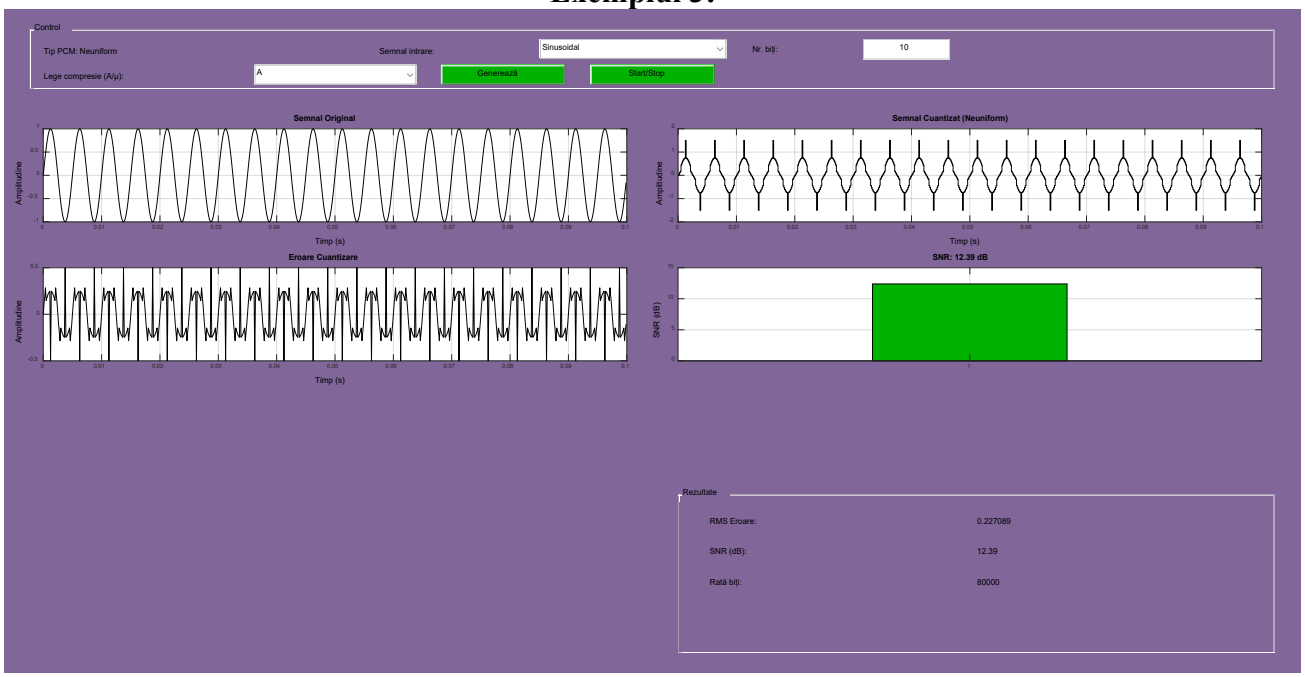

Figura 3.3. Cuantizarea Neuniformă a unui Semnal Sinusoidal cu 10 Biți și Legea A

Această figură ilustrează rezultatele cuantizării neuniforme a unui semnal sinusoidal folosind 10 biți și legea de compresie A. Utilizarea unui număr relativ mare de biți permite o cuantizare mai fină, ceea ce se reflectă într-o calitate superioară a semnalului cuantizat.

Graficul din stânga sus prezintă semnalul sinusoidal original. Semnalul cuantizat, afișat în graficul din dreapta sus, este acum o aproximare mult mai precisă a semnalului original comparativ cu exemplele anterioare cu mai puțini biți. Treptele de cuantizare sunt mult mai fine și mai puțin vizibile. Eroarea de cuantizare, ilustrată în graficul din stânga jos, are o amplitudine semnificativ mai mică, indicând o diferență redusă între semnalul original și cel cuantizat.

Graficul din dreapta jos arată un SNR de 12.39 dB. Această valoare ridicată a SNR-ului indică un raport mult mai mare între puterea semnalului util și puterea zgomotului de cuantizare, sugerând o calitate bună a semnalului cuantizat.

Panoul cu rezultate numerice confirmă acest SNR (12.39 dB), alături de o valoare mică a RMS-ului erorii (0.227089) și o rată de biți de 80000.

Anexă – Script 3

```
function script3_pcm_gui()
    backgroundColor = [0.5 0.4 0.6];
    textColor       = [0 0 0];
    buttonColor     = [0 0.7 0];
    fontSize        = 12;
    lineWidth       = 1;
    f = figure( 'Name', 'PCM Neuniform (Tabele) GUI', ...
                'Units', 'normalized', ...
                'Position', [0.05, 0.05, 0.9, 0.85], ...
                'Color', backgroundColor );
    control_panel = uipanel( 'Parent', f, ...
                             'Title', 'Control', ...
                             'Units', 'normalized', ...
                             'Position', [0.02, 0.88, 0.96, 0.1], ...
                             'BackgroundColor', backgroundColor, ...
                             'ForegroundColor', textColor, ...
                             'FontSize', fontSize );
    uicontrol( 'Parent', control_panel, ...
               'Style', 'text', ...
               'Units', 'normalized', ...
               'Position', [0.01, 0.55, 0.1, 0.3], ...
               'String', 'Tip PCM: Neuniform', ...
               'BackgroundColor', backgroundColor, ...
               'ForegroundColor', textColor, ...
               'FontSize', fontSize, ...
               'HorizontalAlignment', 'left' );
    uicontrol( 'Parent', control_panel, ...
               'Style', 'text', ...
               'Units', 'normalized', ...
               'Position', [0.28, 0.55, 0.13, 0.3], ...
               'String', 'Semnal intrare:', ...
               'BackgroundColor', backgroundColor, ...
               'ForegroundColor', textColor, ...
               'FontSize', fontSize, ...
               'HorizontalAlignment', 'left' );
    signalType = uicontrol( 'Parent', control_panel, ...
                            'Style', 'popupmenu', ...
                            'Units', 'normalized', ...
                            'Position', [0.41, 0.55, 0.15, 0.4], ...
                            'String', {'Sinusoidal', 'Dreptunghiular',
'Triunghiular', 'Dinte fierastrau', 'Audio'}, ...
                            'BackgroundColor', 'white', ...
                            'ForegroundColor', textColor, ...
                            'FontSize', fontSize );
    uicontrol( 'Parent', control_panel, ...
               'Style', 'text', ...
               'Units', 'normalized', ...
               'Position', [0.58, 0.55, 0.09, 0.3], ...
               'String', 'Nr. biți:', ...
```

```
            'BackgroundColor', backgroundColor, ...
            'ForegroundColor', textColor, ...
            'FontSize', fontSize, ...
            'HorizontalAlignment', 'left' );
bitsInput = uicontrol( 'Parent', control_panel, ...
                       'Style', 'edit', ...
                       'Units', 'normalized', ...
                       'Position', [0.67, 0.55, 0.07, 0.4], ...
                       'String', '8', ...
                       'BackgroundColor', 'white', ...
                       'ForegroundColor', textColor, ...
                       'FontSize', fontSize );
uicontrol( 'Parent', control_panel, ...
           'Style', 'text', ...
           'Units', 'normalized', ...
           'Position', [0.01, 0.05, 0.17, 0.3], ...
           'String', 'Lege compresie (A/μ):', ...
           'BackgroundColor', backgroundColor, ...
           'ForegroundColor', textColor, ...
           'FontSize', fontSize, ...
           'HorizontalAlignment', 'left' );
lawSelector = uicontrol( 'Parent', control_panel, ...
                         'Style', 'popupmenu', ...
                         'Units', 'normalized', ...
                         'Position', [0.18, 0.05, 0.13, 0.4], ...
                         'String', {'μ', 'A'}, ...
                         'BackgroundColor', 'white', ...
                         'ForegroundColor', textColor, ...
                         'FontSize', fontSize );
uicontrol( 'Parent', control_panel, ...
           'Style', 'pushbutton', ...
           'Units', 'normalized', ...
           'Position', [0.33, 0.05, 0.1, 0.4], ...
           'String', 'Generează', ...
           'Callback', @genereazaSemnal, ...
           'BackgroundColor', buttonColor, ...
           'ForegroundColor', textColor, ...
           'FontSize', fontSize );
uicontrol( 'Parent', control_panel, ...
           'Style', 'pushbutton', ...
           'Units', 'normalized', ...
           'Position', [0.45, 0.05, 0.1, 0.4], ...
           'String', 'Start/Stop', ...
           'Callback', @playStop, ...
           'BackgroundColor', buttonColor, ...
           'ForegroundColor', textColor, ...
           'FontSize', fontSize );
vertical_space   = 0.07;
horizontal_space = 0.04;
plot_width       = 0.45;
plot_height      = 0.14;
y_top            = 0.82;
```

```
    title_y_offset   = 1.05;
    ax1_x = 0.03;
    ax1_y = y_top - plot_height;
    ax1 = axes( 'Units', 'normalized', ...
                'Position', [ax1_x, ax1_y, plot_width, plot_height], ...
                'Color', [0.3 0.3 0.3], ...
                'FontSize', fontSize, ...
                'XColor', textColor, ...
                'YColor', textColor );
    title_obj = title( ax1, 'Semnal Original', ...
                       'Color', textColor, ...
                       'FontSize', fontSize );
    title_pos = get( title_obj, 'Position' );
    set( title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset,
title_pos(3)] );
    xlabel( ax1, 'Timp (s)', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    ylabel( ax1, 'Amplitudine', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    grid( ax1, 'on' );
    set( ax1, 'GridColor', textColor );
    ax2_x = ax1_x + plot_width + horizontal_space;
    ax2_y = ax1_y;
    ax2 = axes( 'Units', 'normalized', ...
                'Position', [ax2_x, ax2_y, plot_width, plot_height], ...
                'Color', [0.3 0.3 0.3], ...
                'FontSize', fontSize, ...
                'XColor', textColor, ...
                'YColor', textColor );
    title_obj = title( ax2, 'Semnal Cuantizat (Neuniform)', ...
                       'Color', textColor, ...
                       'FontSize', fontSize );
    title_pos = get( title_obj, 'Position' );
    set( title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset,
title_pos(3)] );
    xlabel( ax2, 'Timp (s)', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    ylabel( ax2, 'Amplitudine', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    grid( ax2, 'on' );
    set( ax2, 'GridColor', textColor );
    y_top = y_top - plot_height - vertical_space;
    ax3_x = 0.03;
    ax3_y = y_top - plot_height;
    ax3 = axes( 'Units', 'normalized', ...
                'Position', [ax3_x, ax3_y, plot_width, plot_height], ...
                'Color', [0.3 0.3 0.3], ...
                'FontSize', fontSize, ...
```

```
                 'XColor', textColor, ...
                 'YColor', textColor );
     title_obj = title( ax3, 'Eroare Cuantizare', ...
                        'Color', textColor, ...
                        'FontSize', fontSize );
     title_pos = get( title_obj, 'Position' );
     set( title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset,
title_pos(3)] );
     xlabel( ax3, 'Timp (s)', ...
             'Color', textColor, ...
             'FontSize', fontSize );
     ylabel( ax3, 'Amplitudine', ...
             'Color', textColor, ...
             'FontSize', fontSize );
     grid( ax3, 'on' );
     set( ax3, 'GridColor', textColor );
     ax4_x = ax3_x + plot_width + horizontal_space;
     ax4_y = ax3_y;
     ax4 = axes( 'Units', 'normalized', ...
                 'Position', [ax4_x, ax4_y, plot_width, plot_height], ...
                 'Color', [0.3 0.3 0.3], ...
                 'FontSize', fontSize, ...
                 'XColor', textColor, ...
                 'YColor', textColor );
     title_obj = title( ax4, 'SNR', ...
                        'Color', textColor, ...
                        'FontSize', fontSize );
     title_pos = get( title_obj, 'Position' );
     set( title_obj, 'Position', [title_pos(1), title_pos(2) * title_y_offset,
title_pos(3)] );
     xlabel( ax4, 'Timp (s)', ...
             'Color', textColor, ...
             'FontSize', fontSize );
     ylabel( ax4, 'SNR (dB)', ...
             'Color', textColor, ...
             'FontSize', fontSize );
     grid( ax4, 'on' );
     set( ax4, 'GridColor', textColor );
     resultsPanel_y      = 0.03;
     resultsPanel_height = 0.25;
     resultsPanel = uipanel( 'Title', 'Rezultate', ...
                             'Units', 'normalized', ...
                             'Position', [0.52, resultsPanel_y, 0.46,
resultsPanel_height], ...
                             'BackgroundColor', backgroundColor, ...
                             'ForegroundColor', textColor, ...
                             'FontSize', fontSize );
     y_rez_top  = 0.9;
     rez_height = 1 / (5 + 4 * 0.2);
     rez_space  = 0.2 * rez_height;
     uicontrol( 'Parent', resultsPanel, ...
                'Style', 'text', ...
```

```
                'Units', 'normalized', ...
                'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], ...
                'String', 'RMS Eroare:', ...
                'BackgroundColor', backgroundColor, ...
                'ForegroundColor', textColor, ...
                'FontSize', fontSize, ...
                'HorizontalAlignment', 'left' );
     rmsErrorText = uicontrol( 'Parent', resultsPanel, ...
                               'Style', 'text', ...
                               'Units', 'normalized', ...
                               'Position', [0.5, y_rez_top - rez_height, 0.45,
rez_height], ...
                               'String', '-', ...
                               'BackgroundColor', backgroundColor, ...
                               'ForegroundColor', textColor, ...
                               'FontSize', fontSize, ...
                               'HorizontalAlignment', 'left' );
     y_rez_top = y_rez_top - rez_height - rez_space;
     uicontrol( 'Parent', resultsPanel, ...
                'Style', 'text', ...
                'Units', 'normalized', ...
                'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], ...
                'String', 'SNR (dB):', ...
                'BackgroundColor', backgroundColor, ...
                'ForegroundColor', textColor, ...
                'FontSize', fontSize, ...
                'HorizontalAlignment', 'left' );
     snrText = uicontrol( 'Parent', resultsPanel, ...
                          'Style', 'text', ...
                          'Units', 'normalized', ...
                          'Position', [0.5, y_rez_top - rez_height, 0.45,
rez_height], ...
                          'String', '-', ...
                          'BackgroundColor', backgroundColor, ...
                          'ForegroundColor', textColor, ...
                          'FontSize', fontSize, ...
                          'HorizontalAlignment', 'left' );
     y_rez_top = y_rez_top - rez_height - rez_space;
     uicontrol( 'Parent', resultsPanel, ...
                'Style', 'text', ...
                'Units', 'normalized', ...
                'Position', [0.05, y_rez_top - rez_height, 0.45, rez_height], ...
                'String', 'Rată biți:', ...
                'BackgroundColor', backgroundColor, ...
                'ForegroundColor', textColor, ...
                'FontSize', fontSize, ...
                'HorizontalAlignment', 'left' );
     bitRateText = uicontrol( 'Parent', resultsPanel, ...
                              'Style', 'text', ...
                              'Units', 'normalized', ...
                              'Position', [0.5, y_rez_top - rez_height, 0.45,
rez_height], ...
```

```
                            'String', '-', ...
                            'BackgroundColor', backgroundColor, ...
                            'ForegroundColor', textColor, ...
                            'FontSize', fontSize, ...
                            'HorizontalAlignment', 'left' );
    global semnal_fe semnal_orig semnal_cuantizat player;
    semnal_fe        = 8000;
    semnal_cuantizat = [];
    player           = [];
    function genereazaSemnal(~, ~)
        sig  = signalType.Value;
        bits = str2double(bitsInput.String);
        lege = lawSelector.Value;
        if isnan(bits) || bits <= 0
            errordlg('Numărul de biți trebuie să fie un număr pozitiv.',
'Eroare');
            return;
        end
        durata = 0.1;
        t      = 0 : 1 / semnal_fe : durata - 1 / semnal_fe;
        switch sig
            case 1
                semnal = sin(2 * pi * 200 * t);
            case 2
                semnal = square(2 * pi * 200 * t);
            case 3
                semnal = sawtooth(2 * pi * 200 * t, 0.5);
            case 4
                semnal = sawtooth(2 * pi * 200 * t);
            case 5
                [filename, pathname] = uigetfile( {'*.wav;*.mp3;*.ogg', 'Fișiere
audio (*.wav, *.mp3, *.ogg)'}, ...
                                                  'Selectează un fișier audio' );
                if isequal(filename, 0)
                    return;
                end
                cale_completa = fullfile(pathname, filename);
                try
                    [y, semnal_fe_temp] = audioread(cale_completa);
                    semnal_fe        = semnal_fe_temp;
                    if size(y, 2) > 1
                        y = mean(y, 2);
                    end
                    lungime_max = semnal_fe * 2;
                    if length(y) > lungime_max
                        y = y(1 : lungime_max);
                    end
                    semnal = y';
                    t = 0 : 1 / semnal_fe : (length(semnal) - 1) / semnal_fe;
                catch ME
                    errordlg(['Eroare la citirea fișierului audio: ',
ME.message], 'Eroare');
```

```
                    return;
                end
            end
            V = max(abs(semnal));
            semnal_cuantizat = zeros(size(semnal));
            if lege == 1
                tabela = [
                    0, 31, 0, "11111111";
                    32, 95, 32 + 16, "11101111";
                    96, 223, 96 + 64, "11011111";
                    224, 479, 224 + 128, "11001111";
                    480, 991, 480 + 256, "10111111";
                    992, 2015, 992 + 512, "10101111";
                    2016, 4063, 2016 + 1024, "10011111";
                    4064, 8191, 4064 + 2048, "100000000"
                ];
            else
                tabela = [
                    0, 15, 0 + 8, "01111111";
                    16, 31, 16 + 8, "10000000";
                    32, 63, 32 + 16, "10010000";
                    64, 127, 64 + 32, "10100000";
                    128, 255, 128 + 64, "10110000";
                    256, 511, 256 + 128, "11000000";
                    512, 1023, 512 + 768, "11010000";
                    1024, 2047, 1024 + 512, "11100000";
                    2048, 4095, 2048 + 1024, "11110000";
                    4096, 8191, 4096 + 2048, "11110000"
                ];
            end
            tabela_valori = double(tabela(:, 1 : 3));
            num_levels       = 2^bits;
            for i = 1 : length(semnal)
                valoare_absoluta_normalizata = abs(semnal(i)) * 4096 / V;
                semn                         = sign(semnal(i));
                cod_gasit                    = "";
                valoare_cuantizata_curenta   = 0;
                for j = 1 : size(tabela, 1)
                    limita_inferioara = tabela_valori(j, 1);
                    limita_superioara = tabela_valori(j, 2);
                    if valoare_absoluta_normalizata >= limita_inferioara &&
valoare_absoluta_normalizata <= limita_superioara
                        index_nivel = round((num_levels - 1) *
(valoare_absoluta_normalizata / 4096));
                        valoare_reprezentativa = tabela_valori(j, 3);
                        valoare_cuantizata_curenta = (index_nivel / (num_levels -
1)) * 4096 * (valoare_reprezentativa / 4096);
                        cod_gasit                    = tabela(j, 4);
                        break;
                    end
                end
                semnal_cuantizat(i) = semn * valoare_cuantizata_curenta * V / 4096;
```

```
        if lege == 2
            cod_inversat = cod_gasit;
            for k = 2 : 2 : length(cod_inversat)
                if cod_inversat(k) == '0'
                    cod_inversat(k) = '1';
                else
                    cod_inversat(k) = '0';
                end
            end
        else
            cod_inversat = cod_gasit;
        end
    end
    err      = semnal - semnal_cuantizat;
    P_signal = rms(semnal)^2;
    P_err    = rms(err)^2;
    if P_err > 0
        snr_val = 10 * log10(P_signal / P_err);
    else
        snr_val = Inf;
    end
    rata_biti = bits * semnal_fe;
    axes(ax1);
    plot( t, semnal, ...
          'LineWidth', lineWidth, ...
          'Color', textColor );
    title( ax1, 'Semnal Original', ...
           'Color', textColor, ...
           'FontSize', fontSize );
    xlabel( ax1, 'Timp (s)', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    ylabel( ax1, 'Amplitudine', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    grid( ax1, 'on' );
    set( ax1, 'GridColor', textColor );
    axes(ax2);
    stairs( t, semnal_cuantizat, ...
            'LineWidth', lineWidth, ...
            'Color', textColor );
    title( ax2, 'Semnal Cuantizat (Neuniform)', ...
           'Color', textColor, ...
           'FontSize', fontSize );
    xlabel( ax2, 'Timp (s)', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    ylabel( ax2, 'Amplitudine', ...
            'Color', textColor, ...
            'FontSize', fontSize );
    grid( ax2, 'on' );
    set( ax2, 'GridColor', textColor );
```

```
        axes(ax3);
        plot( t, err, ...
              'LineWidth', lineWidth, ...
              'Color', textColor );
        title( ax3, 'Eroare Cuantizare', ...
               'Color', textColor, ...
               'FontSize', fontSize );
        xlabel( ax3, 'Timp (s)', ...
                'Color', textColor, ...
                'FontSize', fontSize );
        ylabel( ax3, 'Amplitudine', ...
                'Color', textColor, ...
                'FontSize', fontSize );
        grid( ax3, 'on' );
        set( ax3, 'GridColor', textColor );
        axes(ax4);
        bar( snr_val, ...
             'FaceColor', buttonColor );
        title( ax4, ['SNR: ', num2str(snr_val, '%.2f'), ' dB'], ...
               'Color', textColor, ...
               'FontSize', fontSize );
        ylabel( ax4, 'SNR (dB)', ...
                'Color', textColor, ...
                'FontSize', fontSize );
        grid( ax4, 'on' );
        set( ax4, 'GridColor', textColor );
        set( rmsErrorText, 'String', num2str(rms(err), '%.6f') );
        set( snrText, 'String', num2str(snr_val, '%.2f') );
        set( bitRateText, 'String', num2str(rata_biti, '%.0f') );
        semnal_orig     = semnal;
        semnal_cuantizat = semnal_cuantizat;
    end
    function playStop(~, ~)
        if isempty(semnal_cuantizat)
            return;
        end
        if isempty(player) || strcmp(player.Running, 'off')
            player = audioplayer(semnal_cuantizat, semnal_fe);
            play(player);
        else
            stop(player);
        end
    end
    function reversed_bits = reverse_bits(bits)
        reversed_bits = fliplr(bits);
    end
end
```